

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Отчёт о выполнении лабораторных работ
по дисциплине:

Защита информации

Студент: _____

Салимли Айзек Мухтар Оглы

Преподаватель: _____

Силиненко Александр Витальевич

«_____» _____ 20__ г.

Санкт-Петербург, 2026

РЕФЕРАТ

Объем отчета: 134 с., 6 табл., 7 источн., 4 прил., 75 рис.

Ключевые слова: Защита информации, банк угроз, базовые вектора уязвимостей, дискреционная модель, мандатная модель, межсетевой экран, клиент-серверное приложение, симметричное шифрование, асимметричное шифрование, хэш-функция, сессионный ключ, аутентификация, авторизация.

Отчет содержит информацию о проделанной работе по дисциплине «защита информации», которая включала в себя N лабораторных работ.

- **Практическая работа №1** - Банк угроз ФСТЭК: изучение структуры разделов «Угрозы» и «Уязвимости», поиск уязвимостей по заданным критериям;
- **Практическая работа №2** - Аутентификация и авторизация: разработка системы доступа к конфиденциальным данным, утилиты управления учётными записями и программы brute force для исследования стойкости паролей;
- **Практическая работа №3** - DAC и MAC: реализация дискреционной и мандатной моделей доступа на базе системы аутентификации из работы №2 (владение, ACL, метки безопасности, модель Белла-Лападулы);
- **Практическая работа №4** - Межсетевое экранирование: локальное взаимодействие через loopback интерфейс, разрешение взаимодействия с DNS-сервером, утилита ping для проверки, разрешение по протоколам HTTP/HTTPS, блокировка пакетов неудовлетворяющих условиям;
- **Практическая работа №5** - Разработка клиент-серверного приложения: обеспечение конфиденциального обмена сообщениями между клиентами и сервером, использование симметричного шифрования и обеспечение целостности данных с использованием хэш-функций.

Термины и определения

В отчете применяют следующие термины с соответствующими определениями:

Таблица 1: Термины и определения

Термин	Определение
Информация	Сведения о лицах, предметах, фактах, событиях, явлениях и процессах независимо от формы их представления.
Свойства Безопасности информации	Конфиденциальность, Целостность, Доступность.
Угроза	Потенциально возможное событие, действие или фактор (внешний или внутренний), способный нарушить конфиденциальность, целостность или доступность данных, нанеся ущерб информационным системам или пользователям.
Риск	Критерий угрозы - вероятность реализации угрозы, использующей уязвимости информационных активов, что приводит к ущербу: нарушению конфиденциальности, целостности или доступности данных.
Банк угроз	официальный общедоступный ресурс, содержащий систематизированный перечень уязвимостей, угроз, способов их реализации и объектов воздействия в информационных системах[1].
ФСТЭК	Федеральная служба по техническому и экспортному контролю.
Программное обеспечение	Совокупность программ, процедур, правил, документации и данных, необходимых для функционирования систем обработки информации.
Операционная система	Совокупность программ, обеспечивающих управление ресурсами ЭВМ и взаимодействие пользователя с машиной.
CWE (Common Weakness Enumeration)	Международная система классификации недостатков, дефектов и уязвимостей в программном и аппаратном обеспечении
Наличие эксплойта	Специальная программа, фрагмент кода или последовательность команд, использующие уязвимости (ошибки, слабые места) в программном обеспечении (ПО) или операционных системах для проведения атаки.
CVSS 2.0 (Common Vulnerability Scoring System, v2.0)	Вторая версия открытого стандарта для оценки серьезности уязвимостей ИТ-систем
CVSS 3.0 (Common Vulnerability Scoring System, v3.0)	Третья версия открытого стандарта для оценки серьезности уязвимостей ИТ-систем
Аутентификация	Процесс установления подлинности пользователя или устройства с помощью пароля, ключа или другого идентификатора.

Продолжение на следующей странице

Таблица 1: Термины и определения (продолжение)

Термин	Определение
Авторизация	Процесс установления доступа пользователя к ресурсам системы с помощью аутентификации.
Хэширование	Процесс преобразования данных в хеш-значение.
Слабый пароль	Пароль, который легко угадать или подобрать.
Дискреционная модель	Модель, в которой доступ к ресурсам определяется правами пользователя.
Мандатная модель	Модель, в которой доступ к ресурсам определяется уровнем доступа пользователя.
Уровень доступа	Уровень, определяющий права пользователя на доступ к ресурсам системы.
Уровень конфиденциальности	Уровень, определяющий степень секретности информации.
MAC	Mandatory Access Control.
DAC	Discretionary Access Control.
DNS	Domain Name System - система преобразования доменных имён в IP-адреса.
HTTP	HyperText Transfer Protocol - протокол прикладного уровня передачи данных в сети (порт 80).
HTTPS	HTTP Secure - защищённая версия HTTP с шифрованием TLS/SSL (порт 443).
ICMP	Internet Control Message Protocol - протокол сетевого уровня для служебных сообщений (ping, уведомления об ошибках).
TCP	Transmission Control Protocol - транспортный протокол с установлением соединения и надёжной доставкой.
UDP	User Datagram Protocol - транспортный протокол без установления соединения.
Мэжсетевой экран (МЭ)	Программное или программно-аппаратное средство фильтрации сетевого трафика между сетями или сегментами.
tcpdump	Утилита для перехвата и отображения сетевых пакетов, проходящих через сетевые интерфейсы.
iptables	Механизм ядра Linux для фильтрации пакетов, NAT и управления сетевым трафиком.
looback	Виртуальный сетевой интерфейс для локального обмена данными (адрес 127.0.0.1).
anchors	Якоря в pf - отдельные файлы с правилами, подключаемые к основной конфигурации.
Симметричное шифрование	Шифрование, в котором используется один и тот же ключ для шифрования и расшифрования данных.
Асимметричное шифрование	Шифрование, в котором используются два разных ключа для шифрования и расшифрования данных.
Клиент-серверное приложение	Приложение, в котором клиент и сервер взаимодействуют друг с другом.
Раунд	В алгоритме шифрования ChaCha20 - это одна итерация из всех 20 циклов.

Содержание

Термины и определения	3
Введение	7
1 Анализ уязвимостей программного обеспечения	8
1.1 Введение	8
1.2 Раздел «Угрозы»	8
1.3 Раздел «Уязвимости»	9
1.4 Проверка собственной операционной системы	11
1.4.1 Описание аппаратного обеспечения	12
1.4.2 Поиск уязвимостей ОС ноутбука	13
1.4.3 Поиск уязвимостей ОС смартфона	18
1.5 Меры для устранения найденных уязвимостей	20
1.5.1 macOS	21
1.5.2 iOS	21
Заключение лабораторной работы №1	21
2 Разработка и исследование системы аутентификации и авторизации	22
2.1 Введение	22
2.2 Постановка задачи	22
2.2.1 Цель	22
2.2.2 Задачи	22
2.3 Требования к разрабатываемой системе	22
2.3.1 Операционная система	22
2.3.2 Структура каталогов	22
2.3.3 Утилита управления аутентификацией и авторизацией	23
2.3.4 Утилита доступа к конфиденциальным данным	23
2.3.5 Программа взлома паролей (brute force)	23
2.3.6 Общие требования к программам	24
2.3.7 Требования к исследованию	24
2.4 Сведение об аппаратном и программном обеспечении	24
2.5 Сведение об разработке	24
2.5.1 Принцип работы SHA256	25
2.5.2 Принцип работы разработанной программы	25
2.5.3 Реализация выдачи доступа	26
2.5.4 Реализация атаки brute force	26
2.5.5 Реализация управления пользователями	27
2.6 Примеры работы программы	28
2.7 Атака brute force	30
Заключение	31
3 Реализация моделей дискреционного и мандатного управления доступом	32
3.1 Введение	32
3.2 Постановка задачи	32
3.2.1 Цели	32
3.2.2 Задачи	32
3.3 Требования к работе	32
3.4 Сведения о DAC и MAC	33
3.4.1 DAC (Discretionary Access Control)	33
3.4.2 MAC (Mandatory Access Control)	33
3.5 Реализация DAC	33
3.6 Реализация MAC	34

3.7	Примеры работы программы	34
3.8	Атака brute force	39
	Заключение	39
4	Мэжсетевое экранирование	41
4.1	Введение	41
4.2	Постановка задачи	41
4.2.1	Цели	41
4.2.2	Задачи	41
4.3	Проектирование системы	42
4.4	Принцип обработки пакетов в pf	43
4.5	Реализация	45
4.5.1	Правила фильтрации	45
4.5.2	Пример добавления правила	46
4.6	Проверки реализованной политики	47
4.7	Пример контроля трафика	51
4.8	Дополнительные аргументы pfctl	52
	Заключение	53
5	Разработка клиент-серверного приложения для конфиденциального обмена сообщениями	54
5.1	Введение	54
5.2	Постановка задачи	54
5.2.1	Цели	54
5.2.2	Задачи	54
5.3	Алгоритм ChaCha20	54
5.4	Хэш-функция Blake2b	55
5.4.1	Добавление соли	57
5.5	Алгоритм Диффи-Хеллмана для генерации сессионного ключа	57
5.6	Дополнительная реализация	58
5.6.1	Структура приложения	58
5.7	Примеры работы приложения	59
5.7.1	Без шифрования	59
5.7.2	С симметричным шифрованием	60
5.7.3	Проверка целостности данных	60
5.7.4	Проверка сессионного ключа	61
5.7.5	Проверка хеша	62
5.7.6	Общение между клиентами и группами клиентов	62
	Заключение	64
	Заключение	65
	Список литературы	66
	Приложение А	67
	Приложение Б	83
	Приложение В	116
	Приложение Г	120

Введение

Защита информации представляет собой совокупность мер, направленных на обеспечение трёх основных свойств информационных систем: конфиденциальности (ограничение доступа к данным только для субъектов, имеющих соответствующие права), целостности (сохранение данных в неизменном виде и защита от несанкционированной модификации) и доступности (обеспечение возможности использования информации и ресурсов для уполномоченных пользователей при необходимости). В настоящем отчёте рассматриваются и реализуются методы и механизмы, позволяющие достичь указанных свойств в практике построения защищённых систем.

Отчет содержит информацию о проделанной работе по дисциплине «Защита информации», которая включала в себя N лабораторных работ.

В практической работе №1 был изучен Банк данных угроз безопасности информации ФСТЭК РФ. Изучена структура раздела «Угрозы» и раздела «Уязвимости» Банка угроз. Выполнен поиск уязвимостей по заданным критериям для операционной системы ноутбука (macOS) и смартфона (iOS). Проанализированы базовые векторы CVSS у найденных уязвимостей и описаны меры для их устранения.

В практической работе №2 разработана система доступа пользователей к конфиденциальным данным: выделен каталог для хранения файлов, реализована утилита управления учётными данными (пароли, права доступа), утилита доступа к конфиденциальным данным с аутентификацией и авторизацией. Разработана программа взлома паролей методом грубой силы (brute force) и исследована стойкость паролей в зависимости от длины и алгоритма хэширования SHA256.

В практической работе №3 на базе системы аутентификации из работы №2 реализованы модели дискреционного (DAC) и мандатного (MAC) управления доступом. Для DAC: владение объектами, произвольное назначение прав через команды grant/revoke, контроль на основе списка доступа (ACL). Для MAC: метки безопасности для субъектов и объектов (несекретно, ДСП, секретно), свойства модели Белла-Лападулы («нет чтения сверху», «нет записи вниз»).

В практической работе №4 изучен межсетевой экран pf в macOS (обработка пакетов, правила, pfctl). Реализована политика доступа: разрешены loopback, DNS, исходящий ping, входящий ping с указанного IP, HTTP/HTTPS; остальной трафик блокируется. Правила задают IP источника/приёмника, протокол, порты (TCP/UDP) и тип ICMP. Проверка выполнена через ping, nslookup, curl и tcpdump.

В практической работе №5 разработано клиент-серверное приложение для конфиденциального обмена сообщениями между клиентами и сервером. Использовано симметричное шифрование алгоритмом ChaCha20 и обеспечение целостности данных с использованием хэш-функции Blake2b. Так же реализовано общение между клиентами и группами клиентов. Реализован алгоритм сессионного ключа. В хэш добавляется соль, так же реализована аутентификация пользователей.

1 Анализ уязвимостей программного обеспечения

1.1 Введение

Анализ уязвимостей программного обеспечения (далее ПО) - процесс по выявлению недостатков ПО, которые могут привести к нарушению безопасности информационной системы (см. Таб. 1).

Уязвимость представляет собой «слабое» место в ПО, позволяющее злоумышленнику осуществить несанкционированный доступ, атаку или получить контроль над информацией. Анализ уязвимостей направлен на предотвращение потенциальных угроз и устранение существующих проблем безопасности.

Целью лабораторной работы №1 - является получение навыков работы с банком угроз Федеральной службы по техническому и экспортному контролю (ФСТЭК России).

Банк угроз[1] содержит информацию об уязвимостях ПО и методах их эксплуатации. В рамках работы необходимо изучить структуру Банка угроз и освоить поиск уязвимостей, для собственной операционной системы компьютера и операционной системы смартфона.

1.2 Раздел «Угрозы»

Раздел «Угрозы»[1] содержит информацию об аспектах, связанных с угрозами безопасности информации. В нем представлена классификация угроз по их природе, воздействию и потенциальным последствиям. Каждая угроза сопровождается подробным описанием методов атаки, целей и уязвимостей, используемых злоумышленниками. Для оценки степени серьезности воздействия предусмотрен параметр уровня опасности. Также раздел включает рекомендации по защите, позволяющие определить меры для предотвращения или минимизации угрозы[1].

В разделе можно фильтровать данные по:

- Контекстному поиску;
- По источнику угрозы;
- Последствиям реализации угрозы которые включают в себя пункты:
 - Нарушение конфиденциальности;
 - Нарушение целостности;
 - Нарушение доступности.

Ниже на рисунке 1, представлен снимок экрана, раздела угроз:

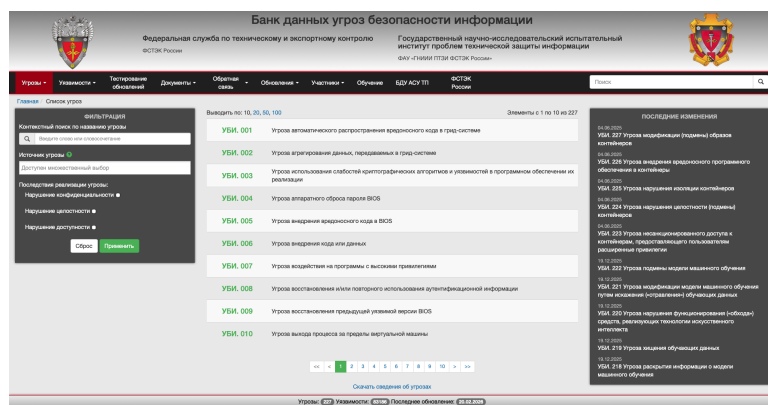


Рис. 1: Раздел угроз

1.3 Раздел «Уязвимости»

Раздел «Список уязвимостей»[1], представляет перечень потенциальных недостатков или уязвимых точек в ПО, аппаратуре или в информационных системах которые могут быть использованы злоумышленниками для нарушения свойств безопасности информации (см. Таб. 1).

Основным отличием уязвимости и угрозы - в том, что уязвимость это недостаток ПО (или системы), а угроза - это риск того что уязвимостью воспользуются.

То есть уязвимость существует сама по себе, а угроза существует условно.

Уязвимости делятся по определенным критериям:

- Классификация уязвимости - отнесение уязвимости к конкретному классу, например в ФСТЭК классы делятся на: уязвимость кода, уязвимость архитектуры и многофакторная;
- Год добавления - дата, когда уязвимость была найдена и добавлена в банк угроз;
- Уровень опасности - делиться на 4 типа: Низкий, Средний, Высокий, Критический;
- Идентификатор типа ошибки - идентификатор, установленный в соответствии с общим перечнем ошибок CWE (см. Таб. 1);
- Наличие эксплойта (см. Таб. 1) - Может существовать, быть в открытом доступе, не быть или уточняться;
- Способ эксплуатации - какую именно угрозу несет уязвимость (например SQL-инъекция);
- Способ устранения - может быть обновление ПО или организационные меры;
- Операционная система - операционная система, под управлением которой функционирует программное обеспечение с обнаруженной уязвимостью.

Наиболее опасная уязвимость - в ФСТЭК является возможность злоумышленника выполнять произвольные команды.

Это можно увидеть, воспользовавшись сортировкой угроз (см. Рис. 2) по уровню опасности (по критичности) описаной ранее:

Выводить по: 10, 20, 50, 100	Сортировка: ▼	Элементы с 1 по 10 из 83186
BDU:2026-01979	Уязвимость по идентификатору по выявлению по критичности	многомного интерфейса api.values.get микропрограммного defstream GXP, позволяющая нарушителю выполнить 18.02.2026
BDU:2026-02014	Уязвимость компонента libvpx	браузеров Mozilla Firefox, Firefox ESR и почтового клиента Thunderbird, позволяющая нарушителю вызвать отказ в обслуживании 16.02.2026
BDU:2026-02017	Уязвимость функции multi_ssld файла /cgi-bin/wireless.cgi	микропрограммного обеспечения маршрутизаторов Wavlink WL-WN579A3, позволяющая нарушителю выполнить произвольные команды 15.02.2026
BDU:2026-02019	Уязвимость функции Delete_Mac_list файла /cgi-bin/wireless.cgi	микропрограммного обеспечения маршрутизаторов Wavlink WL-WN579A3, позволяющая нарушителю выполнить произвольные команды 15.02.2026
BDU:2026-02018	Уязвимость файла /cgi-bin/login.cgi	микропрограммного обеспечения маршрутизаторов Wavlink WL-WN579A3, позволяющая нарушителю выполнить произвольные команды 15.02.2026
BDU:2026-01950	Уязвимость функций Introspector.getBeanInfo() и PropertyDescriptor.getReadMethod()	библиотеки рендеринга шаблонов Jinjava, позволяющая нарушителю выполнить произвольный код 03.02.2026

Рис. 2: Раздел угроз

Для визуального восприятия уязвимости так же помечаются цветом в соответствии с уровнем опасности:

- Низкий уровень опасности
- Средний уровень опасности
- Высокий уровень опасности

- **Критический уровень опасности**

Так же в разделе «Уязвимости», есть подраздел «Инфографика», который позволяет визуально оценить следующие данные[1]:

- Количество критических уязвимостей в ПО различных производителей (см. Рис 3)
- Распределение уязвимостей по уровням опасности (см. Рис 4)
- Распределение уязвимостей по типам ПО (см. Рис 5)
- Количество уязвимостей в ПО различных производителей (см. Рис 6)
- Распределение уязвимостей по типам ошибок (см. Рис 7)
- Количество уязвимостей программного обеспечения различных производителей, связанных с инцидентами информационной безопасности (см. Рис 8)

Ниже перечислены все инфографики описанные выше:

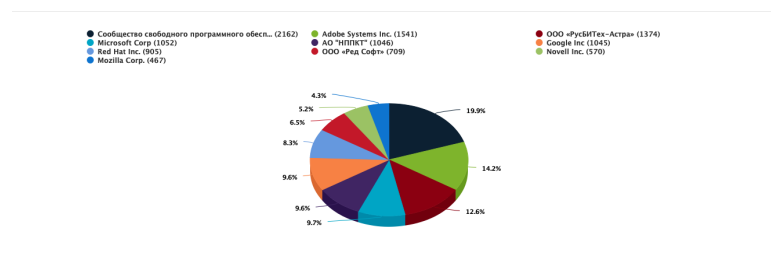


Рис. 3: Количество критических уязвимостей в ПО различных производителей

Как видно из гистограммы 3, количество критических уязвимостей больше наблюдается у сообщества свободного программного обеспечения. Вероятно это связано с тем, что это не большие компании, где нет отдельных специалистов обеспечивающих должную защиту информации.

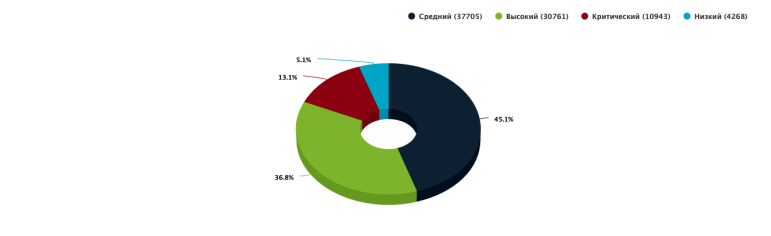


Рис. 4: Распределение уязвимостей по уровням опасности

Из рисунка 4, видно что больше всего уязвимостей имеют средний уровень опасности.

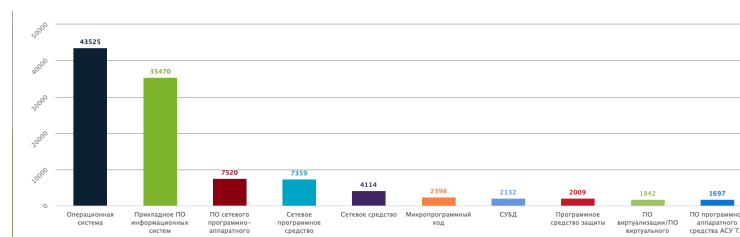


Рис. 5: Распределение уязвимостей по типам ПО

Наиболее уязвимый тип ПО - является операционная система (см. Рис. 5), что обусловлено тем, что ОС - довольно объемное и сложно программное обеспечение.

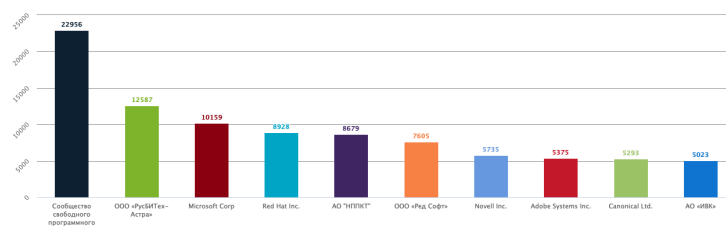


Рис. 6: Количество уязвимостей в ПО различных производителей

Так же как и в случае с критическими уязвимостями (см. Рис. 3), общее количество уязвимостей больше у сообщества свободного программного обеспечения (см. Рис. 6).

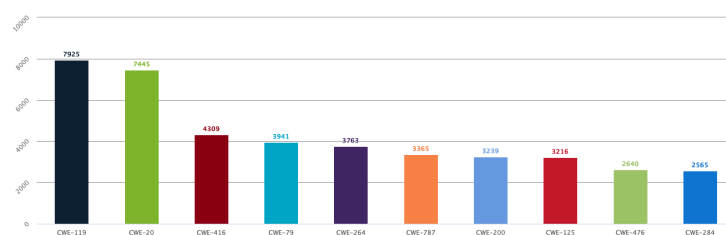


Рис. 7: Распределение уязвимостей по типам ошибок

Наиболее частая уязвимость по типу ошибки - CWE-119 (см. Рис. 7). CWE-119 - это ошибка связанная с неправильным ограничением операций в границах буфера памяти.

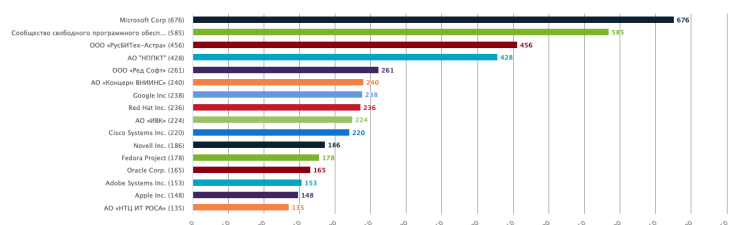


Рис. 8: Количество уязвимостей ПО производителей, связанных с инцидентами ИБ

Из производителей, с наиболее частыми инцидентами в сфере ИБ является Microsoft Corp. На втором месте сообщество свободного программного обеспечения.

Из крупных производителей стоит так же отметить Google, который расположился на 7-ом месте, Apple inc. расположена на 15 месте.

В списке так же присутствуют компании занимающиеся в основном разработкой дистрибутивов Linux. (См. Рис. 8).

1.4 Проверка собственной операционной системы

В лабораторной работе №1, так же необходимо выполнить анализ уязвимостей операционной системы используемого ноутбука и смартфона.

1.4.1 Описание аппаратного обеспечения

В качестве используемых аппаратных средств:

1. Ноутбук: Apple MacBook Pro M3 Pro 18Gb/512Gb:

- Имя ОС: macOS Tahoe;
- Версионный индекс: 26.3 (25D125);
- Статус релиза: Стабильная;
- Ядро: Darwin Kernel Version 25.3.0;
- Архитектура: ARM64;
- Архитектура памяти: UMA.

2. Смартфон: Apple iPhone 12 64Gb/4Gb:

- Имя ОС: iOS;
- Версионный индекс: 26.3 (23D127);
- Статус релиза: Стабильная.

Более подробная информация представлена на рисунке 9, для ноутбука и 10, для смартфона:

```
(base) monadayzek@Ayzeks-MBP ~ % screenfetch
```

```
-/+/:.  
:++++.  
/+++/.  
.:--:-./+/-~`:::-  
.://+++++/::://+++++/::~  
.:////////////////////////:  
/////////////////////////  
-+++++++`  
/+++++++/  
/SSSSSSSSSSSSSSSSSSSSS.  
:SSSSSSSSSSSSSSSSSSSSS-  
oSSSSSSSSSSSSSSSSSSSSSo`  
`syyyyyyyyyyyyyyyyyyy+y+`  
`oSSSSSSSSSSSSSSSSSSSSS/  
:oooooooooooooooooooooo+.  
: +oo+/-.-.-:/+o+/-
```

```
monadayzek@Ayzeks-MBP.HomeLAN  
OS: 64bit macOS  
Kernel: arm64 Darwin 25.3.0  
Uptime: 5d 15h 36m  
Packages: 130  
Shell: zsh 5.9  
Resolution: 3024x1964, 1920x1080  
DE: Aqua  
WM: Quartz Composer  
WM Theme: Blue (Dark)  
Font: AndaleMono  
Disk: 127G / 494G (28%)  
CPU: Apple M3 Pro  
GPU: Apple M3 Pro  
RAM: 2281MiB / 18432MiB
```

Рис. 9: Информация об операционной системе (ноутбук)

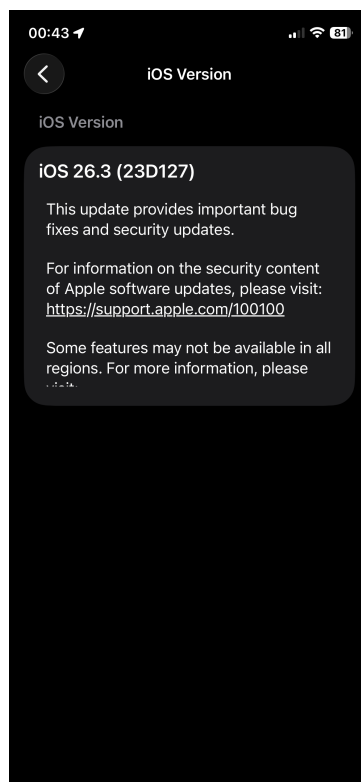


Рис. 10: Информация об операционной системе (смартфон)

1.4.2 Поиск уязвимостей ОС ноутбука

Для поиска уязвимостей операционной системы ноутбука в Банке угроз[1], необходимо указать в поисковике следующую информацию:

- Производитель ПО: Apple;
- Тип ПО: Операционная система;
- Программное обеспечение: macOS;
- Аппаратная платформа: ARM64;
- Операционная система: MacOS Tahoe до 26.2 (крайняя версия ФСТЭК);
- Уровень опасности*: Критический.

* - По поставленной задаче лабораторной работы №1, требуется выявить **критические** уязвимости ПО. Если **критические** уязвимости не найдены, ищутся уязвимости **высокой** опасности, если нет **высокой** опасности, **средние** иначе **низкие**.

Как видно из рисунка 11, выявлено лишь три уязвимостей, две из которых **среднего** уровня, и одна **низкого** уровня.

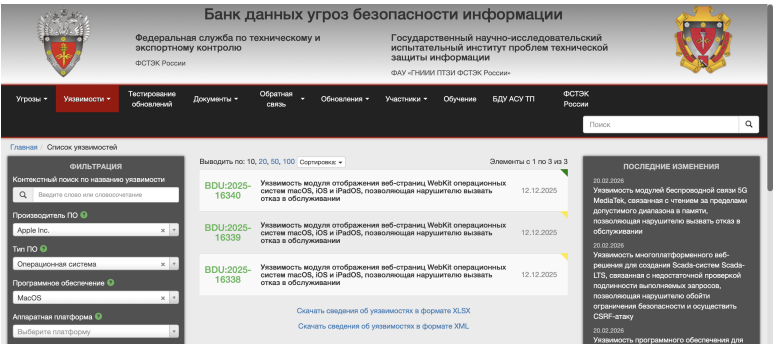


Рис. 11: Уязвимости macOS 26.2

Все уязвимости связаны с отображением страниц WebKit причем модули уязвимы как для macOS так и для iOS и iPadOS. Уязвимость позволяет злоумышленнику вызвать отказ в обслуживании.

Более подробное описание двух средних уязвимостей представлено на рисунке 13, и 12, для низкого уровня на рисунке 14.

Описание уязвимости					
Уязвимость модуля отображения веб-страниц WebKit операционных систем macOS, iOS и iPadOS связана с использованием памяти после ее освобождения. Эксплуатация уязвимости может позволить нарушителю, действующему удаленно, вызвать отказ в обслуживании.					
Уязвимое ПО	Вендор	Наименование ПО	Версия ПО	Тип ПО	Архитектура (Платформа)
	Apple Inc.	macOS	Tahoe до 26.2	Операционная система	Не указана
Операционные системы и аппаратные платформы	Apple Inc. macOS Tahoe до 26.2 Apple Inc. iOS до 18.0 Apple Inc. iOS до 18.7.3 Apple Inc. iPadOS до 18.7.3 Apple Inc. iPadOS до 26.2				
Тип ошибки	Использование после освобождения [CWE-416]				
Класс уязвимости	Уязвимость кода				
Дата выявления	12.12.2025				
Базовый вектор уязвимости	CVSS 2.0: AV/N/A/CU/L/NC/N/A/P CVSS 3.0: AV/N/A/CU/PR/NU/RS/LUC/NT/NA/L CVSS 4.0: Вектор не задан				
Уровень опасности уязвимости	Средний уровень опасности (Базовая оценка CVSS 2.0 составляет 5) Средний уровень опасности (Базовая оценка CVSS 3.1 составляет 4,9)				
Возможные меры по устранению уязвимости	Использование рекомендаций: https://support.apple.com/en-us/125884 https://support.apple.com/en-us/125885 https://support.apple.com/en-us/125886 https://support.apple.com/en-us/125892				
Статус уязвимости	Подтверждена производителем				
Наличие эксплойта	Данные уточняются				
Способ эксплуатации	Манипулирование структурой данных				
Способ устранения	Обновление программного обеспечения				
Информация об устранении	Уязвимость устранена				
Ссылки на источники	https://support.apple.com/en-us/125884 https://support.apple.com/en-us/125885 https://support.apple.com/en-us/125886 https://support.apple.com/en-us/125892 https://www.cybersecurity-help.eu/vdb/582025/121309				
Идентификаторы других систем описаний уязвимостей	CVE-2025-43538				

Рис. 12: Уязвимость среднего уровня BDU:2025-16339

Описание уязвимости	Уязвимость модуля отображения веб-страниц WebKit операционных систем macOS, iOS и iPadOS связана с использованием памяти после ее освобождения. Эксплуатация уязвимости может позволить нарушителю, действующему удаленно, вызвать отказ в обслуживании				
Уязвимость ПО	▼ раскрыть				
	Вендор	Наименование ПО	Версия ПО	Тип ПО	Архитектура (Платформа)
	Apple Inc.	macOS	Tahoe до 26.2	Операционная система	Не указана
Операционные системы и аппаратные платформы	▼ раскрыть				
	Apple Inc. macOS Tahoe до 26.2 Apple Inc. iOS до 26.2 Apple Inc. iOS до 18.7.3 Apple Inc. iPadOS до 18.7.3 Apple Inc. iPadOS до 26.2				
Тип ошибки	Использование памяти после освобождения [CVE-416]				
Класс уязвимости	Уязвимость кода				
Дата выявления	12.12.2025				
Базовый вектор уязвимости	CVSS 2.0: A/N/A/CU/Au/N/C/N/N/A/P CVSS 3.0: A/N/A/CU/PR/NUI/RS/UC/N/N/A/L CVSS 4.0: Вектор не задан				
Уровень опасности уязвимости	Средний уровень опасности (базовая оценка CVSS 2.0 составляет 5) Средний уровень опасности (базовая оценка CVSS 3.1 составляет 4,3)				
Возможные меры по устранению уязвимости	Использование рекомендаций: https://support.apple.com/ru-ua/125884 https://support.apple.com/ru-ua/125885 https://support.apple.com/ru-ua/125886 https://support.apple.com/ru-ua/125887				
Статус уязвимости	Подтверждена производителем				
Наличие эксплойта	Данные уточняются				
Способ эксплуатации	Манипулирование структурами данных				
Способ устранения	Обновление программного обеспечения				
Информация об устранении	Уязвимость устранена				
Ссылки на источники	▼ раскрыть				
	https://support.apple.com/ru-ua/125884 https://support.apple.com/ru-ua/125885 https://support.apple.com/ru-ua/125886 https://support.apple.com/ru-ua/125887 https://www.cybersecurity-help.cz/vdb/SB2025121309				
Идентификаторы других систем описаний уязвимостей	CVE: CVE-2025-43536				

Рис. 13: Уязвимость среднего уровня BDU:2025-16338

Описание уязвимости	Уязвимость модуля отображения веб-страниц WebKit операционных систем macOS, iOS и iPadOS связана с ошибками синхронизации при использовании общего ресурса. Эксплуатация уязвимости может позволить нарушителю, действующему удаленно, вызвать отказ в обслуживании				
Уязвимость ПО	▼ раскрыть				
	Вендор	Наименование ПО	Версия ПО	Тип ПО	Архитектура (Платформа)
	Apple Inc.	macOS	Tahoe до 26.2	Операционная система	Не указана
Операционные системы и аппаратные платформы	▼ раскрыть				
	Apple Inc. macOS Tahoe до 26.2 Apple Inc. iOS до 26.2 Apple Inc. iOS до 18.7.3 Apple Inc. iPadOS до 18.7.3 Apple Inc. iPadOS до 26.2				
Тип ошибки	Одновременное выполнение с использованием общего ресурса с неграмотной синхронизацией («Ситуация гоним») [CVE-362]				
Класс уязвимости	Уязвимость кода				
Дата выявления	12.12.2025				
Базовый вектор уязвимости	CVSS 2.0: A/N/A/CU/Au/N/C/N/N/A/P CVSS 3.0: A/N/A/CU/PR/NUI/RS/UC/N/N/A/L CVSS 4.0: Вектор не задан				
Уровень опасности уязвимости	Низкий уровень опасности (базовая оценка CVSS 2.0 составляет 2,6) Низкий уровень опасности (базовая оценка CVSS 3.1 составляет 3,1)				
Возможные меры по устранению уязвимости	▼ раскрыть				
	Использование рекомендаций: https://support.apple.com/ru-ua/125884 https://support.apple.com/ru-ua/125885 https://support.apple.com/ru-ua/125886 https://support.apple.com/ru-ua/125887				
Статус уязвимости	Подтверждена производителем				
Наличие эксплойта	Данные уточняются				
Способ эксплуатации	Манипулирование сроками и состоянием				
Способ устранения	Обновление программного обеспечения				
Информация об устранении	Уязвимость устранена				
Ссылки на источники	▼ раскрыть				
	https://support.apple.com/ru-ua/125884 https://support.apple.com/ru-ua/125885 https://support.apple.com/ru-ua/125886 https://support.apple.com/ru-ua/125887 https://support.apple.com/ru-ua/125889				
Идентификаторы других систем описаний уязвимостей	CVE: CVE-2025-43531				

Рис. 14: Уязвимость низкого уровня BDU:2025-16340

Как видно, все три уязвимости относятся к типу уязвимости кода, и неотличимы друг от друга.

Ниже представлены таблица 2, и таблица 3, где перечислены все метрики векторов уязвимостей для CVSS 2.0 и CVSS 3.0 (см. Таб. 1) соответственно, с пояснением:

Таблица 2: Метрики вектора уязвимости CVSS 2.0

Аббревиатура	Расшифровка	Значения и смысл
AV	Access Vector (вектор доступа)	Local (L) - локальный доступ; Adjacent Network (A) - доступ из смежной сети; Network (N) - сетевой доступ. Определяет способ взаимодействия с системой.
AC	Access Complexity (сложность доступа)	High (H) - высокая; Medium (M) - средняя; Low (L) - низкая. Характеризует условия, необходимые для эксплуатации.
Au	Authentication (аутентификация)	Multiple (M) - множественная; Single (S) - единственная; None (N) - не требуется. Указывает на необходимость подтверждения подлинности.
C	Confidentiality Impact (влияние на конфиденциальность)	None (N) - отсутствует; Partial (P) - частичное; Complete (C) - полное. Степень раскрытия информации.
I	Integrity Impact (влияние на целостность)	None (N) - отсутствует; Partial (P) - частичное; Complete (C) - полное. Возможность искажения данных.
A	Availability Impact (влияние на доступность)	None (N) - отсутствует; Partial (P) - частичное; Complete (C) - полное. Степень нарушения доступности сервиса.

Таблица 3: Метрики вектора уязвимости CVSS 3.0

Аббревиатура	Расшифровка	Значения и смысл
AV	Attack Vector (вектор атаки)	Network (N) - сетевой; Adjacent (A) - из смежной сети; Local (L) - локальный; Physical (P) - физический. Отражает удалённость атаки.
AC	Attack Complexity (сложность атаки)	Low (L) - низкая; High (H) - высокая. Наличие условий вне контроля атакующего.
PR	Privileges Required (необходимые привилегии)	None (N) - не требуются; Low (L) - низкие; High (H) - высокие. Уровень прав доступа до атаки.
UI	User Interaction (взаимодействие с пользователем)	None (N) - не требуется; Required (R) - требуется. Участие другого лица в успешной атаке.
S	Scope (область действия)	Unchanged (U) - не изменяется; Changed (C) - изменяется. Возможность воздействия на другие компоненты.
C	Confidentiality Impact (влияние на конфиденциальность)	High (H) - высокое; Low (L) - низкое; None (N) - отсутствует. Степень потери конфиденциальности.
I	Integrity Impact (влияние на целостность)	High (H) - высокое; Low (L) - низкое; None (N) - отсутствует. Возможность модификации данных.
A	Availability Impact (влияние на доступность)	High (H) - высокое; Low (L) - низкое; None (N) - отсутствует. Степень нарушения доступности.

Для более детального анализа, стоит рассмотреть базовый вектор уязвимости каждой уязвимости:

- BDU:2025-16338:
 - CVSS 2.0: AV:N/AC:L/Au:N/C:N/I:N/A:P (Оценка 5)
 - CVSS 3.0: AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:N/A:L (Оценка 4,3)
 - CVSS 4.0: -
- BDU:2025-16339:
 - CVSS 2.0: AV:N/AC:L/Au:N/C:N/I:N/A:P (Оценка 5)
 - CVSS 3.0: AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:N/A:L (Оценка 4,3)
 - CVSS 4.0: -

- BDU:2025-16340:
 - CVSS 2.0: AV:N/AC:H/Au:N/C:N/I:N/A:P (Оценка 2,6)
 - CVSS 3.0: AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:N/A:L (Оценка 3,1)
 - CVSS 4.0: -

Так как вектора идентичны для средних уровней можно расписать их едино:

CVSS 2.0: AV - способ получения доступа: **N** - сетевой/ **AC** - сложность получения доступа: **L** - низкая/ **Au** - аутентификация : **N** - не требуется/ **C** - влияние на конфиденциальность: **N** - не оказывает/ **I** - влияние на целостность: **N** - не оказывает/ **A** - влияние на доступность: **P** - частичное.

Таким образом для CVSS 2.0, можно прочесть уязвимость как - сетевое подключение доступа с низкой сложнастью без аутентификации не влияющая на конфиденциальность не оказывающая влияние на целостность и частично влияющая на доступность.

CVSS 3.0: AV - Вектор атаки: **N** - сетевой/ **AC** - сложность атаки: **L** - низкая/ **PR** - уровень привелегий: **N** - не требуется/ **UI** - взаимодействие с пользователем: **R** - требуется/ **S** - влияние на другие компоненты системы: **U** - не оказывает/ **C** - влияние на конфиденциальность: **N** - не оказывает/ **I** - влияние на целостность: **N** - не оказывает/ **A** - влияние на доступность: **L** - низкое.

Таким образом для CVSS 3.0, можно прочесть уязвимость как - сетевой вектор атаки с низкой сложностью, не требующий уровень привелегий с взаимодействием с пользователем, не влияющая на другие компоненты, не влияющая на конфиденциальность, без оказания влияния на целостность с низким влиянием на доступность.

Вектор для низкого уровня уязвимости отличается лишь пунктом «сложность атаки»(AC), и имеет индекс Н, то есть высокая сложность. Таким образом для низкого уровня уязвимости:

CVSS 2.0: AV - способ получения доступа: **N** - сетевой/ **AC** - сложность получения доступа: **H** - высокая/ **Au** - аутентификация : **N** - не требуется/ **C** - влияние на конфиденциальность: **N** - не оказывает/ **I** - влияние на целостность: **N** - не оказывает/ **A** - влияние на доступность: **P** - частичное.

То есть - сетевое подключение доступа с высокой сложнастью без аутентификации не влияющая на конфиденциальность не оказывающая влияние на целостность и частично влияющая на доступность.

CVSS 3.0: AV - Вектор атаки: **N** - сетевой/ **AC** - сложность атаки: **H** - высокая/ **PR** - уровень привелегий: **N** - не требуется/ **UI** - взаимодействие с пользователем: **R** - требуется/ **S** - влияние на другие компоненты системы: **U** - не оказывает/ **C** - влияние на конфиденциальность: **N** - не оказывает/ **I** - влияние на целостность: **N** - не оказывает/ **A** - влияние на доступность: **L** - низкое.

То есть - сетевой вектор атаки с высокой сложностью, не требующий уровень привелегий с взаимодействием с пользователем, не влияющая на другие компоненты, не влияющая на конфиденциальность, без оказания влияния на целостность с низким влиянием на доступность.

1.4.3 Поиск уязвимостей ОС смартфона

Для поиска уязвимостей операционной системы смартфона в Банке угроз[1], необходимо указать в поисковике следующую информацию:

- Производитель ПО: Apple;
- Тип ПО: Операционная система;

- Программное обеспечение: iOS;
- Версия ПО: iOS до 26.2 (крайняя версия ФСТЭК);
- Уровень опасности: Критический.

На рисунке 15, видно что для операционной системы iOS, существуют уязвимости критического уровня, которых довольно много (108 критических уязвимостей).

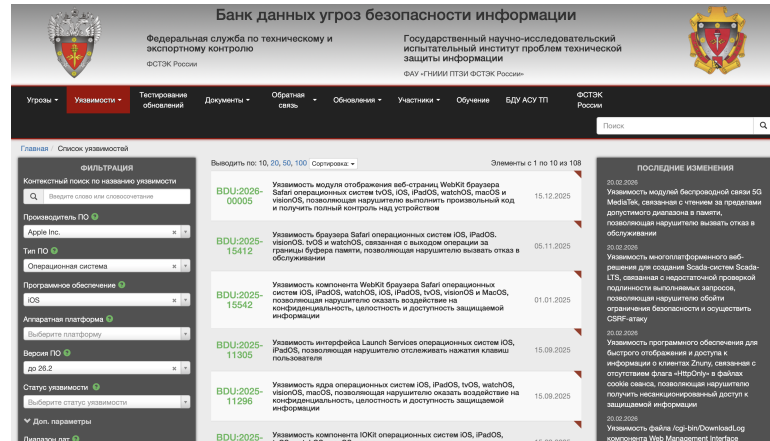


Рис. 15: Критические уязвимости в iOS

Возьмем первые две из отсортированных по критичности:

- BDU:2026-00005;
- BDU:2025-15412.

BDU:2026-00005, позволяет злоумышленнику, выполнить произвольный код и получить полный контроль над устройством, а BDU:2025-15412, связана с выходом операции за границы буфера памяти, которая может позволить злоумышленнику вызвать отказ в обслуживании.

Более подробное описание двух уязвимостей представлены на рисунке 16, и 17.

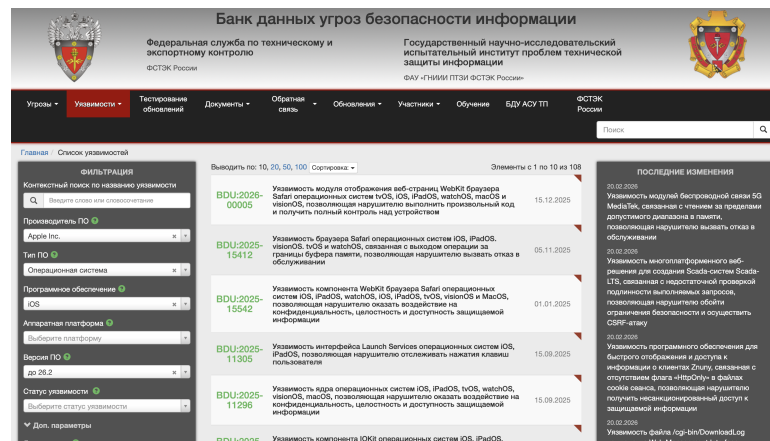


Рис. 16: Уязвимость критического уровня BDU:2026-00005

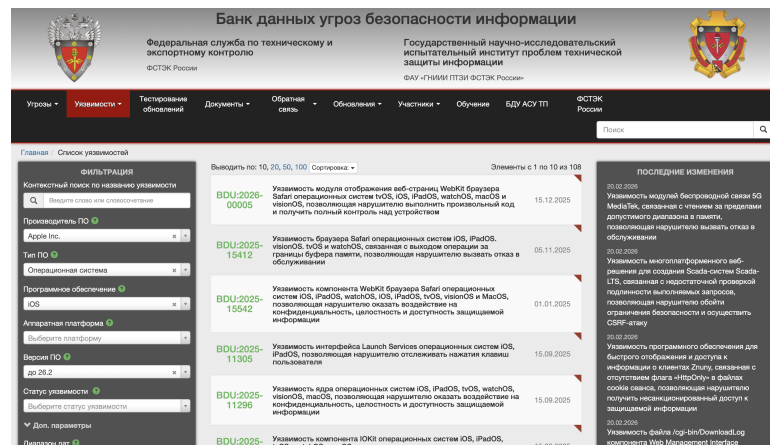


Рис. 17: Уязвимость критического уровня BDU:2025-15412

Так же опишем их базовые вектора уязвимостей:

- BDU:2026-00005:
 - CVSS 2.0: AV:N/AC:L/Au:N/C:C/I:C/A:C
 - CVSS 3.0: AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H
 - CVSS 4.0: -
- BDU:2025-15412:
 - CVSS 2.0: AV:N/AC:L/Au:N/C:C/I:C/A:C
 - CVSS 3.0: AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H
 - CVSS 4.0: -

Вектора полностью идентичны, их можно описать едино:

CVSS 2.0: AV - способ получения доступа: N - сетевой/ AC - сложность получения доступа: L - низкая/ Au - аутентификация: N - не требуется/ C - влияние на конфиденциальность: C - полное/ I - влияние на целостность: C - полное/ A - влияние на доступность: C - полное.

То есть это сетевой способ получения доступа с низкой сложностью, без аутентификации с влиянием на конфиденциальность, целостность и доступность.

CVSS 3.0: AV - вектор атаки: N - сетевой/ AC - сложность атаки: L - низкая/ PR - уровень привелегий: N - не требуется/ UI - взаимодействие с пользователем: R - требуется/ S - влияние на другие компоненты: U - не оказывает/ C - влияние на конфиденциальность: H - высокое/ I - влияние на целостность: H - высокое/ A - влияние на доступность: H - высокое.

То есть сетевая атака с низкой сложностью, без требования привелегий, со взаимодействием с пользователем, не влияющий на другие компоненты, высоко влияющий на конфиденциальность, целостность и доступность.

1.5 Меры для устранения найденных уязвимостей

Ниже описаны меры для устранения найденных уязвимостей для операционных систем macOS и iOS соответственно.

1.5.1 macOS

Среди найденных уязвимостей для macOS 26.2, в разделе «Способ устранения» Банка угроз[1], является обновление программного обеспечения, так как модуль WebKit, для пользователя является не контролируемым модулем (нет прямого доступа).

Такой способ касается всех найденных уязвимостей (BDU:2025-16340, BDU:2025-16338, BDU:2025-16339).

По выходу macOS версии 26.3, будет совершено обновление, во избежания использования злоумышленником данных уязвимостей.

1.5.2 iOS

Способом устранения в Банке угроз[1], для уязвимости BDU:2026-00005 - рекомендуется обновление программного обеспечения. Так же по выходу iOS версии 26.3, будет совершено обновление программного обеспечения.

Для уязвимости BDU:2025-15412, в подробном описании уязвимости в Банке угроз[1], рекомендуется обновление программного обеспечения, так же стоит отметить, что уязвимость (исходя из вектора уязвимостей), требует взаимодействия с пользователем, что требует от пользователя бдительности.

По выходу iOS версии 26.3, будет совершено обновление программного обеспечения, во избежания уязвимостей и возможных угроз.

Заключение

В ходе выполнения лабораторной работы №1, был изучен Банк угроз ФСТЭК[1], в котором были выявлены уязвимости для имеющихся операционных систем ноутбука (macOS) и смартфона (iOS). В ходе исследования уязвимостей, было выявлено, что операционная система macOS, достаточно хорошо защищена, так как были найдены всего две уязвимости среднего и одна низкого уровня. Уязвимости были связаны с модулем WebKit, который мог предоставить частичный доступ к информации.

Так же было выявлено, что iOS, недостаточно защищенная операционная система, были выявлены критические уязвимости, связанные с полным нарушением безопасности информации, а именно влияние на конфиденциальность, целостность и доступность. Одна из уязвимостей (относительно новая), была в том, что злоумышленник мог получить полный доступ к устройству, другая же, уязвимостей была связана с утечкой памяти, что позволяло злоумышленнику вызвать отказ в обслуживании.

Все уязвимости найденные как для macOS так и для iOS, устраняются путем обновления программного обеспечения.

2 Разработка и исследование системы аутентификации и авторизации

2.1 Введение

Одним из ключевых механизмов защиты информации от несанкционированного доступа является аутентификация - процесс установления подлинности субъекта на основе предъявленного идентификатора. В рамках дисциплины «Защита информации» исследование методов аутентификации и авторизации позволяет сформировать фундаментальное понимание принципов разграничения доступа к конфиденциальным данным.

Актуальность работы обусловлена необходимостью анализа стойкости различных методов хэширования паролей в зависимости от их длины и сложности. Проблема слабых паролей остается одной из наиболее распространенных уязвимостей в корпоративных и частных информационных системах. Изучение эффективности атак методом грубой силы (brute force) на пароли, хэшированные с использованием алгоритма SHA256, позволяет оценить зависимость времени взлома от стойкости алгоритма. В данной работе, описана реализация системы защиты утилиты аутентификации и авторизации, и утилиты атаки на пароли.

2.2 Постановка задачи

2.2.1 Цель

- Разработать систему доступа пользователей к конфиденциальным данным;
- Исследовать стойкость паролей к атаке методом грубой силы.

2.2.2 Задачи

- Разработать систему доступа пользователей к конфиденциальным данным, включающую:
 1. выделенный каталог для хранения всех файлов системы;
 2. утилиту для работы с данными аутентификации и авторизации: паролями и правами доступа;
 3. утилиту доступа к конфиденциальным данным, обеспечивающую аутентификацию и авторизацию пользователя при работе с конфиденциальными данными.
- Разработать программу взлома паролей методом грубой силы и исследовать стойкость паролей в зависимости от длины паролей и алгоритма хэширования.

2.3 Требования к разрабатываемой системе

2.3.1 Операционная система

Работа выполняется в ОС macOS с правами суперпользователя (root). Все действия, требующие повышенных привилегий, выполняются через `sudo`.

2.3.2 Структура каталогов

В корне файловой системы создаётся дерево каталогов:

- `/Users/Shared/practice2/etc` - для хранения файла аутентификации и авторизации;
- `/Users/Shared/practice2/confdata` - для конфиденциальных файлов;
- `/Users/Shared/practice2/bin` - для исполняемых утилит;

- /Users/Shared/practice2/log - для файлов регистрации (логов).

Права доступа к указанным каталогам: только пользователь **root** имеет право чтения, записи и выполнения.

2.3.3 Утилита управления аутентификацией и авторизацией

- Файл учётных данных: /Users/Shared/practice2/etc/passwd. Формат строки:

<логин>:<хэш_пароля>:<UID>:<права>:<ФИО>

Права доступа к файлу: чтение и запись только для **root**.

- Возможные права пользователя на конфиденциальные данные: **r** (чтение), **w** (запись), **d** (удаление).
- Функции утилиты:
 - добавление нового пользователя (запрос логина, ФИО, прав, пароля с подтверждением). Пароль хранится в виде хэша согласно индивидуальному заданию;
 - алфавит пароля: латинские буквы (заглавные и строчные), цифры 0-9, спецсимволы !@#\$%^&*(). Первый символ - буква;
 - редактирование данных существующего пользователя (включая смену пароля);
 - удаление пользователя;
 - регистрация всех действий с файлом **passwd**.

2.3.4 Утилита доступа к конфиденциальным данным

- При запуске запрашивает логин и пароль, проверяет их по **passwd**.
- При успехе выводит приветствие с ФИО, краткую справку и приглашение к вводу команд.
- При неверных данных повторяет запрос до получения корректной пары; выход по **Ctrl-C** (SIGINT).
- Поддерживаемые команды:
 - **read** - вывод содержимого файла (требуется право **r**);
 - **append** - добавление данных в существующий файл (право **w**);
 - **copy** - копирование файла в каталог **confdata** или внутри него. Запрещено копирование из **confdata** наружу и перезапись существующих файлов (требуются **r** и **w**);
 - **remove** - удаление файла (право **d**);
 - **exit** - завершение работы;
 - **help** - справка.
- Утилита доступна для запуска всем пользователям ОС.
- Все действия с конфиденциальными данными регистрируются.

2.3.5 Программа взлома паролей (brute force)

- Последовательно вызывает утилиту доступа, перебирая пароли по заданному алфавиту с учётом алгоритма хэширования.

- Фиксирует время перебора и количество итераций до успешного подбора.

2.3.6 Общие требования к программам

- Язык программирования - любой.
- Исходный код должен содержать комментарии.

2.3.7 Требования к исследованию

- Исследуются длины паролей: 3, 4, 5, 6, 7, 8 символов.
- Рассчитывается максимальное число итераций для каждой длины.
- По результатам для длин 3-4 символа оценивается ожидаемое максимальное время взлома для длин 5-8.
- Проводится экспериментальная проверка теоретических оценок.
- Эксперимент прерывается при превышении 8 часов непрерывного перебора.
- Все замеры выполняются на компьютере без активных фоновых задач.

2.4 Сведение об аппаратном и программном обеспечении

Работа выполнена в среде MacOS. MacOS - так же является UNIX-подобной системой, которая имеет стандарт POSIX. Что соответствует требованиям к поставленной задаче.

- Аппаратное обеспечение:
 - Тип - ноутбук;
 - Наименование - Apple MacBook Pro M3 Pro 14';
 - Оперативная память - 18Gb;
 - Память - 512Gb SSD;
 - Архитектура процессора - ARM64;
 - Архитектура памяти - UMA;
 - Конфигурация - 11 CPU/14 GPU.
- Программное обеспечение:
 - Тип - операционная система;
 - Наименование - macOS Tahoe;
 - Версия ПО - 26.3.1;
 - Ядро - Darwin Kernel Version 25.3.0.

Более подробное описание было представлено в предыдущей практической работе на рисунке 9.

2.5 Сведение об разработке

Для реализации поставленной задачи, было принято решение, использовать несколько инструментов:

- Python 3.13.1 (conda enviroment) - для написания скриптов;

- C (clang 17.0.0) - для создания процесса;
- shell - для запуска скрипта.

Используемый алгоритм хэширования - SHA256 из библиотеки Python hashlib.

2.5.1 Принцип работы SHA256

1. Дополнение сообщения: к исходному сообщению добавляются бит '1', нулевые биты и 64-битное представление исходной длины, чтобы итоговая длина стала кратной 512 битам.
2. Разбиение на блоки: сообщение делится на блоки по 512 бит.
3. Инициализация: используются восемь 32-битных констант $H_0^{(0)} \dots H_7^{(0)}$ (первые 32 бита дробных частей квадратных корней первых восьми простых чисел).
4. Раундовое преобразование: каждый блок обрабатывается в 64 раунда с использованием:
 - функций $\text{Ch}(x, y, z)$ и $\text{Maj}(x, y, z)$;
 - циклических сдвигов $\Sigma_0(x)$ и $\Sigma_1(x)$;
 - 64 раундовых констант K_t (первые 32 бита дробных частей кубических корней первых 80 простых чисел).
5. Формирование дайджеста: после обработки всех блоков полученные восемь 32-битных слов конкатенируются в 256-битный хэш.

Из свойств алгоритма:

1. Однонаправленность: вычислительно невозможно восстановить исходное сообщение по хэшу.
2. Устойчивость к коллизиям: сложность нахождения двух сообщений с одинаковым хэшем составляет 2^{128} операций.

2.5.2 Принцип работы разработанной программы

Дерево каталогов выглядит следующим образом и имеет следующие права (см. рисунок 18):

```
drwxr-xr-x@ 9 root  admin  288 10 Mar 20:00 bin
drwx-----@ 3 root  admin   96 10 Mar 20:20 confdata
drwx-----@ 3 root  admin   96 10 Mar 17:56 etc
drwx-----@ 4 root  admin  128 10 Mar 18:40 log
```

Рис. 18: Директории и права

Такие права были установлены так как:

- Логирование может читать только пользователь root (log/*.log);
- Хэшированные пароли, права, логины так же может видеть только root (etc/passwd);
- Исполняемые файлы пользователей (bin/*.sh)
- Исполняемые файлы владельца (bin/user_manager.py, bin/config.py)
- Конфиденциальные данные доступны только авторизированные пользователям или root (confdata/*)

2.5.3 Реализация выдачи доступа

Основная логика выдачи доступа реализована в скрипте `access.py`, а учётные данные о пользователях и их правах хранятся в файле `/Users/practice2/etc/passwd`. Этот файл, а также журналы `/Users/practice2/log/access.log`, недоступны обычному пользователю (права 600, владелец `root`), однако именно из них программа должна читать логины, хэшированные пароли и права, а также записывать события доступа. Поэтому при выдаче доступа нужно было одновременно обеспечить: невозможность прямого чтения/записи этих файлов пользователем и возможность контролируемого чтения/записи из `access.py` без использования `sudo`.

Логика выдачи доступа устроена следующим образом. Обычный пользователь запускает утилиту `run_access`, которая на самом деле является небольшим `setuid`-бинарником на C (`run_access.c`), установленным с владельцем `root` и битом SUID. Ядро запускает этот бинарник с эффективным идентификатором пользователя `EUID=root`, после чего он вызывает `/usr/bin/python3` с аргументом `/Users/practice2/bin/access.py`. Внутри `access.py` реализованы функции `elevate()`, `drop()` и `drop_to_real()`: при работе с файлом `passwd` и журналами выполняется временное повышение `EUID` до владельца файла (`root`), затем сразу же происходит возврат в контекст реального пользователя. Все проверки прав доступа к конфиденциальным файлам в каталоге `confdata/` выполняются уже с `EUID` обычного пользователя.

Скрипт `/Users/practice2/access.py` организует аутентификацию и выдачу прав следующим образом. В начале он импортирует модули `os`, `sys`, `getpass`, `hashlib`, `logging`, `shutil` и параметры из `config.py`. Модуль `os` используется для работы с файловой системой и идентификаторами пользователя: `os.stat()` (получение владельца файла `passwd`), `os.getuid()/os.geteuid()` (реальный и эффективный UID), `os.seteuid()` (временное повышение/понижение прав), `os.makedirs()` и `os.chmod()` (создание каталогов и установка прав доступа). Функции `read_passwd()` и `write_passwd()` оборачивают операции чтения/записи файла `passwd` в вызовы `elevate()/drop()`, а функция `_setup_logging()` аналогично открывает `access.log` под повышенными правами, после чего дальнейшая работа идёт уже от имени реального пользователя. Основной цикл команд (`read`, `append`, `copy`, `remove`, `perm`, `edit my info`) опирается исключительно на данные о правах из `passwd` и не требует `root`-доступа.

Попытка реализовать всё это только на Python оказалась невозможной в рамках требований macOS. Во-первых, установка бита SUID на самом `.py`-файле не даёт эффекта: ядро всё равно запускает интерпретатор `/usr/bin/python3` с правами вызывающего пользователя, а не владельца скрипта. Во-вторых, вызов `os.seteuid(0)` из процесса без привилегий запрещён - Python не может «сам по себе» временно выдать себе `EUID` владельца файла. Поэтому была выбрана следующая схема: небольшой статически проверяемый бинарник на C, которому операционная система доверяет бит SUID, и уже из него - запуск Python-скрипта, который внутри умеет только понижать/поднимать `EUID` в допустимых границах.

Для удобства запуска поверх бинарника и Python-скриптов использованы небольшие shell-обёртки в каталоге `/Users/practice2/bin`. Они просто выполняют команду вида `exec /Users/practice2/bin/run_access /Users/practice2/bin/access.py` (и для утилиты подбора пароля `crack.py`) с пробросом аргументов пользователя. Каталог `/Users/practice2/bin` добавляется в переменную окружения `PATH`, поэтому пользователь запускает утилиту короткой командой, без указания полного пути. Вся чувствительная часть (получение прав `root` и доступ к `etc/passwd` и `log/access.log`) реализована в небольшом C-бинарнике и в вызовах `os.seteuid()` внутри `access.py`, что и позволило корректно решить проблему выдачи доступа без использования `sudo` и без нарушения ограничений безопасности macOS.

2.5.4 Реализация атаки brute force

Для реализации атаки полного перебора был разработан скрипт `crack.py`, который читает хэш пароля целевого пользователя из файла `/Users/practice2/etc/passwd` и пытается восстановить исходный пароль по этому хэшу. Для хэширования паролей как в основной программе, так

и в скрипте подбора используется алгоритм SHA256 из модуля `hashlib`, поэтому при генерации пробного пароля он сначала преобразуется в SHA256, а затем полученный дайджест побайтово сравнивается с сохранённым значением. Если хэши совпали, считается, что пароль найден, и утилита выводит найденный пароль, количество перебранных вариантов и затраченное время; если после перебора всех вариантов подходящей длины совпадения не было, выводится сообщение о неудаче и статистика по числу попыток и времени работы.

Для ограничения времени эксперимента введено жёсткое ограничение в 8 часов. В начале подбора фиксируется момент старта, а в цикле генерации паролей на каждой итерации проверяется текущее время; если с начала прошло больше 8 часов, перебор немедленно прерывается, и в выводе явно указывается, что подбор остановлен по таймауту, даже если пароль ещё не был найден. Это позволяет не «убить» машину слишком долгим экспериментом при больших длинах пароля.

Число возможных паролей для заданной длины L оценивается как

$$N_L = 52 \cdot 72^{L-1},$$

где 52 - количество допустимых символов для первой позиции (латинские буквы в верхнем и нижнем регистре), а 72 - количество возможных символов для каждой из последующих позиций (буквы, цифры и спецсимволы). Таким образом, первая буква всегда является буквой, а все остальные символы могут быть любыми из расширенного алфавита. Теоретическое суммарное количество вариантов от длины 1 до L вычисляется как

$$N_{\leq L} = \sum_{k=1}^L N_k,$$

и именно это значение выводится режимом расчёта теоретической сложности.

Скрипт поддерживает два режима запуска. В основном режиме пользователь вызывает утилиту как `/Users/practice2/bin/run_crack <LOGIN> <LENGTH>`, где `LOGIN` - имя пользователя, для которого выполняется подбор, а `LENGTH` - максимальная длина пароля. В этом случае сначала выводится теоретическое количество комбинаций и затем запускается реальный перебор с измерением времени. Во втором режиме, `/Users/practice2/bin/run_crack -max-iterations <LENGTH>`, утилита не выполняет реальный перебор паролей, а только рассчитывает и выводит теоретическое число итераций и суммарное количество вариантов для длин от 1 до указанной, что позволяет оценить сложность атаки brute force без запуска длительного эксперимента.

2.5.5 Реализация управления пользователями

Управление пользователями в системе реализовано отдельной утилитой `/Users/practice2/bin/user_manager.py`, представляющей собой Python-скрипт. Данная утилита предназначена только для администратора и запускается от имени `root`, так как работает напрямую с файлом `/Users/practice2/etc/passwd` и изменяет содержащиеся в нём записи. Обычный пользователь не имеет прав на запуск этой программы и на модификацию `passwd`.

Через `user_manager.py` администратор может добавлять новых пользователей, редактировать уже существующих, изменять их основные данные (ФИО, идентификатор, хэш пароля) и присваивать или отзывать права доступа (на чтение, запись, удаление файлов в `confdata/`). Также поддерживается удаление пользователя: в этом случае соответствующая строка полностью убирается из `passwd`. Логика работы проста: при выборе операции утилита читает текущий файл `passwd`, находит (или создаёт) нужную запись по логину, вносит изменения в поля (пароль предварительно хэшируется тем же SHA256), после чего формирует новое содержимое и атомарно перезаписывает файл. Таким образом, все изменения прав и состава пользователей выполняются централизованно и контролируются только администратором.

2.6 Примеры работы программы

Ниже на рисунках 19 - 26, представлены результаты работы программы:

На первом этапе через вход в суперпользователя, создаем пользователя New (см. рисунок 19):

```
Ayzeks-MBP:bin root# python user_manager.py
user_manager.py {add|edit|delete}
Ayzeks-MBP:bin root# python user_manager.py add
Adding new user
Login: New
LnFnSn: Some Name
Permissions: (r - read, w - write, d - delete (can combine)): rw
Password:
Confirm password:
User added
```

Рис. 19: Суперпользователь Root добавляет пользователя New

На рисунке 20, суперпользователь меняет данные о пользователе New:

```
Ayzeks-MBP:bin root# python user_manager.py edit
Enter login of user: New
Keep empty to make no change
LFS(ФИО) (Some Name): Another name
Permissions: (rw): rd
Change password? (y/n): y
New password:
Confirm password:
User changed
```

Рис. 20: Суперпользователь Root меняет данные пользователя New

После выхода через exit, можно попробовать авторизоваться через нового пользователя (рис. 21):

```
(base) monadayzek@Ayzeks-MBP ~ % run_access
Login: New
Password:
Info-Protection Lab2, Another name
use help
> help
We have here:
read <filename>
append <filename>
copy <source> <dest>
remove <filename>
perm - permissions
edit my info
exit
help
>
```

Рис. 21: Авторизация и пропись команды help

Пользователь может прочесть файл или удалить, так как при изменении пользователя, суперпользователь дал права rd(20), ниже на рисунке 22 представлены действия пользователя по чтению и удалению файла:


```

Login: New
Password:
Info-Protection Lab2, Another name
use help
> read data.txt
Super secret data !

> remove data.txt
File data.txt deleted
> read data.txt
Not found: data.txt
> █

```

Рис. 22: Чтение и удаление файла пользователем с правами rw

Далее на рисунке 23, представлена структура хранения хэшей согласно требованиям:

```

(base) monadayzek@AyzeKs-MBP ~ % sudo cat /Users/practice2/etc/passwd
Ayzek:7c04837eb356565e28bb14e5a1dedb240a5ac2561f8ed318c54a279f6a9665e:1000:rw:Salim1 Ayzek
Olga:7c04837eb356565e28bb14e5a1dedb240a5ac2561f8ed318c54a279f6a9665e:1001:rw:Vaganova Olga
easy:7c04837eb356565e28bb14e5a1dedb240a5ac2561f8ed318c54a279f6a9665e:1002:r:EaEa
Easy:f37598d21e9ebda7c2ebce28f2e9f3dc08f615271c0286e463e7e645eb13583d:1003:r:Easy User
New:f37598d21e9ebda7c2ebce28f2e9f3dc08f615271c0286e463e7e645eb13583d:1004:rd:Another name

```

Рис. 23: Хранение данных в passwd

Как видно, структура хранения имеет вид - Логин:Хэш-значение:UID:Права:ФИО.

Ниже показан вывод логирования, на рисунке 24 представлено логирование действий, а на рисунке 25 - логирование авторизаций:

```

(base) monadayzek@AyzeKs-MBP ~ % sudo cat /Users/practice2/log/access.log
2026-03-10 19:06:49,579 - INFO - User Ayzek viewed their permissions: rwd
2026-03-10 19:08:34,684 - INFO - User Ayzek updated their info (password and/or fullname)
2026-03-10 20:20:04,842 - INFO - User Ayzek viewed their permissions: rwd
2026-03-10 20:21:14,577 - INFO - User Ayzek readed file ConfData.txt
2026-03-10 20:21:25,158 - INFO - User Ayzek added data to file ConfData.txt
2026-03-10 20:21:30,210 - INFO - User Ayzek readed file ConfData.txt
2026-03-10 20:37:33,040 - INFO - User Easy readed file ConfData.txt
2026-03-10 20:37:36,971 - INFO - User Easy viewed their permissions: r
2026-03-10 20:37:48,022 - INFO - User Easy updated their info (password and/or fullname)
2026-03-11 14:43:57,563 - INFO - User New readed file ConfData.txt
2026-03-11 14:44:10,523 - INFO - User New deleted file ConfData.txt
2026-03-11 14:45:20,103 - INFO - User New readed file data.txt
2026-03-11 14:45:27,302 - INFO - User New deleted file data.txt

```

Рис. 24: Логирование действий пользователей

```

(base) monadayzek@AyzeKs-MBP ~ % sudo cat /Users/practice2/log/auth.log
2026-03-10 17:56:19,788 - INFO - Added Ayzek (UID=1000)
2026-03-10 17:56:34,896 - INFO - Added Olga (UID=1001)
2026-03-10 18:27:20,270 - INFO - Changed user Ayzek
2026-03-10 18:40:10,721 - INFO - Changed user Ayzek
2026-03-10 18:40:28,545 - INFO - Added easy (UID=1002)
2026-03-10 18:51:23,967 - INFO - Changed user Ayzek
2026-03-10 20:35:58,712 - INFO - Added Easy (UID=1003)
2026-03-10 20:36:25,932 - INFO - Changed user Easy
2026-03-11 11:58:09,234 - INFO - Changed user Ayzek
2026-03-11 13:53:51,408 - INFO - Added New (UID=1004)
2026-03-11 13:54:50,651 - INFO - Changed user New

```

Рис. 25: Логирование действий авторизации

В каталоге confdata, находятся обычные текстовые файлы, которые были созданы супер-пользователем путем VIM (или др. редактором):

```

[AyzeKs-MacBook-Pro:practice2 root# vim confdata/new.txt
[AyzeKs-MacBook-Pro:practice2 root# ls confdata/
new.txt

```

Рис. 26: Каталог confdata

2.7 Атака brute force

Для исследования зависимости итераций и времени на нахождения пароля, было создано 9 пользователей со следующими атрибутами пароля:

- u1 : $L = 1, p \in \mathbb{C} \cup \mathbb{N}$
- u2 : $L = 2, p \in \mathbb{C} \cup \mathbb{N}$
- u3 : $L = 2, p \in \mathbb{C} \cup \mathbb{N}$
- u4 : $L = 2, p \in \mathbb{C} \cup \mathbb{N}$
- u5 : $L = 2, p \in \mathbb{C} \cup \mathbb{N}$
- onlychar : $L = 4, p \in \mathbb{L}$
- abc3 : $L = 3, p = abc$
- sym : $L = 4, p \in \mathbb{C}/\mathbb{L}$
- u8 : $L = 8, p \in (\mathbb{C}/\mathbb{L}) \cup \mathbb{N}$,

где L - длина пароля, p - значение пароля, $\mathbb{C}$ - множество символов и букв, $\mathbb{N}$ - натуральные числа включая 0, $\mathbb{L}$ - множество букв.

Графики результатов разделены на 2 части: по длинам пароля, по значениям пароля и по длине = 8.

Результаты проведения эксперимента представлены на рисунке 27 для длин пароля и на рисунке 28 для значений пароля.

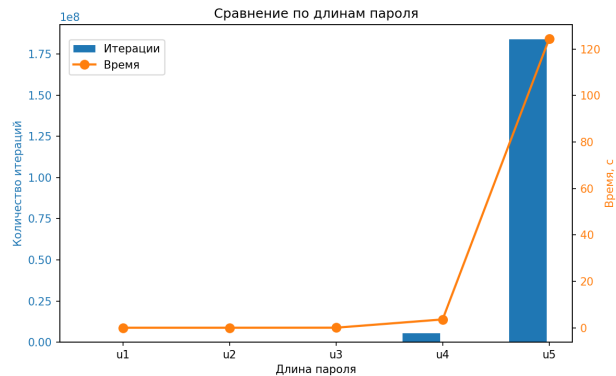


Рис. 27: Результаты проведения эксперимента для длин пароля

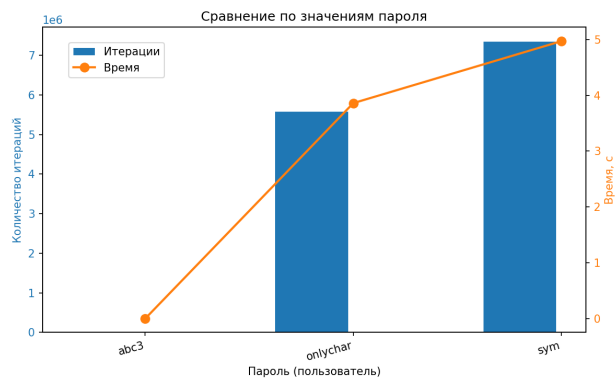


Рис. 28: Результаты проведения эксперимента для значений пароля

Как видно при увеличении длины пароля, количество итераций и времени на нахождения пароля увеличивается экспоненциально.

Для пользователя `u8`, было подсчитано:

$$\frac{N_8}{N_5} = \frac{52 \cdot 72^{8-1}}{52 \cdot 72^{5-1}} = 72^3 = 373248.$$

То есть для нахождения пароля длиной 8, нужно в 373248 раз больше итераций, чем для пароля длиной 5. Для нахождения пароля длиной 5 заняло 124.40 секунд, тогда для нахождения пароля длиной 8 заняло бы явно больше 8 часов. В коде сработал бы таймаут.

Заключение

В ходе лабораторной работы была реализована и исследована простая система управления доступом к конфиденциальным данным. Для этого были спроектированы отдельные каталоги `/Users/practice2` и `/Users/practice3` с жёстко заданными правами, реализованы утилиты аутентификации и доступа (`access.py`), подбора паролей (`crack.py`) и административного управления пользователями (`user_manager.py`). Для корректной работы с защищёнными файлами `etc/passwd` и журналами при сохранении ограничений macOS был использован `setuid`-бинарник на C и аккуратное управление эффективным идентификатором пользователя из Python-кода.

Также были реализованы и протестированы модели дискреционного и мандатного управления доступом (DAC и MAC) с использованием тех же принципов хранения и проверки прав. В рамках экспериментов исследована сложность атаки `brute force` для различных длин паролей и сопоставлены теоретические оценки с практическими результатами. В целом поставленные в работе задачи по проектированию и реализации защищённого доступа к файлам, организации хранения служебных данных и моделированию атак были выполнены.

Исходные коды практической работы №2, доступны в приложении А, где:

- `access.py` - скрипт авторизации пользователя;
- `run_access.c` - утилита для подготовки исполняемого кода;
- `run_access` - shell-скрипт для исполнения скрипта авторизации пользователя с выдачей EUID;
- `crack.py` - скрипт для атаки `brute force`;
- `user_manager.py` - скрипт для управления пользователями (`chmod 700`);
- `config.py` - скрипт задающий конфигурацию основным утилитам.

3 Реализация моделей дискреционного и мандатного управления доступом

3.1 Введение

Помимо базовой аутентификации и разграничения прав по субъектам, в защищённых системах применяют модели дискреционного (DAC) и мандатного (MAC) управления доступом. Дискреционная модель позволяет владельцу объекта самостоятельно назначать и отзывать права доступа другим субъектам; таким образом реализуется гибкое управление доступом на уровне файлов и пользователей. Мандатная модель опирается на метки конфиденциальности субъектов и объектов: доступ разрешён только при соблюдении правил (например, «не читать выше своего уровня», «не записывать ниже»), что снижает риск утечки при компрометации учётной записи. В данной главе кратко описывается реализация обеих моделей в рамках лабораторного стенда и их связь с утилитами доступа к конфиденциальным данным.

3.2 Постановка задачи

3.2.1 Цели

- изучить особенности моделей дискреционного (DAC) и мандатного (MAC) управления доступом;
- на базе разработанной в практической работе 2 системы аутентификации и авторизации реализовать DAC и MAC.

3.2.2 Задачи

- На базе разработанной в практической работе 2 системы аутентификации и авторизации разработать систему, реализующую DAC, включая:
 - владение объектами;
 - произвольное назначение прав доступа к объекту;
 - контроль на основе матрицы или списка доступа.
- На базе разработанной в практической работе 2 системы аутентификации и авторизации разработать систему, реализующую MAC, включая:
 - метки безопасности для субъектов и объектов;
 - свойства безопасности модели Белла-Лападулы.

3.3 Требования к работе

- Требования к дереву каталогов, остается прежним, однако создается новая родительская директория `practice3`;
- Требования к DAC:
 - у каждого объекта должен быть владелец, который может произвольно назначать и передавать права доступа к этому объекту;
 - контроль доступа к объектам должен осуществляться на основе матрицы или списка доступа, в которых хранится информация о правах доступа субъектов к объектам.
- Требования к MAC:
 - каждому субъекту и объекту должна присваиваться метка безопасности суперпользователем;

- предусмотреть не менее трех уровней меток безопасности: «несекретно», «для служебного пользования (ДСП)», «секретно»;
- субъекты не могут менять метки безопасности;
- контроль доступа к объектам должен осуществляться на основе свойств безопасности модели Белла-Лападулы: «нет чтения сверху», «нет записи вниз» и дискреционного свойства.

3.4 Сведения о DAC и MAC

3.4.1 DAC (Discretionary Access Control)

Дискреционное управление доступом (DAC) - модель разграничения доступа, в которой владелец ресурса самостоятельно определяет права субъектов на объекты (чтение, запись, исполнение). Права фиксируются в матрице доступа или списках ACL. Особенность: гибкость управления при потенциальной уязвимости к несанкционированной передаче прав.

3.4.2 MAC (Mandatory Access Control)

Мандатное управление доступом (MAC) - модель, основанная на присвоении меток безопасности субъектам и объектам. Доступ разрешается только при выполнении правил сравнения меток, централизованно заданных политикой безопасности. Ключевой принцип реализуется моделью Белла-Лападулы, обеспечивающей иерархический контроль потоков информации и предотвращающей несанкционированное понижение уровня конфиденциальности. В отличие от DAC, пользователь не может изменять политику доступа.

3.5 Реализация DAC

Реализация дискреционной модели выполнена поверх предыдущей практической работы: используется тот же принцип аутентификации по `/Users/practice3/etc/passwd`, те же утилиты доступа и логирования. Все файлы системы размещены в `/Users/practice3/` (каталоги `etc`, `confdata`, `bin`, `log`). Такое построение добавляет дополнительный уровень разграничения доступа и способствует обеспечению конфиденциальности, целостности и доступности информации: владелец файла явно задаёт, кто имеет право читать, изменять или удалять его данные, а все действия фиксируются в логах.

Владение реализовано так: при создании файла командой `create` пользователь, создавший файл, записывается как владелец (`owner`) этого объекта. Создаётся запись в служебном хранилище, и владельцу сразу выдаются полные права `rwd` на созданный файл. Передача прав выполняется командами `grant` и `revoke`. Выдать права (`grant <файл> <пользователь> <права>`) может только владелец файла; отозвать права (`revoke <файл> <пользователь>`) также может только владелец. Нельзя отозвать права у самого себя (владелец остаётся единственным, кто может управлять списком доступа). Проверка при операциях чтения, записи и удаления: у субъекта должна быть соответствующая буква (`r`, `w` или `d`) в его записи для данного файла.

Информация о доступе хранится не в виде одной матрицы «субъект-объект», а по объектам: для каждого файла ведётся свой список прав. Список доступа (ACL, Access Control List) - это перечень пар «субъект : права» для одного объекта. В нашей реализации ACL хранится в файле `/Users/practice3/etc/acl.json` в формате JSON: для каждого имени файла задаётся поле `owner` (логин владельца) и поле `acl` - словарь вида `{ "логин": "rwd ... }`. Таким образом, матрица доступа представлена в виде набора ACL по файлам, что эквивалентно хранению по столбцам матрицы доступа.

3.6 Реализация MAC

Реализация мандатной модели также выполнена поверх той же базы: каталог `/Users/practice3/`, общий `passwd` и общая структура утилит доступа. В `passwd` для каждого пользователя хранится не только логин, хэш пароля и права, но и уровень конфиденциальности (метка субъекта). Используются три уровня: 0 - unclassified, 1 - DSP, 2 - secret. Метки объектов (файлов) хранятся отдельно в `/Users/practice3/etc/file_labels.json`: для каждого имени файла записан числовой уровень (0, 1 или 2). При создании файла командой `create` файлу присваивается метка, равная уровню создателя; таким образом, метка объекта задаётся один раз при создании и в дальнейшем определяет, кто может читать или записывать этот объект по правилам Белла-Лападулы.

Доступ по чтению разрешён только если уровень субъекта не ниже уровня объекта (No Read Up): пользователь с уровнем 0 не может читать файлы с уровнем 1 или 2; с уровнем 1 - не может читать уровень 2. Доступ по записи разрешён только если уровень субъекта не выше уровня объекта (No Write Down): пользователь с уровнем 2 не может записывать в файлы с уровнем 1 или 0, чтобы не понижать конфиденциальность данных. В коде это реализовано функциями `check_mac_read` (разрешение при `subject_level >= object_level`) и `check_mac_write` (разрешение при `subject_level <= object_level`). Перед каждой операцией чтения или записи выполняется соответствующая проверка; при нарушении выводится сообщение об отказе с указанием уровней. Удаление файла по-прежнему контролируется дискреционным ACL: право `d` у субъекта на данный объект и владение; при этом при создании и удалении обновляется и `file_labels.json`.

Список доступа (ACL) в MAC хранится так же, как в DAC: в `/Users/practice3/etc/acl.json` - владелец и права `rwd` по пользователям для каждого файла. Дополнительно в `/Users/practice3/etc/file_labels.json` хранится метка конфиденциальности каждого файла. Итоговое решение доступа: сначала проверяется DAC (есть ли у пользователя нужное право в ACL), затем MAC (соблюдаются ли правила No Read Up и No Write Down по меткам). Таким образом, мандатная политика накладывается поверх дискреционной и обеспечивает запрет недопустимых потоков информации по уровням.

3.7 Примеры работы программы

Ниже на рисунках 29-37, представлен пример работы программы с контролем доступа DAC:

```
(base) monadayzek@Ayzeks-MBP ~ % run_access_DAC
Login: Ayzek
Password:

Info-Protection Lab3 (DAC), Salimliii
Type 'help' for commands

Ayzek> help
We have here:

create <filename>
read <filename>
write <filename>
delete <filename>
grant <file> <user> <perms> (owner only)
revoke <file> <user> (owner only)
list files with access info
acl <filename> (owner only)
info (user)
edit my info
help
exit
Ayzek> █
```

Рис. 29: Пользователь Ayzek вызвал инструкцию help

Создание файла пользователем, автоматически назначает пользователя владельцем файла дав права rwd (рис. 30):

```
AyzeK> create numpy.py
File 'numpy.py' created!
  Owner: AyzeK (permissions: rwd)
AyzeK> write numpy.py
Enter text (empty line to save):
import numpy as np
arr = np.array([1,2,3])
print(arr)

Data written!
AyzeK> read numpy.py
import numpy as np
arr = np.array([1,2,3])
print(arr)

AyzeK> █
```

Рис. 30: Пользователь AyzeK создал файл numpy.py

Владелец файла может передать право другому пользователю (рис. 31):

```
AyzeK> grant numpy.py AyzeK_DAC r
Granted 'r' on 'numpy.py' to AyzeK_DAC
AyzeK> █
```

Рис. 31: Пользователь AyzeK передал право на файл numpy.py пользователю AyzeK_DAC

Переключившись на пользователя AyzeK_DAC, мы можем проверить, что он может читать файл numpy.py и не может записывать или удалять его (рис. 32):

```
Oops! Wrong login or password. Try again.
Login: AyzeK_DAC
[Password:

Info-Protection Lab3 (DAC), Salimli AyzeK
Type 'help' for commands

AyzeK_DAC> read numpy.py
import numpy as np
arr = np.array([1,2,3])
print(arr)

AyzeK_DAC> write numpy.py
ACCESS DENIED (DAC): You don't have write permission
AyzeK_DAC> delete numpy.py
ACCESS DENIED (DAC): You don't have delete permission
AyzeK_DAC> █
```

Рис. 32: Пользователь AyzeK_DAC может читать файл numpy.py и не может записывать или удалять его

Ниже на рисунке 33, представлен простой лист прав и владельцев, файлов и прав на них:

```

Ayzek_DAC> list
File                               Owner      Perms
-----
numpy.py                          Ayzek      r
secret.txt                        Ayzek      -
share.txt                         Ayzek      -
text.txt                          Ayzek      -
Ayzek_DAC>

```

Рис. 33: Лист прав и владельцев, файлов и прав на них

Владелец файла может увидеть ACL для файла numpy.py (рис. 34):

```

Ayzek> acl numpy.py
File: numpy.py
Owner: Ayzek
Access Control List:
Ayzek: rwd
Ayzek_DAC: r
Ayzek>

```

Рис. 34: ACL для файла numpy.py

Далее на рисунках 35 и 36, представлен содержимое файлов passwd и acl.json для DAC:

```

Ayzek_DAC:f37508d21e9ebda7c2ebce28f2e9f3dc08f615271c0286e463e7e645e013583d:1000:0:Saliml1 Ayzek
Ayzek_MAC:7c04837eb356565e28bb14e5a1dedb240a5ac2561f8ed318c54a279fb6a9665e:1001:2:Saliml1
Ayzek:7c04837eb356565e28bb14e5a1dedb240a5ac2561f8ed318c54a279fb6a9665e:1002:2:Saliml1
Second:7c04837eb356565e28bb14e5a1dedb240a5ac2561f8ed318c54a279fb6a9665e:1003:0:Second User

```

Рис. 35: Содержимое файла passwd для DAC

```

Ayzek> acl numpy.py
File: numpy.py
Owner: Ayzek
Access Control List:
Ayzek: rwd
Ayzek_DAC: r
Ayzek>

```

Рис. 36: Содержимое файла acl.json для DAC

Так как реализация MAC и DAC имеют единый каталог, на рисунке 35, видны цифры от 0 до 2, которые обозначают уровень конфиденциальности субъекта и объекта. Но они не имеют отношения к DAC, так как данные о правах берутся из файла acl.json, а пароль и логин из passwd.

Ниже представлена матрица доступа для DAC (рис. 37):

```

Ayzeks-MacBook-Pro:bin root# user_manager_DAC.py matrix
DAC: Subj, Obj
Subject  numpy.py[Ayzek]  secret.txt[Ayzek]  share.txt[Ayzek]  text.txt[Ayzek]
-----
Ayzek_DAC  r                  -                  -                  -
Ayzek_MAC  -                  -                  -                  -
Ayzek      rwd               rwd               rwd               rwd
Second     -                  -                  -                  -

```

Рис. 37: Матрица доступа для DAC

Матрица представляет собой субъекты, объекты и права между ними.

Далее на рисунках 38-45, представлен пример работы программы с контролем доступа MAC:

Ниже добавление пользователя с указанием уровня конфиденциальности (рис. 38):

```
Ayzeks-MacBook-Pro:bin root# python user_manager_MAC.py add
Adding new user (MAC)
Login: NewMAC
LnFnSn: New MAC User
Security levels (MAC):
  0 - unclassified
  1 - DSP
  2 - secret
Security level (0/1/2): 2
Password:
Confirm password:
User added with security level: secret
Ayzeks-MacBook-Pro:bin root#
```

Рис. 38: Добавление пользователя с указанием уровня конфиденциальности

По правилам, пользователь NewMAC, может создать файл только с уровнем конфиденциальности 2 (secret) (рис. 39):

```
NewMAC> help
We have here:

create <filename>
read <filename> (No Read Up)
write <filename> (No Write Down)
delete <filename>
grant <file> <user> <perms> (owner only)
revoke <file> <user> (owner only)
list (files with access info)
acl <filename> (owner only)
info (user)
edit my info
help
exit
NewMAC> create SecretFile.java
File 'SecretFile.java' created!
Owner: NewMAC (permissions: rwd)
Security label: secret
NewMAC>
```

Рис. 39: Создание файла с уровнем конфиденциальности 2 (secret)

Теперь можно создать пользователя с уровнем конфиденциальности 1 (ДСП) (рис. 40):

```
Ayzeks-MacBook-Pro:~ root# python /Users/practice3/bin/user_manager_MAC.py add
Adding new user (MAC)
Login: NewDSP
LnFnSn: New USER DSP
Security levels (MAC):
  0 - unclassified
  1 - DSP
  2 - secret
Security level (0/1/2): 1
Password:
Confirm password:
User added with security level: DSP
Ayzeks-MacBook-Pro:~ root#
```

Рис. 40: Добавление пользователя с уровнем конфиденциальности 1 (ДСП)

Пользователь NewDSP, может создать файл только с уровнем конфиденциальности 1 (ДСП) (рис. 41):

```

(base) monadayzek@AyzeKS-MBP ~ % run_access_MAC
Login: NewDSP
[Password:

Info-Protection Lab3 (MAC), New USER DSP
Security level: DSP
Type 'help' for commands

NewDSP> create DSP.txt
File 'DSP.txt' created!
Owner: NewDSP (permissions: rwd)
Security label: DSP
NewDSP> write DSP.txt
Enter text (empty line to save):
This is DSP file!

Data written!
NewDSP>

```

Рис. 41: Создание файла с уровнем конфиденциальности 1 (ДСП)

По правилу Белла-Лападулы, пользователь NewDSP не может читать файлы с уровнем конфиденциальности 2 (secret), а пользователь NewMAC может читать файлы с уровнем конфиденциальности 1 (ДСП) и 2 (secret), но не может записывать в них как показано на рисунке 42:

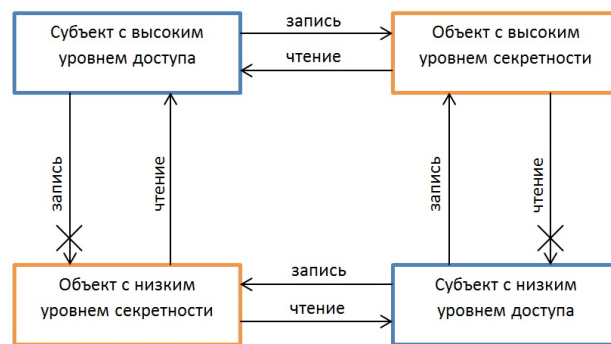


Рис. 42: Правило Белла-Лападулы

Далее на рисунке 43, показан сценарий попытки чтения файла SecretFile.java, пользователем NewDSP, а на рисунке 44, показан сценарий попытки записи в файл DSP.txt пользователем NewMAC. Для проведения сценария сначала нужно дать права друг другу на чтение и запись.

```

(base) monadayzek@AyzeKS-MBP ~ % run_access_MAC
Login: NewMAC
[Password:

Info-Protection Lab3 (MAC), New MAC User
Security level: secret
Type 'help' for commands

NewMAC> grant SecretFile.java NewDSP rwd
Granted 'rwd' on 'SecretFile.java' to NewDSP
NewMAC> exit
(base) monadayzek@AyzeKS-MBP ~ % run_access_MAC
Login: NewDSP
[Password:

Info-Protection Lab3 (MAC), New USER DSP
Security level: DSP
Type 'help' for commands

NewDSP> read SecretFile.java
ACCESS DENIED (MAC): No Read Up - your level (DSP) < file level (secret)
NewDSP>

```

Рис. 43: Сценарий попытки чтения файла SecretFile.java, пользователем NewDSP

```
NewMAC> write DSP.txt
ACCESS DENIED (MAC): No Write Down - your level (secret) > file level (DSP)
NewMAC>
```

Рис. 44: Сценарий попытки записи в файл DSP.txt пользователем NewMAC

Как видно из рисунка 43, если не дать права друг другу, то ошибка, будет касаться DAC контроля доступа (нет права).

Ниже на рисунке 45, представлена матрица доступа для MAC:

```
Ayzeks-MacBook-Pro:~ root# python /Users/practice3/bin/user_manager_MAC.py matrix
-----
macdac): Subj x Obj
NRU = No Read Up, NWD = No Write Down
U = unclassified, D = DSP, S = secret
-----
Subject(lvl)  DSP.txt[D]  SecretFile.java[S]numpy.py[U]  secret.txt[S]  share.txt[U]  text.txt[U]
-----
AyzeK_DAC(U) - !R      - !R      - !R      - !R      - !R
AyzeK_MAC(S) - !W      - !W      - !W      - !W      - !W
AyzeK(S)      - !W      - !W      - !W      - !W      - !W
second(U)     - !R      - !R      - !R      - !R      - !R
NewMAC(S)     rwd !W    rwd !W    rwd !W    rwd !W    rwd !W
NewDSP(D)     rwd      rwd !R    - !W      - !W      - !W
-----
Legend: !R = MAC blocks read (NRU), !W = MAC blocks write (NWD)
```

Рис. 45: Матрица доступа для MAC

Ниже матрицы указана легенда: !R - блокировка чтения, !W - блокировка записи, так же рядом указаны права доступа. Символы (U,D,S) обозначают уровень конфиденциальности субъекта и объекта.

3.8 Атака brute force

Атака не влияет на выбор метода аутентификации, так как пароль хэшируется перед сохранением в файл.

Ниже в таблице 4, продублированы результаты проведения эксперимента для всех паролей:

Таблица 4: Результаты проведения эксперимента для всех паролей

Пользователь	Количество итераций	Время, с
u1	1	0.00
u2	478	0.00
u3	24095	0.02
u4	5414807	3.57
u5	183837715	124.40
abc3	3871	0.00
onlychar	5576615	3.86
sym	7338865	4.97
u8	-	Начался SWAP

Заключение

В данной главе были реализованы и исследованы модели дискреционного (DAC) и мандатного (MAC) управления доступом поверх уже разработанной системы аутентификации и разграничения прав. Для DAC была организована работа со списками доступа (ACL) в виде JSON-структур в каталоге /Users/practice3/etc: для каждого файла хранится владелец и набор прав rwd по пользователям. Такой формат позволяет рассматривать ACL как столбцы матрицы доступа и удобно обрабатывать права в утилите доступа к конфиденциальным данным. Для MAC была добавлена иерархия меток конфиденциальности (unclassified, DSP, secret) для субъектов и объектов, а также отдельный JSON-файл с метками файлов. Доступ реализован по правилам

Белла-Лападулы: «не читать вверх» (No Read Up) и «не записывать вниз» (No Write Down), при этом итоговое решение принимается с учётом как DAC, так и MAC. Проведённые эксперименты показали, что уровень доступности и корректность работы механизмов DAC/MAC не нарушаются с атаками brute force: атака ограничивается уровнем аутентификации, а модели управления доступом продолжают обеспечивать требуемую конфиденциальность и целостность данных.

Исходные коды практической работы №3, доступны в приложении В, где:

- access_DAC.py - скрипт авторизации пользователя для DAC;
- access_MAC.py - скрипт авторизации пользователя для MAC;
- config_DAC.py - скрипт задающий конфигурацию для DAC;
- config_MAC.py - скрипт задающий конфигурацию для MAC;
- user_manager_DAC.py - скрипт для управления пользователями для DAC;
- user_manager_MAC.py - скрипт для управления пользователями для MAC.

Файлы run_access.c, run_access, оставлены исходными, так как расположение вписано в \$PATH. (Смотреть приложение А).

4 Мэжсетевое экранирование

4.1 Введение

Межсетевое экранирование - это одна из базовых технологий защиты компьютерных сетей, реализуемая с помощью межсетевых экранов (МЭ, Firewall, Brandmauer), которые также называют брандмауэрами или файерволами. МЭ представляет собой программное или программно-аппаратное средство, контролирующее трафик между двумя и более сетями или между отдельными узлами. Основная цель его работы - фильтрация потока данных на основе заданного набора правил, что позволяет предотвратить несанкционированный доступ как извне в защищаемую сеть, так и изнутри наружу.

В зависимости от принципов функционирования межсетевые экраны делятся на несколько типов. Простейшие из них - пакетные фильтры (packet filters) - анализируют заголовки IP-пакетов и протоколов верхнего уровня (TCP, UDP, ICMP), принимая решение о пропуске или блокировке на основе адресов отправителя и получателя, портов и флагов. Классическим примером такого фильтра для ОС Linux является iptables - компонент ядра Linux. Однако данная работа выполняется в macOS, которая не использует iptables, поскольку последний привязан к ядру Linux. В macOS по умолчанию применяется межсетевой экран pf (Packet Filter) - также реализующий контроль состояния соединений (stateful inspection). Packet Filter сопоставим по возможностям с iptables, но отличается более лаконичным синтаксисом правил и использует единый конфигурационный файл /etc/pf.conf, что упрощает управление фильтрацией. В рамках данной работы рассматривается настройка политики доступа с помощью pf на macOS, что позволяет получить практические навыки работы с межсетевым экраном в UNIX-подобной системе.

На рисунке 46, представлен принцип работы межсетевого экрана на схеме, где WAN - внешняя сеть, LAN - внутренняя сеть, а Firewall - межсетевой экран:

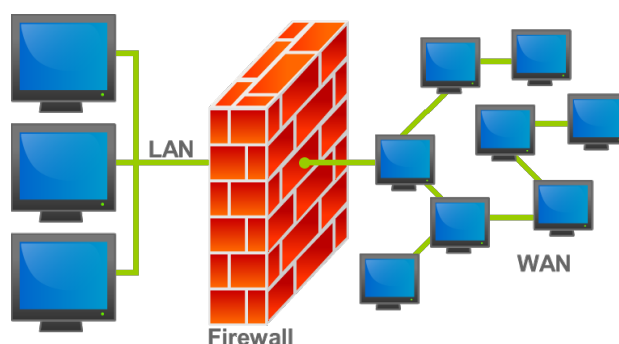


Рис. 46: Принцип работы межсетевого экрана

4.2 Постановка задачи

4.2.1 Цели

Целью работы является получение базовых знаний и навыков работы по настройке межсетевого экрана в GNU/Linux и UNIX-подобных системах.

4.2.2 Задачи

Задачами практической работы №4 являются:

- изучение мэжсетевого экрана iptables (в работе используется Packet Filter так как iptables привязан к ядру Linux), обеспечивающего выполнение задания;
- Реализация следующей политики доступа:

1. Установить утилиту iptables или другую, обеспечивающий выполнение задания (Packet Filter);
2. Изучить команды вывода таблиц фильтрации, добавления, редактирования и удаления правил;
3. Реализовать следующие правила:
 - a) разрешить локальное взаимодействие через loopback интерфейс;
 - b) разрешить взаимодействие с DNS-сервером;
 - c) разрешить использование утилиты ping для проверки достижимости компьютеров с любыми IP-адресами во внешней сети;
 - d) разрешить доступ по протоколам HTTP/HTTPS к внешним серверам;
 - e) блокировать все пакеты не удовлетворяющие условиям;
 - f) произвести проверку корректности реализованной политики с ping, nslookup, web браузером;
 - g) проконтролировать прохождение пакетов через tcpdump.

4.3 Проектирование системы

Общая схема системы представлена на рисунке 47:

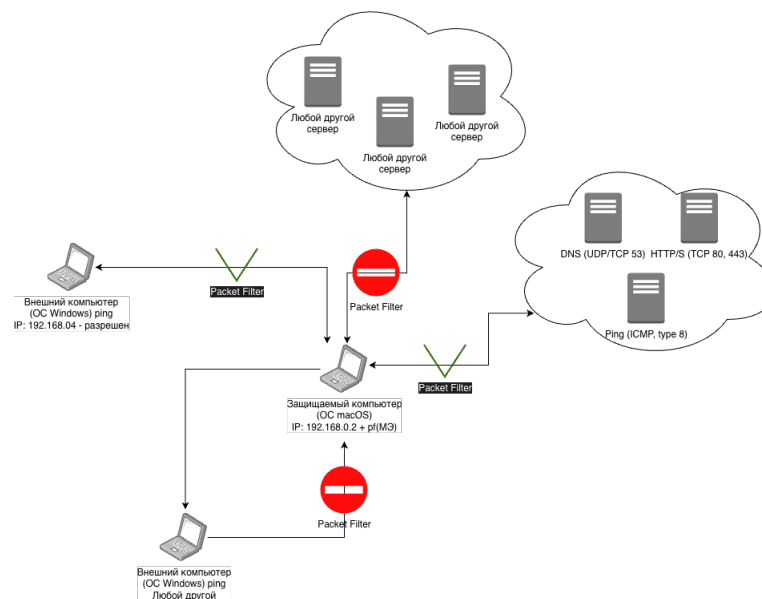


Рис. 47: Общая схема системы

Ее подробное описание представлено в двух таблицах. В таблице 4.3, представлены разрешенные серверы и порты для защищаемого компьютера, а в таблице 4.3, представлены запрещенные серверы и порты для защищаемого компьютера.

Таблица 5: Разрешено

Действие	Результат
Mac → любой хост: ping	Разрешено (ICMP)
Mac → любой хост: DNS (порт 53)	Разрешено
Mac → любой хост: HTTP/HTTPS (80, 443)	Разрешено
Windows 192.168.0.4 → Mac: ping	Разрешено

Таблица 6: Запрещено

Действие	Результат
Любой другой ПК (не 192.168.0.4) → Mac: ping	Запрещено
Любой сервер/ПК → Mac: любой TCP/UDP	Запрещено
Mac → сервер: SSH (22), SMTP (25) и т.п.	Запрещено
Mac → сервер: порты, кроме 53, 80, 443	Запрещено

4.4 Принцип обработки пакетов в pf

Packet Filter (pf) является потомком одноименного межсетевого экрана из ОС OpenBSD и предоставляет собой основной компонент для фильтрации трафика на уровне ядра в macOS. В отличие от iptables, pf интегрирован в сетевой стек XNU (Darwin) и управляется утилитой пользовательского пространства `/sbin/pfctl` [4].

Особенность pf заключается в интеграции с системой инициализации и управлениями службами `launchd` [4].

Основной файл конфигурации pf находится в `/etc/pf.conf`. Он определяет макросы, таблицы, опции и правила фильтрации.

Ниже представлен пример конфигурации pf в собственной операционной система:

```

1 cat /etc/pf.conf
2
3 (base) monadayzek@Ayzeks-MacBook-Pro ~ % sudo cat /etc/pf.conf
4 Password:
5 #
6 # Default PF configuration file.
7 #
8 # This file contains the main ruleset, which gets automatically loaded
9 # at startup. PF will not be automatically enabled, however. Instead,
10 # each component which utilizes PF is responsible for enabling and disabling
11 # PF via -E and -X as documented in pfctl(8). That will ensure that PF
12 # is disabled only when the last enable reference is released.
13 #
14 # Care must be taken to ensure that the main ruleset does not get flushed,
15 # as the nested anchors rely on the anchor point defined here. In addition,
16 # to the anchors loaded by this file, some system services would dynamically
17 # insert anchors into the main ruleset. These anchors will be added only
18 # when
19 # the system service is used and would removed on termination of the service
20 # .
21 #
22 # See pf.conf(5) for syntax.
23 #
24 # com.apple anchor point
25 #

```

```

26 scrub-anchor "com.apple/*"
27 nat-anchor "com.apple/*"
28 rdr-anchor "com.apple/*"
29 dummynet-anchor "com.apple/*"
30 anchor "com.apple/*"
31 load anchor "com.apple" from "/etc/pf.anchors/com.apple"
32 (base) monadayzek@Ayzeks-MacBook-Pro ~ % cat /etc/pf.conf
33 #
34 # Default PF configuration file.
35 #
36 # This file contains the main ruleset, which gets automatically loaded
37 # at startup. PF will not be automatically enabled, however. Instead,
38 # each component which utilizes PF is responsible for enabling and disabling
39 # PF via -E and -X as documented in pfctl(8). That will ensure that PF
40 # is disabled only when the last enable reference is released.
41 #
42 # Care must be taken to ensure that the main ruleset does not get flushed,
43 # as the nested anchors rely on the anchor point defined here. In addition,
44 # to the anchors loaded by this file, some system services would dynamically
45 # insert anchors into the main ruleset. These anchors will be added only
46 # when
47 # the system service is used and would removed on termination of the service
48 # .
49 #
50 # See pf.conf(5) for syntax.
51 #
52 #
53 # com.apple anchor point
54 scrub-anchor "com.apple/*"
55 nat-anchor "com.apple/*"
56 rdr-anchor "com.apple/*"
57 dummynet-anchor "com.apple/*"
58 anchor "com.apple/*"
59 load anchor "com.apple" from "/etc/pf.anchors/com.apple"

```

Для обеспечения сохранности пользовательской конфигурации pf, при обновлениях системы используется механизм anchor (как видно com.apple подгружает свои правила-якоря).

Сами же пользовательские правила размещаются в файле /etc/pf.anchors/*.conf,(rules).

Ниже представлена якорная конфигурация для сервисов AirDrop, Application Firewall:

```

1 (base) monadayzek@Ayzeks-MacBook-Pro pf.anchors % cat /etc/pf.anchors/
  com.apple
2 #
3 # com.apple ruleset, referred to by the default /etc/pf.conf file.
4 # See notes in that file regarding the anchor point in the main ruleset.
5 #
6 # Copyright (c) 2011 Apple Inc. All rights reserved.
7 #
8 # AirDrop anchor point.
9 #
10 anchor "200.AirDrop/*"
11
12 #
13 # Application Firewall anchor point.
14 #
15 anchor "250.ApplicationFirewall/*"

```


Такой подход хорош тем, что не нужно менять главный конфигурационный файл `/etc/pf.conf`, а только подгружать свой набор якорных правил.

Запуском и остановкой службы в macOS управляет демон `launchd`. Apple предоставляет свой собственный демон `/System/Library/LaunchDaemons/com.apple.pfctl.plist`, который выполняет команду `pfctl -f /etc/pf.conf` [4]. Эта команда загружает конфигурацию правил если МЭ включен.

Правила `pf` в macOS, обрабатываются последовательно, последнее совпавшее правило определяет действие (по умолчанию `pass` (пропуск)).

Типовая структура включает:

- Действие: `pass` (пропуск), `block` (блокировать с опцией `drop/return`);
- Направление: `in` (входящие пакеты), `out` (исходящие пакеты);
- Интерфейс: привязка к сетевому интерфейсу (по типу `en0`);
- Протоколы: `TCP`, `UDP`, `ICMP`;
- Сокеты (адрес-порт): отправителя и получателя;
- Параметры: например `icmp-type`, `code`, `quick`;
- Так же есть возможность исключения обратной петли `set skip on lo0`, для повышения производительности.

4.5 Реализация

Для реализации практической части поставленной задачи, были созданы 4 файла:

- `firewall_rules.py` - файл с правилами фильтрации, можно применить, сбросить или изменить;
- `config_pf.py` - файл с конфигурацией.
- `setup_practice4.py` - файл с настройкой `pf`.

Идея реализации: Для начала необходимо создать скрипт или вручную добавить правила в якорный файл `/etc/pf.anchors`. Это обусловлено тем, что если напрямую прописать правила в `/etc/pf.conf`, то при любом обновлении системы, правила будут стерты.

Для этого был создан скрипт, автоматически добавляющий файл `practice4` в `/etc/pf.anchors`, таким образом созданный файл будет подгружаться при запуске `pfctl`, и будет прописан как основные правила службы `pf`.

4.5.1 Правила фильтрации

В файле скрипта нужно прописать правила фильтрации указанные в таблицах 4.3 и 4.3.

Для этого, был создан скрипт `firewall_rules.py`, который просто применяет `append` к созданному файлу и добавляет строку со следующими правилами:

- `#ALLOWED_PING_FROM_IP=192.168.0.4` - подстановка IP из внешней сети, с которого разрешён входящий ping;
- `set block-policy drop` - блокированные пакеты отбрасываются тихо, без отправки ICMP или TCP RST;
- `scrub in all` - нормализация входящих пакетов (защита от фрагментации);
- `block all` - политика по умолчанию: блокировать весь трафик;
- После чего идут правила `pass`:

1. `pass quick on lo0` - разрешить loopback (локальный интерфейс);
2. `pass out quick proto udp to any port 53 keep state` - исходящий DNS по UDP, порт 53;
3. `pass out quick proto tcp to any port 53 keep state` - исходящий DNS по TCP, порт 53;
4. `pass out quick inet proto icmp all icmp-type 8 code 0 keep state` - исходящий ping (ICMP echo request);
5. `pass in quick inet proto icmp all icmp-type 0 code 0` - входящий ответ на ping (ICMP echo reply);
6. `pass in quick inet proto icmp from 192.168.0.4 icmp-type 8 code 0 keep state` - входящий ping только с указанного IP;
7. `pass out quick proto tcp to any port { 80 443 } keep state` - HTTP и HTTPS на портах 80 и 443.

4.5.2 Пример добавления правила

Ниже представлен вывод файла `/etc/pf.anchors/practice4`:

```

1  (base) monadayzek@Ayzeks-MacBook-Pro pf.anchors % cat practice4
2  # ALLOWED_PING_FROM_IP = 192.168.0.4
3
4  set block-policy drop
5  scrub in all
6
7  block all
8
9  # Loopback
10 pass quick on lo0
11
12 # DNS - UDP/TCP 53
13 pass out quick proto udp to any port 53 keep state
14 pass out quick proto tcp to any port 53 keep state
15
16 # Ping from ICMP type 8
17 pass out quick inet proto icmp all icmp-type 8 code 0 keep state
18
19 # ANSW to ping ICMP type 0
20 pass in quick inet proto icmp all icmp-type 0 code 0
21
22 # Ping in 192.168.0.4
23 pass in quick inet proto icmp from 192.168.0.4 icmp-type 8 code 0 keep
    state
24
25 # HTTP/HTTPS
26 pass out quick proto tcp to any port { 80 443 } keep state

```

Так же можно добавлять, удалять, сбрасывать и изменять правила фильтрации. Можно напрямую редактировать файл `/etc/pf.anchors/practice4`, средствами VIM или иным редактором.

Ниже показан пример добавления нового правила которая разрешает использование SSH по порту 22:

```

1  (base) monadayzek@Ayzeks-MacBook-Pro pf.anchors % echo "pass in proto
    tcp to any port 22 keep state" >> practice4

```

Так как SSH работает поверх протокола TCP, мы объявили `proto tcp` и указали порт 22.

Чтобы заблокировать SSH, можно просто убрать команду разрешения, или напрямую запретить, прописав:

```
1 (base) monadayzek@Ayzezs-MacBook-Pro pf.anchors % echo "block in proto  
tcp to any port 22" >> practice4
```

Что бы сбросить все правила, можно выполнить скрипт `firewall_rules.py` с аргументом `flush`:

```
1 (base) monadayzek@Ayzezs-MacBook-Pro lab-4 % sudo python firewall_rules.  
py flush
```

Для вывода правил, можно выполнить скрипт `firewall_rules.py` с аргументом `show`:

```
1 (base) monadayzek@Ayzezs-MacBook-Pro lab-4 % sudo python firewall_rules.  
py show  
2 Password:  
3 scrub-anchor "com.apple/*" all fragment reassemble  
4 anchor "com.apple/*" all  
5 anchor "practice4" all  
6 No ALTQ support in kernel
```

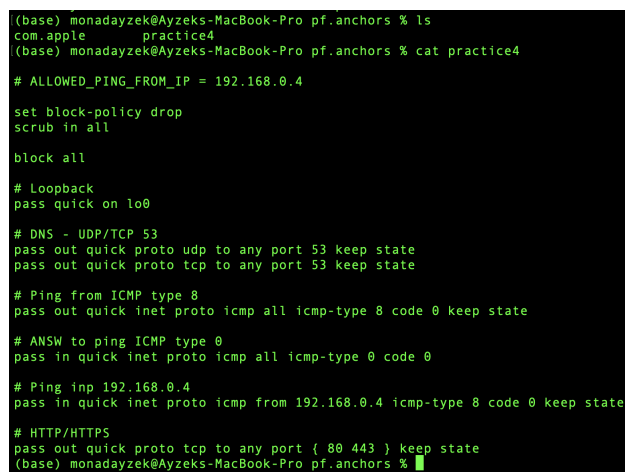
4.6 Проверки реализованной политики

Для осуществления проверок, необходимо для начала запустить созданный скрипт `setup_practice4.py`:

```
1 (base) monadayzek@Ayzezs-MacBook-Pro lab-4 % sudo python setup_practice4  
.py
```

Как было указано ранее, этот скрипт добавляет якорь в `pf.conf` и записывает правила в файл `/etc/pf.anchors/practice4`.

На рисунке 48, показан созданный якорь в `/etc/pf.anchors` :



```
(base) monadayzek@Ayzezs-MacBook-Pro pf.anchors % ls  
com.apple practice4  
(base) monadayzek@Ayzezs-MacBook-Pro pf.anchors % cat practice4  
  
# ALLOWED_PING_FROM_IP = 192.168.0.4  
  
set block-policy drop  
scrub in all  
  
block all  
  
# Loopback  
pass quick on lo0  
  
# DNS - UDP/TCP 53  
pass out quick proto udp to any port 53 keep state  
pass out quick proto tcp to any port 53 keep state  
  
# Ping from ICMP type 8  
pass out quick inet proto icmp all icmp-type 8 code 0 keep state  
  
# ANSW to ping ICMP type 0  
pass in quick inet proto icmp all icmp-type 0 code 0  
  
# Ping inp 192.168.0.4  
pass in quick inet proto icmp from 192.168.0.4 icmp-type 8 code 0 keep state  
  
# HTTP/HTTPS  
pass out quick proto tcp to any port { 80 443 } keep state  
(base) monadayzek@Ayzezs-MacBook-Pro pf.anchors %
```

Рис. 48: Созданный якорь в `/etc/pf.anchors` и правила в файле `practice4`

Подстановка в главном конфигурационном файле так же прописывается при выполнении скрипта. Главный файл изображен на рисунке 49.

```
#
# com.apple anchor point
#
scrub-anchor "com.apple/*"
nat-anchor "com.apple/*"
rdr-anchor "com.apple/*"
dummynet-anchor "com.apple/*"
anchor "com.apple/*"
load anchor "com.apple" from "/etc/pf.anchors/com.apple"

anchor "practice4"
load anchor "practice4" from "/etc/pf.anchors/practice4"
(base) monadayzek@Ayzeks-MacBook-Pro pf.anchors %
```

Рис. 49: Главный конфигурационный файл /etc/pf.conf

Как видно из рисунка 49, в главном конфигурационном файле прописана подстановка якоря practice4.

Далее что бы активировать правила, необходимо выполнить скрипт firewall_rules.py с аргументом apply:

```
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 % sudo python firewall_rules.py apply
Rules applied. ALLOWED_PING_FROM_IP = 192.168.0.4
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 %
```

Рис. 50: Активация правил

На рисунке 51 показан вывод информации о pf после активации правил с помощью команды pfctl -s info:

```
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 % sudo pfctl -s info
No ALTQ support in kernel
ALTQ related functions disabled
Status: Enabled for 0 days 00:01:41          Debug: Urgent

State Table                                Total          Rate
current entries                            113
searches                                  5984            59.2/s
inserts                                   138             1.4/s
removals                                   58             0.6/s
Counters
match                                     368             3.6/s
bad-offset                                0              0.0/s
fragment                                  0              0.0/s
short                                     0              0.0/s
normalize                                 0              0.0/s
memory                                    0              0.0/s
bad-timestamp                             0              0.0/s
congestion                                0              0.0/s
ip-option                                  0              0.0/s
proto-cksum                               0              0.0/s
state-mismatch                             0              0.0/s
state-insert                               0              0.0/s
state-limit                                0              0.0/s
src-limit                                  0              0.0/s
synproxy                                   0              0.0/s
dummynet                                   0              0.0/s
invalid-port                               0              0.0/s
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 %
```

Рис. 51: Информация о pf после активации правил

Далее можно посмотреть правила с помощью команды pfctl -sr:

```
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 % sudo pfctl -sr
No ALTQ support in kernel
ALTQ related functions disabled
scrub-anchor "com.apple/*" all fragment reassemble
anchor "com.apple/*" all
anchor "practice4" all
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 %
```

Рис. 52: Правила pf после активации правил

Красная стрелка на рисунке 52, указывает на наши активные правила.

После того как правила активированы, можно проверить loopback командой `ping -c 3 127.0.0.1`, как показано на рисунке 53:

```
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 % ping -c 3 127.0.0.1
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.089 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.202 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.123 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.089/0.138/0.202/0.047 ms
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 %
```

Рис. 53: Проверка loopback

Как видно из вывода утилиты `ping`, было отправлено 3 ICMP-пакета типа echo request на адрес 127.0.0.1 (loopback), и на каждый из них был получен ответ (echo reply). Потеря пакетов составила 0%, среднее время отклика - 0,138 мс.

Далее можно проверить DNS командой `nslookup google.com`, как показано на рисунке 54:

```
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 % nslookup google.com
Server:      192.168.0.1
Address:     192.168.0.1#53

Non-authoritative answer:
Name:   google.com
Address: 172.253.130.101
Name:   google.com
Address: 172.253.130.138
Name:   google.com
Address: 172.253.130.100
Name:   google.com
Address: 172.253.130.113
Name:   google.com
Address: 172.253.130.139
Name:   google.com
Address: 172.253.130.102
```

Рис. 54: Проверка DNS

Запрос был отправлен на сервер 192.168.0.1 на порт 53, который является портом DNS. Получен неавторитативный ответ (Non-authoritative answer), содержащий несколько IP-адресов домена google.com.

Далее идет проверка исходящего ping командой `ping -c 3 8.8.8.8`, как показано на рисунке 55:

```
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 % ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: icmp_seq=0 ttl=109 time=18.117 ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=109 time=13.801 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=109 time=10.435 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 10.435/14.118/18.117/3.144 ms
(base) monadayzek@Ayzeks-MacBook-Pro lab-4 %
```

Рис. 55: Проверка исходящего ping

Из вывода видно, что на три отправленных ICMP-запроса получено три ответа от узла 8.8.8.8, потеря пакетов отсутствует.

Ниже на рисунках 56 и 57, показаны тесты проверки протоколов HTTP (из `http://httpforever.com`) и HTTPS (из `https://github.com`):


```

PS C:\Users\olgav> ping 192.168.0.2

Обмен пакетами с 192.168.0.2 по 32 байтами данных:
Превышен интервал ожидания для запроса.
Превышен интервал ожидания для запроса.
Превышен интервал ожидания для запроса.
Превышен интервал ожидания для запроса.

Статистика Ping для 192.168.0.2:
    Пакетов: отправлено = 4, получено = 0, потеряно = 4
    (100% потерь)
PS C:\Users\olgav> |

```

Рис. 62: Проверка блокировки запрещенного трафика (без разрешенного IP)

Видно, что ПК не смог отправить ping на узел 192.168.0.2, что подтверждает блокировку трафика.

4.8 Дополнительные аргументы pfctl

Можно так же просматривать правила для конкретного якоря:

```

1 (base) monadayzek@Ayzeks-MacBook-Pro pf.anchors % sudo pfctl -sr -a
   practice4
2 No ALTQ support in kernel
3 ALTQ related functions disabled
4 scrub in all fragment reassemble
5 block drop all
6 pass quick on lo0 all flags S/SA keep state
7 pass out quick proto udp from any to any port = 53 keep state
8 pass out quick proto tcp from any to any port = 53 flags S/SA keep state
9 pass out quick proto tcp from any to any port = 80 flags S/SA keep state
10 pass out quick proto tcp from any to any port = 443 flags S/SA keep state
11 pass out quick inet proto icmp all icmp-type echoreq code 0 keep state
12 pass in quick inet proto icmp all icmp-type echorep code 0 keep state

```

Можно посмотреть сами якоря (аналог цепочки):

```

1 (base) monadayzek@Ayzeks-MacBook-Pro pf.anchors % sudo pfctl -s Anchors
2 No ALTQ support in kernel
3 ALTQ related functions disabled
4 com.apple
5 practice4

```

Можно вывести таблицы текущих состояний соединений:

```

1 (base) monadayzek@Ayzeks-MacBook-Pro pf.anchors % sudo pfctl -s states
2 No ALTQ support in kernel
3 ALTQ related functions disabled
4 ALL tcp 192.168.0.2:64414 -> 104.18.19.125:443 ESTABLISHED:ESTABLISHED
5 ALL tcp 192.168.0.2:64450 -> 3.173.21.63:443 ESTABLISHED:ESTABLISHED
6 ALL tcp 192.168.0.2:64454 -> 93.186.225.194:443 ESTABLISHED:
   ESTABLISHED
7 ALL tcp 192.168.0.2:64458 -> 87.240.137.207:443 ESTABLISHED:
   ESTABLISHED
8 ALL tcp 192.168.0.2:64472 -> 93.186.237.7:443 ESTABLISHED:ESTABLISHED
9 ALL tcp 192.168.0.2:64505 -> 95.213.56.3:443 ESTABLISHED:ESTABLISHED
10 ...
11 ...

```

Так же можно выводить NAT:

```

1 (base) monadayzek@Ayzeks-MacBook-Pro pf.anchors % sudo pfctl -sn

```



```
2  No ALTQ support in kernel
3  ALTQ related functions disabled
4  nat-anchor "com.apple/*" all
5  rdr-anchor "com.apple/*" all
```

Заключение

В данной главе спроектирована и реализована политика межсетевого экранирования с использованием Packet Filter (pf) в среде macOS. Изучена архитектура pf, принципы обработки пакетов и механизм якорных правил в `/etc/pf.anchors`, обеспечивающий сохранность пользовательской конфигурации при обновлениях системы. Разработан набор скриптов `firewall_rules.py`, `setup_practice4.py`, `test_policy.py`, `tcpdump_monitor.py`, `config_pf.py` для автоматизированной настройки и управления правилами фильтрации.

Реализованная политика доступа предусматривает: разрешение трафика через интерфейс `loopback`, исходящих DNS-запросов (UDP/TCP, порт 53), исходящего ping (ICMP), входящего ping с доверенного узла 192.168.0.4, а также HTTP/HTTPS (порты 80 и 443); блокировку всего остального трафика по умолчанию. Проведённые проверки подтвердили корректность работы: разрешённый трафик успешно проходит, запрещённые соединения (SSH, PostgreSQL, входящий ping с неразрешённых адресов) блокируются. Контроль прохождения пакетов выполнен с помощью `tcpdump` и утилиты `pfctl`, что позволило верифицировать соответствие фактического поведения политике доступа.

Исходный код представлен в приложении В.

5 Разработка клиент-серверного приложения для конфиденциального обмена сообщениями

5.1 Введение

В данной работе, разработано клиент-серверное приложение для защищённого обмена короткими сообщениями (до 80 символов) по протоколу TCP. Для этого решались следующие задачи: освоение криптографических функций библиотеки OpenSSL (cryptography использует API OpenSSL), реализация обмена сообщениями, обеспечение обмена сообщениями между клиентами, а также проверка корректности через анализ трафика. В соответствии с индивидуальным заданием применяется симметричное шифрование ChaCha20 и хэш-функция Blake2B. Ключ шифрования длиной 32 байта хранится в файле с ограниченными правами доступа. Помимо базовых требований, реализованы: выработка сессионного ключа по протоколу Диффи-Хеллмана, добавление соли в хэш, аутентификация пользователей, а также поддержка широковещательной и групповой рассылки сообщений.

Актуальность темы обусловлена необходимостью защиты передаваемых данных в сетевых приложениях от перехвата и несанкционированной модификации, особенно в условиях открытых сетей.

5.2 Постановка задачи

5.2.1 Цели

Целью работы является разработка клиент-серверного приложения для конфиденциального обмена сообщениями с использованием шифрования и контроля целостности данных.

5.2.2 Задачи

Задачами решаемыми при выполнении работы являются:

- Получение базовых знаний по использованию криптографических функций библиотеки OpenSSL (cryptography с OpenSSL API);
- Разработка клиент-серверного приложения для конфиденциального обмена сообщениями с контролем целостности;
- Проверка работы приложения в различных режимах с помощью анализа трафика.

5.3 Алгоритм ChaCha20

ChaCha20 - это потоковый шифр, основанный на функции шифрования которая преобразует входное состояние с помощью последовательности раундов(см. Таблицу 1) [5]. Состояние представлено в виде матрицы 4×4 32-битных слов (16 слов, 512 бит). Начальное состояние формируется следующим образом:

- Первые 4 слова - константы в ASCII представлении строк "expand 32-byte k";
- Следующие 8 слов - 256-битный ключ $(k_0, k_1, \dots, k_7)$;
- Следующее слово - 32-битный счетчик блока;
- Последние 3 слова - 96-битный nonce.

Алгоритм выполняет 20 раундов (10 двоичных раундов). Каждый раунд состоит из применения 8 четверть-раундов: Сначала к каждому из четырех столбцов матрицы, затем к каждой из четырех диагоналей. Четверть-раунд работает с четырьмя словами и выполняет операции - $\oplus$ - XOR, $\ll$ - циклический сдвиг влево, $+$ - сложение по модулю 2^{32} .

Пусть слова - a, b, c, d . Тогда операции выполняются следующим образом [5].

- $a = a + b; d = d \oplus a; d \lll 16;$
- $c = c + d; b = b \oplus c; b \lll 12;$
- $a = a + b; d = d \oplus a; d \lll 8;$
- $c = c + d; b = b \oplus c; b \lll 7;$

После выполнения 20 раундов к полученному состоянию прибавляется по модулю 2^{32} исходное состояние. Полученные 16 слов = 64 байта составляют блок ключевого потока. Этот поток побайтно совершает $\oplus$ с открытым текстом. Для генерации следующего блока счетчик увеличивается на 1, и процесс повторяется [5].

Сам алгоритм шифрования используется в коде следующим образом:

```
1 def derive_message_key(master_key: bytes, salt: bytes) -> bytes:
2     hkdf = HKDF(
3         algorithm=hashes.SHA256(),
4         length=CHACHA_KEY_SIZE,
5         salt=salt,
6         info=b"lab5-chacha20-msg",
7     )
8     return hkdf.derive(master_key)
9
10 def chacha20_encrypt(plaintext: bytes, master_key: bytes) -> bytes:
11     salt = os.urandom(SALT_SIZE)
12     msg_key = derive_message_key(master_key, salt)
13     nonce = os.urandom(CHACHA_NONCE_SIZE)
14     cipher = Cipher(algorithms.ChaCha20(msg_key, nonce), mode=None)
15     enc = cipher.encryptor()
16     return salt + nonce + enc.update(plaintext) + enc.finalize()
17
18 def chacha20_decrypt(blob: bytes, master_key: bytes) -> bytes:
19     salt = blob[:SALT_SIZE]
20     nonce = blob[SALT_SIZE : SALT_SIZE + CHACHA_NONCE_SIZE]
21     ciphertext = blob[SALT_SIZE + CHACHA_NONCE_SIZE :]
22     msg_key = derive_message_key(master_key, salt)
23     cipher = Cipher(algorithms.ChaCha20(msg_key, nonce), mode=None)
24     dec = cipher.decryptor()
25     return dec.update(ciphertext) + dec.finalize()
```

В данной реализации используется поточный шифр ChaCha20, построенный на принципе генерации ключевого потока и побайтового XOR с открытым текстом. Для каждого сообщения генерируются криптографически стойкая соль (16 байт) и nonce (16 байт). Мастер-ключ и соль подаются в HKDF (HMAC-based Key Derivation Function), который вырабатывает уникальный 32-байтный ключ для данного сообщения. Использование HKDF с солью обеспечивает то, что при шифровании одного и того же текста разными пакетами получается разный шифртекст, что устраняет уязвимости, связанные с повторным использованием nonce. Шифр ChaCha20 создаётся через Cipher с алгоритмом ChaCha20 и режимом без дополнительной обёртки (поточковый режим). Результат шифрования формируется как конкатенация: `salt || nonce || ciphertext`. При расшифровании получатель извлекает соль и nonce из начала блока, восстанавливает тот же `msg_key` через HKDF и применяет обратное преобразование ChaCha20 к шифртексту.

5.4 Хэш-функция Blake2b

Blake2b - это криптографическая хэш-функция, оптимизированная для 64-битных платформ, которая основана на архитектуре ARX (Addition-Rotation-XOR). Математически алгоритм строится на итеративном применении функции сжатия к блокам данных [6].

Пусть размер слова $w = 64$ бита, состоянием будет матрица 4×4 состоящая из 16 слов (1024 бита). Имеется 12 раундов - итераций перемешивания. Операции - сложение по модулю 2^{64} , XOR - $\oplus$, и циклический сдвиг.

Тогда - инициализация состояние происходит следующим образом: Внутреннее состояние $v_0, \dots, v_{15}$ - инициализируется с помощью текущего значения хеша h , векторов инициализации IV , счетчика байтов t и флагов финализации f .

Тогда матрица внутреннего состояния:

$$\begin{bmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \end{bmatrix} \leftarrow \begin{bmatrix} h_0 & h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 & h_7 \\ IV_0 & IV_1 & IV_2 & IV_3 \\ IV_4 \oplus t_0 & IV_5 \oplus t_1 & IV_6 \oplus f_0 & IV_7 \oplus f_1 \end{bmatrix}$$

Сама функция перемешивания - это функция $G(a, b, c, d)$, обрабатывающая четыре 64-битных слова и сообщение m с константами $const$:

- $a = a + b + m_{const_i}$;
- $d = (d \oplus a) \ggg 32$;
- $c = c + d$;
- $b = (b \oplus c) \ggg 24$;
- $a = a + b + m_{const_{i+1}}$;
- $d = (d \oplus a) \ggg 16$;
- $c = c + d$;
- $b = (b \oplus c) \ggg 63$;

В коде функция используется следующим образом:

```

1 def blake2b_digest(data: bytes) -> bytes:
2     return hashlib.blake2b(data, digest_size=BLAKE2B_DIGEST).digest()
3
4 if use_integrity:
5     h = blake2b_digest(pt)
6     if test_bad_hash:
7         h = bytes((b + 1) % 256 for b in h)
8     inner.extend(h)
9
10 if integrity:
11     expected = blake2b_digest(pt)
12     if recv_hash is None or recv_hash != expected:
13         raise ValueError("Integrity check: invalid Blake2b")

```

Хэш-функция Blake2b применяется для обеспечения контроля целостности сообщения. При формировании пакета (функция `pack_message`) вычисляется криптографический образ открытого текста `pt` с помощью `blake2b_digest`, формируя 32-байтный дайджест. Этот дайджест добавляется к телу пакета. При приёме (функция `parse_message`) получатель заново вычисляет Blake2b от расшифрованного либо считанного открытого текста и сравнивает полученное значение с принятым хэшем. При несовпадении выбрасывается исключение «контроль целостности: неверный Blake2b», что означает обнаружение искажений или подделки данных. Использование Blake2b обеспечивает криптографическую стойкость: злоумышленник не может подобрать допустимый текст и хэш без знания ключа или без нарушения предположений о стойкости Blake2b.

5.4.1 Добавление соли

5.5 Алгоритм Диффи-Хеллмана для генерации сессионного ключа

Алгоритм Диффи-Хеллмана позволяет двум сторонам создать общий секретный ключ через незащищенный канал, при этом не передавая ключ.

В основе используется сложность вычисления дискретного логарифма:

Стороны договариваются о двух открытых числах - g - генератор и p - большое простое число.

Пусть одна сторона - Алиса генерирует число a , а другая - Боб число b . a, b — секретны [7].

Теперь Алиса вычисляет $A = g^a \mod p$, и отправляет A Бобу.

А боб вычисляет $B = g^b \mod p$, и отправляет B Алисе.

Алиса считает $K = B^a \mod p$, и Боб считает $K = A^b \mod p$.

Тогда обе стороны приходят к одному и тому же числу:

$$K = (g^b)^a = (g^a)^b = g^{ab} \mod p$$

Злоумышленник может прочесть g, p, A, B , но не может вычислить a, b значит не может вычислить K . Это потребует очень большого количества времени [7].

В коде используется следующим образом:

```
1 def _raw_pub(key: x25519.X25519PublicKey) -> bytes:
2     return key.public_bytes(
3         encoding=serialization.Encoding.Raw,
4         format=serialization.PublicFormat.Raw,
5     )
6
7 def dh_client(sock) -> bytes:
8     priv = x25519.X25519PrivateKey.generate()
9     write_frame(sock, bytes([KIND_DH_CLIENT]) + _raw_pub(priv.public_key()))
10    body = read_frame(sock)
11    if len(body) != 33 or body[0] != KIND_DH_SERVER:
12        raise ValueError("Expected DH server")
13    peer = x25519.X25519PublicKey.from_public_bytes(body[1:33])
14    return derive_session_key_from_shared_secret(priv.exchange(peer))
15
16 def dh_server(sock) -> bytes:
17    body = read_frame(sock)
18    if len(body) != 33 or body[0] != KIND_DH_CLIENT:
19        raise ValueError("Expected DH client")
20    peer = x25519.X25519PublicKey.from_public_bytes(body[1:33])
21    priv = x25519.X25519PrivateKey.generate()
22    write_frame(sock, bytes([KIND_DH_SERVER]) + _raw_pub(priv.public_key()))
23    return derive_session_key_from_shared_secret(priv.exchange(peer))
24
25 def derive_session_key_from_shared_secret(shared_secret: bytes) -> bytes:
26    hkdf = HKDF(algorithm=hashes.SHA256(), length=CHACHA_KEY_SIZE,
27                salt=None, info=b"lab5-x25519-session")
28    return hkdf.derive(shared_secret)
```

В реализации применяется протокол обмена ключами на базе эллиптических кривых - X25519 (алгоритм Диффи-Хеллмана для кривой Curve25519). Клиент генерирует пару ключей X25519 (`private_key`, `public_key`) и отправляет серверу кадр: байт-тип `KIND_DH_CLIENT` и 32 байта открытого ключа. Сервер, получив кадр, создаёт свою пару ключей, извлекает открытый

ключ клиента, вычисляет общий секрет $K = [a_{\text{srv}}] \cdot P_{\text{clt}}$ (скалярное умножение: приватный ключ сервера на публичную точку клиента) и отправляет клиенту свой открытый ключ. Клиент вычисляет $K = [a_{\text{clt}}] \cdot P_{\text{srv}}$. В силу свойств ECDH обе стороны получают один и тот же общий секрет. Далее `shared_secret` подаётся в HKDF с фиксированным `info`, что даёт 32-байтный сессионный ключ для ChaCha20. Каждое TCP-соединение получает свой сессионный ключ, что повышает безопасность и обеспечивает forward secrecy: компрометация ключа одного сеанса не затрагивает прошлые сессии.

HKDF (HMAC-based Key Derivation Function) - Это алгоритм «вытягивания» и «расширения». Он берет сырой секрет из ECDH и превращает его в один или несколько криптографически стойких ключей.

ECDH (Elliptic Curve Diffie-Hellman) - Два участника обмениваются публичными ключами и вычисляют «сырой» общий секрет. Сам по себе этот секрет нельзя использовать как ключ шифрования.

5.6 Дополнительная реализация

В программе помимо базовых требований реализованы дополнительные требования, а так же дополнительный функционал:

- Реализованы дополнительные требования:
 - Добавление соли в хэш;
 - Режим работы с сессионным ключом.
- Дополнительный функционал:
 - Аутентификация пользователей;
 - Широковещательная и групповая рассылка сообщений.

5.6.1 Структура приложения

Приложение состоит из следующих частей:

- `confdata/`
 - `key.txt` - файл для хранения ключа шифрования;
 - `passwd` - файл для хранения пользователей и их хешей с солью.
- `scripts/`
 - `connect.sh` - скрипт для подключения к серверу и аутентификации;
 - `start_server.sh` - скрипт для запуска сервера;
 - `connect_test.sh` - скрипт запуска тестового сервера без шифрования;
 - `connect_test.sh` - скрипт подключения к тестовому серверу без шифрования.
- `src/`
 - `server.py` - серверное приложение;
 - `client.py` - клиентское приложение;
 - `auth.py` - аутентификация пользователей;
 - `add_user.py` - добавление пользователей;

- crypto.py - шифрование сообщений, генерация сессионного ключа и контроль целостности данных;
- forttest/ - директория с тестовым сервером без шифрования с той же логикой работы клиент-серверного приложения.

Файлы скриптов написанные на bash, нужны только для упрощенного запуска приложения без ввода дополнительных аргументов.

Полный код представлен в приложении Г.

5.7 Примеры работы приложения

5.7.1 Без шифрования

Ниже представлен результат работы приложения без шифрования, с запуском скрипта `scripts/connect_test.sh` и `scripts/connect_test.sh`.

Для просмотра результата необходимо вызвать утилиту `TCPDump`, где будет видно отправителя и его сообщение.

Сначала Алиса подключается к тестовому серверу без шифрования и отправляет сообщение "Hello! I hope no one listen me... как показано на рисунке 63:

```
(base) monadayzek@Ayzeks-MacBook-Pro lab-5 % ./scripts/connect_test.sh -u alice -p secret
alice joined chat
Hello! I hope no one listen me...
alice : Hello! I hope no one listen me...
```

Рис. 63: Алиса подключается к тестовому серверу без шифрования и отправляет сообщение «Hello, i hope no one listen me. . . »

Далее на рисунке 64, в красных рамках, отмечено сообщение Алисы, которое легко читается, а так же видно как Алиса подключалась к серверу:

```
14:49:18.499034 IP localhost.etlservicemgr > localhost.55051: Flags [P.], seq 4:25, ack 37, win 6380, options [nop,nop,TS val 941906168 ecr 1468315994], length 21
0x0000: 4500 0049 0000 4000 4006 0000 7f00 0001  E..I..@.@.....
0x0010: 7f00 0001 2329 d70b 04f3 16fe ec5d c768  ....#.....).h
0x0020: 8018 18ec fe3d 0000 0101 080a 3824 58f8  .........85X.
0x0030: 5784 b95a 0000 0011 616c 6963 6520 6a6f  W..Z....alice.jo
0x0040: 696e 6564 2063 6061 74 0000 0000 0000  ined.chat
14:49:18.499048 IP localhost.55051 > localhost.etlservicemgr: Flags [P.], seq 25, ack 37, win 6380, options [nop,nop,TS val 1468315994 ecr 941906168], length 0
0x0000: 4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.....
0x0010: 7f00 0001 d70b 2329 ec5d c768 04f3 1713  ....#.....).h
0x0020: 8018 18ec fe28 0000 0101 080a 5784 b95a  .........W..Z
0x0030: 3824 58f8 0000 0011 616c 6963 6520 6a6f  .........85X.
14:49:29.531566 IP localhost.55051 > localhost.etlservicemgr: Flags [P.], seq 37:74, ack 25, win 6380, options [nop,nop,TS val 1468327026 ecr 941906168], length 37
0x0000: 4500 0059 0000 4000 4006 0000 7f00 0001  E..Y..@.@.....
0x0010: 7f00 0001 d70b 2329 ec5d c768 04f3 1713  ....#.....).h
0x0020: 8018 18ec fe40 0000 0101 080a 5784 e472  ....H.....W..r
0x0030: 3824 58f8 0000 0011 616c 6963 6520 6a6f  .........85X.
0x0040: 206f 6f70 6520 6e6f 206f 6e65 206c 6973  .hope.no.one.lis
0x0050: 7465 6e20 6d65 2e2e 2e 0000 0000 0000  ten.me...
14:49:29.531678 IP localhost.etlservicemgr > localhost.55051: Flags [P.], seq 74, win 6380, options [nop,nop,TS val 941917200 ecr 1468327026], length 0
0x0000: 4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.....
0x0010: 7f00 0001 2329 d70b 04f3 1713 ec5d c78d  ....#.....).h
0x0020: 8018 18ec fe28 0000 0101 080a 3824 841e  .........85.
0x0030: 5784 e472 0000 0011 616c 6963 6520 6a6f  W..r....alice.i
14:49:29.531938 IP localhost.etlservicemgr > localhost.55051: Flags [P.], seq 25:70, ack 74, win 6380, options [nop,nop,TS val 941917201 ecr 1468327026], length 45
0x0000: 4500 0061 0000 4000 4006 0000 7f00 0001  E..a..@.@.....
0x0010: 7f00 0001 2329 d70b 04f3 1713 ec5d c78d  ....#.....).h
0x0020: 8018 18ec fe55 0000 0101 080a 3824 8411  ....U.....85.
0x0030: 5784 e472 0000 0011 616c 6963 6520 6a6f  W..r....alice.i
0x0040: 4865 6e6e 6721 2040 2068 6f70 6520 6a6f  Hello!..I..hope.no
0x0050: 206f 6e65 206c 6973 7465 6e20 6d65 2e2e  .one.listen.me..
0x0060: 2e 0000 0000 0000 0000 0000 0000 0000  .
14:49:29.532052 IP localhost.55051 > localhost.etlservicemgr: Flags [P.], seq 70, win 6380, options [nop,nop,TS val 1468327027 ecr 941917201], length 0
0x0000: 4500 0034 0000 4000 4006 0000 7f00 0001  E..4..@.@.....
0x0010: 7f00 0001 d70b 2329 ec5d c78d 04f3 1740  ....#.....).@
0x0020: 8018 18ec fe28 0000 0101 080a 5784 e473  .........W..s
0x0030: 3824 8411 0000 0011 616c 6963 6520 6a6f  .........85..
```

Рис. 64: Результат работы приложения без шифрования

5.7.2 С симметричным шифрованием

Сначала нужно запустить сервер с шифрованием, с помощью скрипта `scripts/start_server.sh`, как показано на рисунке 65:

```
(base) monadayzek@Ayzeks-MacBook-Pro lab-5 % ./scripts/start_server.sh
Yaaaay! 0.0.0.0:9000
```

Рис. 65: Запуск сервера с шифрованием

Для проверки шифрования, необходим запустить скрипт `scripts/connect_test.sh` и `scripts/connect.sh`, как показано на рисунке 66:

```
(base) monadayzek@Ayzeks-MacBook-Pro lab-5 % ./scripts/connect.sh
Login: Ayzek
Password:
Ayzek joined chat
Im super programmer so u cant read my text :)
Ayzek : Im super programmer so u cant read my text :)
█
```

Рис. 66: Запуск скриптов для аутентификации и подключения к серверу

Пользователь Ayzek - отправляет сообщение "Im super programmer so u cant read my text :)". Сейчас нужно перезапустить TCPDump на порт 9000 и посмотреть на трафик, как показано на рисунке 67:

```
15:03:07.427328 IP localhost.55858 > localhost.cslistener: Flags [..], ack 59, win 6380, options [nop,nop,TS val 2384083193 ecr 3298615222], length 0
 0x0000: 4500 0034 0000 4000 4006 0000 7f00 0001 E..4..@.....
 0x0010: 7f00 0001 da32 2328 6461 fa05 c5a9 beb7 .....2#(da....
 0x0020: 8010 18ec fe28 0000 0101 080a 8e1a 4648 .....(.....FH
 0x0030: c49c dfb6 0000 004f 0101 b11b d98d bb53 .....0.....S
 0x0040: 7a5a 6858 6acb cf9e d780 8d19 7f96 5bbc zZhXj.....X.
 0x0050: b34f dd42 0aed 35cb c328 b95d daf2 068f ..0.B..S.(.)...
 0x0060: 7d44 1360 d79a a62e 9a43 6d55 2e6e f9f4 .....CmU.n...
 0x0070: 36bb 3419 1ece a73e 12e6 6883 d71f 66d9 ..04...2...f...
 0x0080: a8ae 4fcb d1fd d8 .....0.....
15:03:10.833828 IP localhost.55858 > localhost.cslistener: Flags [P..], seq 35:118, ack 59, win 6380, options [nop,nop,TS val 2384086600 ecr 3298615222], length 83
 0x0000: 4500 0087 0000 4000 4006 0000 7f00 0001 E....@.....
 0x0010: 7f00 0001 da32 2328 6461 fa05 c5a9 beb7 .....2#(da....
 0x0020: 8010 18ec fe2b 0000 0101 080a 8e1a 4648 .....(.....FH
 0x0030: c49c dfb6 0000 004f 0101 b11b d98d bb53 .....0.....S
 0x0040: 7a5a 6858 6acb cf9e d780 8d19 7f96 5bbc zZhXj.....X.
 0x0050: b34f dd42 0aed 35cb c328 b95d daf2 068f ..0.B..S.(.)...
 0x0060: 7d44 1360 d79a a62e 9a43 6d55 2e6e f9f4 .....CmU.n...
 0x0070: 36bb 3419 1ece a73e 12e6 6883 d71f 66d9 ..04...2...f...
 0x0080: a8ae 4fcb d1fd d8 .....0.....
15:03:10.833905 IP localhost.cslistener > localhost.55858: Flags [..], ack 118, win 6379, options [nop,nop,TS val 3298618629 ecr 2384086600], length 0
 0x0000: 4500 0034 0000 4000 4006 0000 7f00 0001 E..4..@.....
 0x0010: 7f00 0001 2328 da32 c5a9 beb7 6461 fa58 ....#(2....da.X
 0x0020: 8010 18eb fe28 0000 0101 080a c49c ed05 .....(.....FH
 0x0030: 8e1a 4648 .....
15:03:10.834218 IP localhost.cslistener > localhost.55858: Flags [P..], seq 59:150, ack 118, win 6379, options [nop,nop,TS val 3298618629 ecr 2384086600], length 91
 0x0000: 4500 0087 0000 4000 4006 0000 7f00 0001 E....@.....
 0x0010: 7f00 0001 2328 da32 c5a9 beb7 6461 fa58 ....#(2....da.X
 0x0020: 8010 18eb fe2b 0000 0101 080a c49c ed05 .....(.....FH
 0x0030: 8e1a 4648 0000 0057 0101 6958 0c9b b741 ..FH..W..iX...A
 0x0040: ec23 6057 9226 9d3a c371 e311 b398 5333 ..#W.6...q.1..e3
 0x0050: c90b 5502 42e3 35ea de9e bb47 f2e2 0144 ..U.B.5...G...0
 0x0060: 8a22 e354 1f22 f5c5 d31e a387 c577 2c7e ..T.....W.-
 0x0070: eec4 f868 d2b2 6385 4ec4 0eab a142 2c9e ....h..c.N...B..
 0x0080: e4d3 8515 d444 2b96 442b 20b1 c235 12 .....D+-D...S.
15:03:10.834254 IP localhost.55858 > localhost.cslistener: Flags [..], ack 150, win 6379, options [nop,nop,TS val 2384086600 ecr 3298618629], length 0
 0x0000: 4500 0034 0000 4000 4006 0000 7f00 0001 E..4..@.....
 0x0010: 7f00 0001 da32 2328 6461 fa58 c5a9 bf12 .....2#(da.X...
 0x0020: 8010 18eb fe28 0000 0101 080a 8e1a 4648 .....(.....FH
 0x0030: c49c ed05 .....
```

Рис. 67: Зашифрованный трафик

Как видно из рисунка 67, трафик зашифрован и вообще не читаемый, что подтверждает работоспособность шифрования.

5.7.3 Проверка целостности данных

Для проверки целостности данных, нужно вызвать скрипт из `src/` (`client.py`), так как необходимо передать аргумент который портит хэш перед отправкой:

Сначала Боб присоединяется к серверу, так же для проверки к серверу подключается Алиса.

Алиса отправляет сообщение "Hello, I am Alice!" так как у Алисы хеш совпадает с хешем сервера ее сообщение целостное как видно на рисунке. В ответ Боб (с неверным хешем) отправляет сообщение: "Hello Alice! I am Bob with wrong life and hash".

Однако из-за неверного хеша, Алиса получает сообщение: "Hello Alice! I am Bob wiuh wrong life and hash". Нарушена целостность данных. Так же после отправки, сервер "выкидывает" Боба из чата, с сообщением: "bob disconnected (integrity error)". На рисунке 68, показан результат теста:

```
(base) monadayzek@AyzeKS-MBP lab-5 % python src/client.py -e -i --key-file confdata/key.txt -u alice -p
Hello, i am Alice!
alice : Hello, i am Alice!
!!! Integrity FAIL from bob: 'Hello Alice! I am Bob wiuh wrong life and hash
bob disconnected (integrity error)
```

Рис. 68: Алиса видит сообщение с нарушением целостности от Боба

При этом в логировании сервера будет видно нарушение целостности данных (Integrity check: invalid Blake2b), что Боб отключился из чата не используя команду q, как показано на рисунке 69:

```
[server] new connection from ('127.0.0.1', 57214)
[('127.0.0.1', 57214)] auth OK: bob logged in
[bob] joined chat
[server] new connection from ('127.0.0.1', 57227)
[('127.0.0.1', 57227)] auth OK: alice logged in
[alice] joined chat
[alice] broadcast: 'Hello, i am Alice!'
[bob] integrity check failed: invalid Blake2b. Corrupted data received: 'Hello Alice! I am Bob wiuh wrong life and hash (('
[bob] disconnected (no \q)
```

Рис. 69: Логирование сервера при нарушении целостности данных

Сам же Боб, так же видит сообщение о нарушении целостности данных и сообщение об отключении из чата как показано на рисунке 70:

```
alice : Hello, i am Alice!
Hello Alice! I am Bob with wrong life and hash ((
!!! Integrity FAIL from bob: 'Hello Alice! I am Bob wiuh wrong life and hash
bob disconnected (integrity error)
[chat] connection closed
[]
```

Рис. 70: Боб видит сообщение о нарушении целостности данных и сообщение об отключении из чата

5.7.4 Проверка сессионного ключа

Для проверки сессионного ключа, нужно вызвать скрипт из `src/` (`server.py`), и передать ему аргумент `-dh`.

```
1 python src/server.py -e --dh
```

Как видно, мы не передаем аргумент `-key-file`, так как используем режим сессионного ключа. Аргумент `-e` - включает шифрование в целом. А аргумент `-dh` - включает режим сессионного ключа.

Из рисунка 71, видно, что Алиса и Боб написали друг другу сообщения. Так же красной стрелкой указано, что не был передан аргумент ключа:

```
(base) monadayzek@AyzeKs-MacBook-Pro lab-5 % python src/server.py -e --dh
Yaaay! 0.0.0.0:9000
Ho
[server] new connection from ('127.0.0.1', 58050)
[('127.0.0.1', 58050)] auth OK: bob logged in
[bob] joined chat
[server] new connection from ('127.0.0.1', 58070)
[('127.0.0.1', 58070)] auth OK: alice logged in
[alice] joined chat
[alice] broadcast: 'Hi i am Aliceeee'
[bob] broadcast: 'Hi im Bob'
[bob] broadcast: 'Hi Alice'
[alice] broadcast: 'Hi Bob'
```

Рис. 71: Алиса и Боб написали друг другу сообщения в режиме сессионного ключа

Сейчас нужно посмотреть на трафик, при тестовом запуске обычного сервера без шифрования, было видно сообщения в открытом виде. На рисунке 72, показан результат. Видно что данные так же зашифрованы и не читаемы, что подтверждает работу в режиме сессионного ключа:

```
0x0010: 7f00 0001 e2d6 2328 9139 37bf 7390 b0f1 .....#(.97.s...
0x0020: 8010 18ec fe28 0000 0101 080a 2f3a 291b .....(...../):.
0x0030: 8f66 91b7 .....f..
15:51:52.337801 IP localhost.58070 > localhost.cslistener: Flags [P.], seq 1:45, ack 52, win 6380, options [nop,nop,TS val 792352113 ecr 2405863863
], length 44
0x0000: 4500 0060 0000 4000 4006 0000 7f00 0001 E..h..@.....
0x0010: 7f00 0001 e2d6 2328 9139 37bf 7390 b0f1 .....#(.97.s...
0x0020: 8010 18ec fe54 0000 0101 080a 2f3a 5571 .....T...../Uq
0x0030: 8f66 91b7 0000 0028 0101 0607 783a dbec .....f.....gX:..
0x0040: 9efe 20e4 11e6 3f6f 7b36 b33c b6bc 4338 .....70(6<.<C8
0x0050: 2fa7 19e7 1edf f6de 7884 e294 dbba 5c8d .....X.....\
15:51:52.337951 IP localhost.cslistener > localhost.58070: Flags [.], ack 45, win 6380, options [nop,nop,TS val 2405875213 ecr 792352113], length 0
0x0000: 4500 0034 0000 4000 4006 0000 7f00 0001 E..4..@.....
0x0010: 7f00 0001 2328 e2d6 7390 b0f1 9139 37eb .....#(.s...97.
0x0020: 8010 18ec fe28 0000 0101 080a 8f66 be0d .....(.....f..
0x0030: 2f3a 5571 ...../Uq
15:51:52.338796 IP localhost.cslistener > localhost.58050: Flags [P.], seq 53:105, ack 46, win 6380, options [nop,nop,TS val 1945592663 ecr 3613875
160], length 52
0x0000: 4500 0060 0000 4000 4006 0000 7f00 0001 E..h..@.....
0x0010: 7f00 0001 2328 e2c2 795e 08fb bd96 4824 .....#(.y...H5
0x0020: 8010 18ec fe5c 0000 0101 080a 73f7 6357 .....(.....s.cW
0x0030: d767 5bd8 0000 0030 0101 f8a3 94d7 05c0 .....g[...0.....
0x0040: 08a1 cea0 9b2e 5740 11fd a41c cb7f 4e0b .....W0.....N.
0x0050: 1f66 02d2 211d 3f6d 52ba d062 3ce0 28b8 .....f..!7mR..b<(.
0x0060: 2180 470e 9a09 8026 .....XG...8
15:51:52.338916 IP localhost.58050 > localhost.cslistener: Flags [.], ack 105, win 6380, options [nop,nop,TS val 3613886511 ecr 1945592663], length
0
0x0000: 4500 0034 0000 4000 4006 0000 7f00 0001 E..4..@.....
0x0010: 7f00 0001 e2c2 2328 bd96 4824 795e e92f .....#(.HSy^:/
0x0020: 8010 18ec fe28 0000 0101 080a d767 802f .....(.....g./
0x0030: 73f7 6357 .....s.cW
15:51:52.338933 IP localhost.cslistener > localhost.58070: Flags [P.], seq 52:104, ack 45, win 6380, options [nop,nop,TS val 2405875214 ecr 7923521
13], length 52
0x0000: 4500 0060 0000 4000 4006 0000 7f00 0001 E..h..@.....
0x0010: 7f00 0001 2328 e2d6 7390 b0f1 9139 37eb .....#(.s...97.
0x0020: 8010 18ec fe5c 0000 0101 080a 8f66 be0d .....f.....f..
0x0030: 2f3a 5571 0000 0030 0101 c405 fdef 0933 ...../Uq..0.....3
0x0040: 99f5 f02f 507d 1c29 00d3 1fa7 8e91 614b ...../P.).....aK
0x0050: 8b68 4aff e413 a509 216a e2c4 6732 891d .....hJ.....!j..g2..
0x0060: dba6 6546 687a 70d4 .....jeFhZp.
15:51:52.338908 IP localhost.58070 > localhost.cslistener: Flags [.], ack 104, win 6380, options [nop,nop,TS val 792352114 ecr 2405875214], length
0
0x0000: 4500 0034 0000 4000 4006 0000 7f00 0001 E..4..@.....
0x0010: 7f00 0001 e2d6 2328 9139 37eb 7390 b125 .....#(.97.s.%
0x0020: 8010 18ec fe28 0000 0101 080a 2f3a 5572 .....(...../Ur
0x0030: 8f66 be0e .....f..
```

Рис. 72: Трафик в режиме сессионного ключа

5.7.5 Проверка хеша

Для проверки сессионного хеша, и показа того, что хэш генерируется с "солью можно вызвать скрипт из `src/` (`show_hash.py`), как показано на рисунке 73. Идея в том, что скрипт на короткое время подключается к серверу, получает хеш и выводит в консоль отключаясь от сервера. Соль отделяется от хеша символом "\$".

```
(base) monadayzek@AyzeKs-MBP lab-5 % python src/show_hash.py
Login: Ayzek
Password:
Your hash: 4faa556bcd5a5d6c1dc60ef2d704b0eb3f472f684b4b9c87d947b01df095fa955505639edc581b1197d65eecf49e4528b
(base) monadayzek@AyzeKs-MBP lab-5 %
```

Рис. 73: Проверка сессионного хеша

5.7.6 Общение между клиентами и группами клиентов

Для общения между клиентами, был реализован парсер, который парсит знак @, и до окончания слова считает его как пользователя.

Соответственно, если пользователь Алиса, хочет отправить сообщение только Бобу, он может написать: "@bob Hello!".

Для реализации групповой рассылки, парсер считывает все символы @ и до окончания слова считает так же как пользователя.

Если Алиса хочет отправить сообщение только Бобу и Айзеку, она может написать "Hello! @bob, @Ayzek !"

Для теста, к серверу подключаются четыре пользователя: Алиса (alice), Боб (bob), Айзек (Ayzek) и Ольга (Olga), как показано на рисунке 74:

```
[server] new connection from ('127.0.0.1', 59599)
[('127.0.0.1', 59599)] auth OK: Ayzek logged in
[Ayzek] joined chat
[server] new connection from ('127.0.0.1', 59600)
[('127.0.0.1', 59600)] auth OK: Olga logged in
[Olga] joined chat
[server] new connection from ('127.0.0.1', 59601)
[('127.0.0.1', 59601)] auth OK: alice logged in
[alice] joined chat
[server] new connection from ('127.0.0.1', 59684)
[('127.0.0.1', 59684)] auth FAIL: user='Bob'
[server] new connection from ('127.0.0.1', 59687)
[('127.0.0.1', 59687)] auth OK: bob logged in
[bob] joined chat
```

Рис. 74: Подключение четырех пользователей к серверу

Для теста, были реализованы следующие сценарии:

- Пользователь Ayzek отправляет широковещательное сообщение "Hello everyone! Im Ayzek";
- Пользователь Olga отправляет личное сообщение пользователю Ayzek: "Who is this Alice???!!!";
- Пользователь Ayzek, отправляет сообщение только пользователям Bob, Alice: "Never mind...".

На рисунке 75, показан результат теста:

```
[Ayzek] broadcast: 'Hello everyone! Im Ayzek'
[Olga] -> @Ayzek: '@Ayzek Who is this Alice???!!!'
[Ayzek] -> @alice, @bob: '@bob, @alice Never mind...'
```

Рис. 75: Общение между пользователями

Из рисунка 75, видно что Ayzek отправил широковещательное сообщение ([Ayzek] broadcast), после Olga отправила личное сообщение Ayzek ([Olga] -> @Ayzek), после чего Ayzek отправил сообщение группе из пользователей bob и alice ([Ayzek] -> @bob, @alice).

Соответственно, сообщения дошли только до тех, кому сообщения были адресованы.

Заключение

В ходе выполнения работы разработано клиент-серверное приложение для защищённого обмена сообщениями, обеспечивающее конфиденциальность и целостность передаваемых данных. Реализованы базовые требования: шифрование по алгоритму ChaCha20, контроль целостности с помощью Blake2b и обмен по TCP.

Дополнительно внедрены: выработка сессионного ключа по протоколу Диффи-Хеллмана на эллиптической кривой X25519, добавление соли в хэш для усиления стойкости, аутентификация пользователей по паролю, а также широковещательная и групповая рассылка сообщений.

Проведённая проверка с использованием анализа трафика подтвердила корректность работы всех реализованных механизмов.

Исходный код приложения, и скриптов запуска представлен в приложении Г.

Заключение

В ходе выполнения практических работ по дисциплине «Защита информации» были выполнены пять практических работ, связанных с анализом угроз, проектирования политик доступа и применения технических средств защиты.

По результатам работы №1 освоена работа с банком угроз и уязвимостей ФСТЭК: выполнена систематизация записей по заданным критериям и анализ уязвимостей программного обеспечения используемых платформ. В работах №2-№3 решены задачи построения подсистемы аутентификации и авторизации, исследования стойкости паролей к перебору, а также реализации дискреционной и мандатной моделей разграничения доступа с опорой на владение объектами, списки контроля доступа и правила, согласованные с моделью Белла-Лападулы. Работа №4 направлена на настройку межсетевого экрана (Packet Filter): сформирован набор правил фильтрации трафика с разделением разрешённых и запрещённых сценариев взаимодействия по интерфейсам и протоколам. В работе №5 разработано клиент-серверное приложение, обеспечивающее конфиденциальность и целостность сообщений за счёт симметричного шифрования, контроля целостности и криптографического согласования ключей; корректность механизмов подтверждена контролем сетевого трафика.

Итогом является приобретение практических навыков применения нормативных и отраслевых источников по угрозам, реализации политик доступа на уровне ОС и сети, а также интеграции криптографических примитивов в программное обеспечение.

Список литературы

- [1] ФСТЭК России // Банк данных угроз безопасности информации. URL: <https://bdu.fstec.ru/> (дата обращения: 22.02.2026).
- [2] DAC: Was ist Discretionary Access Control? [Электронный ресурс] // Security-Insider. - 2024. - 9 окт. - Режим доступа: <https://www.security-insider.de/discretionary-access-control-benutzerbestimmbares-zugriffsmodell-a-725723079d109135b4045f> (дата обращения: 11.03.2026).
- [3] Добычина А. В. Способ управления доступом в системах с различными реализациями мандатного механизма разграничения доступа [Электронный ресурс] / А. В. Добычина, А. Ю. Кузнецов, А. Н. Бегаев // Известия Тульского государственного университета. Технические науки. - 2019. - Режим доступа: <https://cyberleninka.ru/article/n/sposob-upravleniya-dostupom-v-sistemah-s-razlichnymi-realizatsiyami-mandatnogo-mehanizma> (дата обращения: 11.03.2026).
- [4] Iyan. Setting up correctly Packet Filter (pf) firewall on any macOS (from Sierra to Big Sur) [Электронный ресурс] // Medium. - 2021. - 23 мар. - Режим доступа: <https://iyanmv.medium.com/setting-up-correctly-packet-filter-pf-firewall-on-any-macos-from-sierra-to-big-sur-47e70> (дата обращения: 18.03.2026).
- [5] Bernstein D. J. ChaCha, a variant of Salsa20 / D. J. Bernstein // Workshop Record of SASC. - 2008. - С. 3-7.
- [6] Guo J. Analysis of BLAKE2 / J. Guo, P. Karpman, I. Nikolić, L. Wang, S. Wu // Topics in Cryptology - CT-RSA 2016. - Springer, 2016. - С. 402-421. - DOI: 10.1007/978-3-319-29485-8_23.
- [7] Diffie-Hellman key exchange [Электронный ресурс] // Wikipedia. - Режим доступа: https://en.wikipedia.org/wiki/Diffie-Hellman_key_exchange (дата обращения: 23.03.2026).

access.py

```
1  #!/usr/bin/env python3
2  import os
3  import sys
4  import getpass
5  import hashlib
6  import logging
7  import shutil
8  from config import PASSWD_FILE, CONFDATA_DIR, ACCESS_LOG, LOG_DIR,
9      HASH_ALGO, PASSWORD_ALPHABET, FIRST_CHAR_ALPHABET
10
11  _SAVED_EUID = None
12
13  # Temporarily raise EUID to the owner of PASSWD_FILE (root in lab setup)
14  # Real UID (UID) is not changed, only EUID.
15  def elevate():
16      global _SAVED_EUID
17      if _SAVED_EUID is not None:
18          return
19      try:
20          st = os.stat(PASSWD_FILE)
21      except OSError:
22          return
23      target_euid = st.st_uid
24      current_euid = os.geteuid()
25      if current_euid == target_euid:
26          return
27      _SAVED_EUID = current_euid
28      try:
29          os.seteuid(target_euid)
30      except PermissionError:
31          _SAVED_EUID = None
32
33
34  # Drop temporary elevated EUID back to the original one.
35  def drop():
36      global _SAVED_EUID
37      if _SAVED_EUID is None:
38          return
39      try:
40          os.seteuid(_SAVED_EUID)
41      finally:
42          _SAVED_EUID = None
43
44
45  # Return EUID to the real user (after setuid login).
46  def drop_to_real():
47      if os.geteuid() != os.getuid():
48          try:
49              os.seteuid(os.getuid())
50          except (PermissionError, OSError):
51              pass
52
53
54  def hash_password(password):
```

```

55     h = hashlib.new(HASH_ALGO)
56     h.update(password.encode('utf-8'))
57     return h.hexdigest() # hash!!!!
58
59     # Check access to passwd file and explain why it's empty.
60     # With sudo, euid=0 (root) so the file can be read.
61     # Without sudo, euid is normal user and file is 0600 root, access denied
62
63     def _check_passwd_access():
64         uid, euid = os.getuid(), os.geteuid()
65         path = PASSWD_FILE
66         exists = os.path.exists(path)
67         err_no_elevate = None
68         try:
69             with open(path, "r") as f:
70                 f.read(1)
71         except Exception as e:
72             err_no_elevate = e
73         log_lines = [
74             f" uid={uid} euid={euid} (euid=0 needed to read root file)",
75             f" passwd={path} exists={exists}",
76         ]
77         if err_no_elevate:
78             log_lines.append(f" error opening: {type(err_no_elevate).__name__}: {err_no_elevate}")
79             log_lines.append(" 600")
80         else:
81             log_lines.append(" yeeeeeeesssss")
82             log_lines.append(" ---")
83             text = "\n".join(log_lines) + "\n"
84             print(text, file=sys.stderr, flush=True)
85         try:
86             with open("/tmp/access_debug.log", "a") as f:
87                 f.write(text)
88         except Exception:
89             pass
90
91
92     # Read file passwd and return {login: data}
93     def read_passwd():
94         users = {}
95         elevate()
96         try:
97             if not os.path.exists(PASSWD_FILE):
98                 return users
99             with open(PASSWD_FILE, 'r') as f:
100                 for line in f:
101                     line = line.strip()
102                     if not line or line.startswith('#'):
103                         continue
104                     parts = line.split(':', 4)
105                     if len(parts) != 5:
106                         continue
107                     login, pwd_hash, uid, perms, fullname = parts
108                     try:
109                         uid_int = int(uid)
110                     except ValueError:
111                         continue
112                     users[login] = {
113                         'hash': pwd_hash,

```



```

114         'uid': uid_int,
115         'perms': perms,
116         'fullname': fullname
117     }
118 except Exception:
119     pass
120 finally:
121     drop()
122 return users
123
124
125 def write_passwd(users):
126     elevate()
127     try:
128         with open(PASSWD_FILE, 'w') as f:
129             for login, u in users.items():
130                 f.write(f"{login}:{u['hash']}:{u['uid']}:{u['perms']}:{u['fullname']}\n")
131     try:
132         os.chmod(PASSWD_FILE, 0o600)
133     except OSError:
134         pass
135 finally:
136     drop()
137
138 def validate_password(password):
139     if len(password) < 1:
140         return False, "Cannot be empty"
141     if password[0] not in FIRST_CHAR_ALPHABET:
142         return False, "First character must be a letter"
143     for ch in password:
144         if ch not in PASSWORD_ALPHABET:
145             return False, f"Invalid character: {ch}"
146     return True, "OK"
147
148 # Asking for login and pass. Return user data, login | none, none
149 def authenticate(users):
150     while True:
151         login = input("Login: ").strip()
152         if not login:
153             continue
154         password = getpass.getpass("Password: ")
155         user = users.get(login)
156         if user and user['hash'] == hash_password(password):
157             return user, login
158         print("Oops! Wrong login or password. Try again:")
159
160 def check_perms(user_perms, required):
161     return all(r in user_perms for r in required) # Check user have
162         perms
163
164 def cmd_read(args, user_login, user_perms, user_fullname):
165     if not args:
166         print("read <filename>")
167         return
168     filename = args[0]
169     # Secure (to not out from confdata/)
170     if os.path.isabs(filename) or '..' in filename.split(os.sep):
171         print("Wrong name!")
172         return
173     filepath = os.path.join(CONFDATA_DIR, filename)

```

```

173     if not os.path.exists(filepath):
174         print(f"Not found: {filename}")
175         return
176     if not check_perms(user_perms, 'r'):
177         print("U have not permission to read (")
178         logging.warning(f"User {user_login} try to read {filename}
179             without r-perms")
180         return
181     try:
182         with open(filepath, 'r') as f:
183             content = f.read()
184             print(content)
185             logging.info(f"User {user_login} readed file {filename}")
186     except Exception as e:
187         print(f"Error: {e}")
188
189 def cmd_append(args, user_login, user_perms, user_fullname):
190     if not args:
191         print("append <filename>")
192         return
193     filename = args[0]
194     if os.path.isabs(filename) or '..' in filename.split(os.sep):
195         print("Oops! Wrong name")
196         return
197     filepath = os.path.join(CONFDATA_DIR, filename)
198     if not os.path.exists(filepath):
199         print(f"File {filename} not exist")
200         return
201     if not check_perms(user_perms, 'w'):
202         print("U have no perms for write (")
203         logging.warning(f"User {user_login} try to write(append) in {
204             filename} without perms!")
205         return
206     print("Enter your text (Empty enter to save):")
207     lines = []
208     while True:
209         line = input()
210         if line == "":
211             break
212         lines.append(line)
213     if not lines:
214         print("No data to add!")
215         return
216     try:
217         with open(filepath, 'a') as f:
218             for line in lines:
219                 f.write(line + '\n')
220             print("Added !")
221             logging.info(f"User {user_login} added data to file {filename}")
222     except Exception as e:
223         print(f"Error: {e}")
224
225 def cmd_copy(args, user_login, user_perms, user_fullname):
226     if len(args) != 2:
227         print("copy <source> <dest>")
228         return
229     src, dst = args
230     if '..' in src.split(os.sep) or '..' in dst.split(os.sep):
231         print("Wrong file name")
232         return
233     if not check_perms(user_perms, 'rw'):

```

```

232         print("You should have 'read' and 'write' perms to copy (create)
           ")
233         logging.warning(f"User {user_login} try copy without perms")
234         return
235
236     src_path = src if os.path.isabs(src) else os.path.join(CONFDATA_DIR,
237                                                             src)
238     dst_path = dst if os.path.isabs(dst) else os.path.join(CONFDATA_DIR,
239                                                             dst)
240
241     if not os.path.exists(src_path):
242         print(f"File {src} not exists")
243         return
244     if os.path.exists(dst_path):
245         print(f"File {dst} already exists. No perms to rewrite!!!")
246         return
247
248     src_in_conf = os.path.abspath(src_path).startswith(os.path.abspath(
249         CONFDATA_DIR))
250     dst_in_conf = os.path.abspath(dst_path).startswith(os.path.abspath(
251         CONFDATA_DIR))
252
253     if not dst_in_conf:
254         print("Error!!! Out of confdata/ folder")
255         return
256
257     try:
258         shutil.copy2(src_path, dst_path)
259         print(f"File copied to {dst}")
260         logging.info(f"User {user_login} copied {src} to {dst}")
261     except Exception as e:
262         print(f"Error: {e}")
263
264 def cmd_remove(args, user_login, user_perms, user_fullname):
265     if not args:
266         print("remove <filename>")
267         return
268     filename = args[0]
269     if os.path.isabs(filename) or '..' in filename.split(os.sep):
270         print("Wrong file name")
271         return
272     filepath = os.path.join(CONFDATA_DIR, filename)
273     if not os.path.exists(filepath):
274         print(f"File {filename} not exists")
275         return
276     if not check_perms(user_perms, 'd'):
277         print("U have not perms to delete files (")
278         logging.warning(f"User {user_login} try to delete {filename}
279             without permissions")
280         return
281     try:
282         os.remove(filepath)
283         print(f"File {filename} deleted")
284         logging.info(f"User {user_login} deleted file {filename}")
285     except Exception as e:
286         print(f"Error: {e}")
287
288 def cmd_perm(args, user_login, user_perms, user_fullname):
289     desc = []
290     if 'r' in user_perms:
291         desc.append("r (read)")

```

```

287     if 'w' in user_perms:
288         desc.append("w (write/append)")
289     if 'd' in user_perms:
290         desc.append("d (delete)")
291     if not desc:
292         desc.append("none")
293     print("Your permissions:", ", ".join(desc))
294     logging.info(f"User {user_login} viewed their permissions: {
        user_perms}")
295
296 def cmd_edit_my_info(login, user_fullname):
297     users = read_passwd()
298     if login not in users:
299         print("User not found.")
300         return user_fullname
301     u = users[login]
302     print("Edit your info (press Enter to keep current).")
303     new_fullname = input(f"Full name (FIO) [{user_fullname}]: ").strip()
304     if new_fullname:
305         u['fullname'] = new_fullname
306         user_fullname = new_fullname
307     change_pwd = input("Change password? (y/n): ").strip().lower()
308     if change_pwd == 'y':
309         password = getpass.getpass("New password: ")
310         password_confirm = getpass.getpass("Confirm password: ")
311         if password != password_confirm:
312             print("Passwords do not match.")
313             return user_fullname
314         valid, msg = validate_password(password)
315         if not valid:
316             print(f"Invalid password: {msg}")
317             return user_fullname
318         u['hash'] = hash_password(password)
319         print("Password updated.")
320     write_passwd(users)
321     logging.info(f"User {login} updated their info (password and/or
        fullname)")
322     print("Info updated. Changes are applied everywhere.")
323     return user_fullname
324
325 def cmd_help(args):
326     print("We have here:")
327     print()
328     print("read <filename>")
329     print("append <filename>")
330     print("copy <source> <dest>")
331     print("remove <filename>")
332     print("perm - permissions")
333     print("edit my info")
334     print("exit")
335     print("help")
336
337 def _setup_logging():
338     elevate()
339     try:
340         logging.basicConfig(filename=ACCESS_LOG, level=logging.INFO,
341                             format='%(asctime)s - %(levelname)s - %(
                                message)s')
342         os.makedirs(LOG_DIR, mode=0o700, exist_ok=True)
343         if os.path.exists(ACCESS_LOG):
344             os.chmod(ACCESS_LOG, 0o600)

```

```

345     except (PermissionError, OSError):
346         logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(
            levelname)s - %(message)s',
            stream=sys.stderr, force=True)
348     # not drop EUID here it will be dropped after read_passwd()
349
350
351 def main():
352     # sanity check: when running via setuid binary, uid should be user
        and euid should be 0
353     u, e = os.getuid(), os.geteuid()
354     print(f"uid={u} euid={e}", file=sys.stderr, flush=True)
355     _setup_logging()
356     try:
357         if not os.path.exists(CONFDATA_DIR):
358             elevate()
359             try:
360                 os.makedirs(CONFDATA_DIR, mode=0o700, exist_ok=True)
361             finally:
362                 drop_to_real()
363     except (PermissionError, OSError) as e:
364         print(f"Error: {e}")
365         sys.exit(1)
366
367     users = read_passwd()
368     if not users:
369         _check_passwd_access()
370         print("Not init. No users.")
371         sys.exit(1)
372
373     try:
374         user, login = authenticate(users)
375     except KeyboardInterrupt:
376         print("\n Exited by ^C")
377         sys.exit(0)
378
379     user_perms = user['perms']
380     user_fullname = user['fullname']
381
382     print(f"Info-Protection Lab2, {user_fullname}")
383     print("use help")
384
385     while True:
386         try:
387             cmd_line = input("> ").strip()
388             if not cmd_line:
389                 continue
390             parts = cmd_line.split()
391             cmd = parts[0].lower()
392             args = parts[1:]
393
394             if cmd == 'exit':
395                 break
396             elif cmd == 'help':
397                 cmd_help(args)
398             elif cmd == 'read':
399                 cmd_read(args, login, user_perms, user_fullname)
400             elif cmd == 'append':
401                 cmd_append(args, login, user_perms, user_fullname)
402             elif cmd == 'copy':
403                 cmd_copy(args, login, user_perms, user_fullname)

```

```

404         elif cmd == 'remove':
405             cmd_remove(args, login, user_perms, user_fullname)
406         elif cmd == 'perm':
407             cmd_perm(args, login, user_perms, user_fullname)
408         elif cmd == 'edit' and len(args) >= 2 and args[0] == 'my'
409             and args[1] == 'info':
410             user_fullname = cmd_edit_my_info(login, user_fullname)
411         else:
412             print(f"Unknown command: {cmd}")
413     except KeyboardInterrupt:
414         print("\n ^C exited")
415         break
416     except Exception as e:
417         print(f"Error: {e}")
418
419 if __name__ == "__main__":
420     main()

```

run_access.c

```

1  #include <unistd.h>
2  #include <stdio.h>
3
4  #define PYTHON3 "/usr/bin/python3"
5  #define DEFAULT_SCRIPT "/Users/practice2/bin/access.py"
6
7  #define MAX_ARGS 64
8  int main(int argc, char *argv[])
9  {
10     const char *script = (argc >= 2) ? argv[1] : DEFAULT_SCRIPT;
11     char *args[MAX_ARGS];
12     int i, n = 0;
13     args[n++] = "python3";
14     args[n++] = (char *)script;
15     for (i = 2; i < argc && n < MAX_ARGS - 1; i++)
16         args[n++] = argv[i];
17     args[n] = NULL;
18     if (execv(PYTHON3, args) == -1)
19     {
20         perror("execv");
21         return 1;
22     }
23     return 0;
24 }

```

run_access

```

1  #!/bin/sh
2  exec /Users/practice2/bin/run_access /Users/practice2/bin/crack.py "$@"

```

crack.py

```

1  #!/usr/bin/env python3
2  import os
3  import sys
4  import time
5  import hashlib
6  import itertools
7  import subprocess
8  from config import PASSWD_FILE, HASH_ALGO, PASSWORD_ALPHABET,
9     FIRST_CHAR_ALPHABET

```

```

9
10 # 8 hours
11 MAX_BRUTE_TIME_SEC = 8 * 3600
12
13 _SAVED_EUID = None
14
15 # Temporarily raise EUID to owner of passwd file (root) to read it.
16 def elevate():
17     global _SAVED_EUID
18     if _SAVED_EUID is not None:
19         return
20     try:
21         st = os.stat(PASSWD_FILE)
22     except OSError:
23         return
24     target_euid = st.st_uid
25     if os.geteuid() == target_euid:
26         return
27     _SAVED_EUID = os.geteuid()
28     try:
29         os.seteuid(target_euid)
30     except PermissionError:
31         _SAVED_EUID = None
32
33 # Restore EUID back to original value.
34 def drop():
35     global _SAVED_EUID
36     if _SAVED_EUID is None:
37         return
38     try:
39         os.seteuid(_SAVED_EUID)
40     finally:
41         _SAVED_EUID = None
42
43 def hash_password(password):
44     h = hashlib.new(HASH_ALGO)
45     h.update(password.encode('utf-8'))
46     return h.hexdigest() # hash (passwd)
47
48 # Read passwd file and return map {login: hash} (elevate/drop like in
49 # access.py)
50 def read_passwd():
51     users = {}
52     elevate()
53     try:
54         if not os.path.exists(PASSWD_FILE):
55             return users
56         with open(PASSWD_FILE, 'r') as f:
57             for line in f:
58                 line = line.strip()
59                 if not line or line.startswith('#'):
60                     continue
61                 parts = line.split(':', 4)
62                 if len(parts) != 5:
63                     continue
64                 login, pwd_hash = parts
65                 users[login] = pwd_hash
66     finally:
67         drop()
68     return users # {login : hash}

```

```

69 # Count for L = 52 * 72^{L-1}
70 def max_iterations_for_length(length):
71     if length < 1:
72         return 0
73     n_first = len(FIRST_CHAR_ALPHABET)
74     n_rest = len(PASSWORD_ALPHABET)
75     return n_first * (n_rest ** (length - 1))
76
77 # From 1 to L = Sum L
78 def total_max_iterations_up_to(max_length):
79     return sum(max_iterations_for_length(L) for L in range(1, max_length
80     + 1))
81
82 # from 1-8
83 def generate_passwords(max_length):
84     for length in range(1, max_length + 1):
85         if length == 1:
86             for ch in FIRST_CHAR_ALPHABET: # [a-zA-Z]
87                 yield ch
88         else:
89             for first in FIRST_CHAR_ALPHABET: # forall [a-zA-Z].join([a-
90                 zA-Z0-9!@$$%^&*()_+])
91                 for rest in itertools.product(PASSWORD_ALPHABET, repeat=
92                     length - 1):
93                     yield first + ''.join(rest)
94
95 # Call run_access utility with login and password
96 def call_run_access(login, password):
97     try:
98         # Prepare input for interactive access.py: login and enter key,
99         password and enter key, exit and enter key
100         stdin_input = f"{login}\n{password}\nexit\n"
101         result = subprocess.run(
102             ["/run_access"],
103             input=stdin_input,
104             capture_output=True,
105             text=True,
106             timeout=10
107         )
108         print("\n=== Calling run_access ===")
109         print(f"Login: {login}")
110         print(f>Password: {password}")
111         print("Output:")
112         if result.stdout:
113             print(result.stdout)
114         if result.stderr:
115             print("Stderr:", result.stderr)
116         return True
117     except subprocess.TimeoutExpired:
118         print("run_access timeout")
119         return False
120     except Exception as e:
121         print(f"Error calling run_access: {e}")
122         return False
123
124 # Statistics
125 def brute_force(login, target_hash, max_length):
126     total_attempts = 0
127     start_time = time.time()
128     for password in generate_passwords(max_length):

```



```

126         if time.time() - start_time > MAX_BRUTE_TIME_SEC:
127             elapsed = time.time() - start_time
128             return None, total_attempts, elapsed, True # timeout
129         total_attempts += 1
130         if hash_password(password) == target_hash:
131             elapsed = time.time() - start_time
132             return password, total_attempts, elapsed, False
133     elapsed = time.time() - start_time
134     return None, total_attempts, elapsed, False
135
136 def main():
137     if len(sys.argv) < 2:
138         print("use:")
139         print("crack.py LOGIN LENGHT - Permutation")
140         print("crack.py --max-iterations - max iterations for 3-8")
141         sys.exit(1)
142
143     if sys.argv[1] == "--max-iterations":
144         print("Max iterations by length:")
145         print("Length , For Length , Sum from 1...L")
146         print("-" * 10)
147         for L in range(3, 9):
148             m = max_iterations_for_length(L)
149             total = total_max_iterations_up_to(L)
150             print(f"{L}, {m}, {total}")
151         sys.exit(0)
152
153     if len(sys.argv) != 3:
154         print("use: crack.py <login> <length>")
155         sys.exit(1)
156
157     login = sys.argv[1]
158     try:
159         max_length = int(sys.argv[2])
160     except ValueError:
161         print("Should be > 0")
162         sys.exit(1)
163
164     try:
165         users = read_passwd()
166     except (PermissionError, OSError) as e:
167         print(f"Error: {e}")
168         sys.exit(1)
169     if login not in users:
170         print(f"User {login} not found")
171         sys.exit(1)
172
173     target_hash = users[login]
174     max_iter = total_max_iterations_up_to(max_length)
175     print(f"Generating for login {login}, max length: {max_length}")
176     print(f"(In theory): iterations (1..{max_length}): {max_iter}")
177     found, attempts, elapsed, timeout = brute_force(login, target_hash,
178                                                     max_length)
179
180     if timeout:
181         print("Interrupt: more than 8 hours")
182     if found:
183         print(f"Yes! Found password: {found}")
184         print(f"Count of attempts: {attempts}")
185         print(f"Time spend: {elapsed:.2f} s ({elapsed / 3600:.2f} H)")
186         # Call run_access utility with found credentials

```

```

186         call_run_access(login, found)
187     else:
188         print(f"Not found with {attempts} attempts")
189         print(f"Time spendened: {elapsed:.2f} s ({elapsed / 3600:.2f} H)")
190
191 if __name__ == "__main__":
192     main()

```

user_manager.py

```

1  #!/usr/bin/env python3
2  import os
3  import sys
4  import getpass
5  import hashlib
6  import logging
7  from config import PASSWD_FILE, AUTH_LOG, LOG_DIR, HASH_ALGO,
8      PASSWORD_ALPHABET, FIRST_CHAR_ALPHABET
9
10 # Logging for Authorization
11 logging.basicConfig(filename=AUTH_LOG, level=logging.INFO,
12                     format='%(asctime)s - %(levelname)s - %(message)s')
13
14 def hash_password(password):
15     h = hashlib.new(HASH_ALGO)
16     h.update(password.encode('utf-8'))
17     return h.hexdigest() # return hash
18
19 # Correct password (char + ASCII)
20 def validate_password(password):
21     if len(password) < 1:
22         return False, "Cant be empty!!!"
23     first = password[0]
24     if first not in FIRST_CHAR_ALPHABET:
25         return False, "First letter should be Letter"
26     for ch in password:
27         if ch not in PASSWORD_ALPHABET:
28             return False, f"Cant use: {ch}, only UTF-8"
29     return True, "OK"
30
31 # Read file passwd and return LIST(MAP(USERS)) [{Str:Data}, {}, ...]
32 def read_passwd():
33     users = []
34     if not os.path.exists(PASSWD_FILE):
35         return users
36     with open(PASSWD_FILE, 'r') as f:
37         for line in f:
38             line = line.strip()
39             if not line or line.startswith('#'):
40                 continue
41             parts = line.split(':', 4)
42             if len(parts) != 5:
43                 continue
44             login, pwd_hash, uid, perms, fullname = parts
45             try:
46                 uid_int = int(uid)
47             except ValueError:
48                 continue
49             users.append({
50                 'login': login,
51                 'hash': pwd_hash,

```

```

51         'uid': uid_int,
52         'perms': perms,
53         'fullname': fullname
54     })
55     return users
56
57 # Append list(map(users)) to passwd - ROOT!
58 def write_passwd(users):
59     with open(PASSWD_FILE, 'w') as f:
60         for u in users:
61             f.write(f"{u['login']}:{u['hash']}:{u['uid']}:{u['perms']}:{u['fullname']}\n")
62
63     try:
64         os.chmod(PASSWD_FILE, 0o600)
65     except OSError:
66         pass
67
68 # UID : 1000 + 1 + 1 + ... + 1
69 def get_next_uid(users):
70     if not users:
71         return 1000
72     return max(u['uid'] for u in users) + 1
73
74 # New user
75 def add_user():
76     print("Adding new user")
77     login = input("Login: ").strip()
78     if not login:
79         print("Cant be empty!")
80         return
81     users = read_passwd()
82     if any(u['login'] == login for u in users):
83         print("User with this login already exists.")
84         a = input("Edit this user? (y/n): ").strip().lower()
85         if a == 'y':
86             _edit_user_by_login(login)
87         return
88
89     fullname = input("LnFnSn: ").strip()
90     if not fullname:
91         print("LnFnSn cant be empty")
92         return
93
94     perms = input("Permissions: (r - read, w - write, d - delete (can combine)): ").strip()
95     if not all(ch in 'rwd' for ch in perms):
96         print(f"Oops! Only: r, w, d")
97         return
98
99     password = getpass.getpass("Password: ")
100     password_confirm = getpass.getpass("Confirm password: ")
101     if password != password_confirm:
102         print("Oops! Isn't same")
103         return
104
105     valid, msg = validate_password(password)
106     if not valid:
107         print(f"Invalid pass!!!: {msg}")
108         return
109
110     pwd_hash = hash_password(password)

```

```

110     uid = get_next_uid(users)
111     users.append({
112         'login': login,
113         'hash': pwd_hash,
114         'uid': uid,
115         'perms': perms,
116         'fullname': fullname
117     })
118     write_passwd(users)
119     logging.info(f"Added {login} (UID={uid})")
120     print("User added")
121
122 # Edit user (internal: login already known)
123 def _edit_user_by_login(login):
124     users = read_passwd()
125     user = next((u for u in users if u['login'] == login), None)
126     if not user:
127         print("Oops! Not found")
128         return
129
130     print("Keep empty to make no change")
131     new_fullname = input(f"LFS(FIO) ({user['fullname']}): ").strip()
132     if new_fullname:
133         user['fullname'] = new_fullname
134
135     new_perms = input(f"Permissions: ({user['perms']}): ").strip()
136     if new_perms:
137         if not all(ch in 'rwd' for ch in new_perms):
138             print("Oops! Perms only: r, w, d")
139             return
140         user['perms'] = new_perms
141
142     change_password = input("Change password? (y/n): ").strip().lower()
143     if change_password == 'y':
144         password = getpass.getpass("New password: ")
145         password_confirm = getpass.getpass("Confirm password: ")
146         if password != password_confirm:
147             print("Oops! Not the same")
148             return
149         valid, msg = validate_password(password)
150         if not valid:
151             print(f"Invalid pass: {msg}")
152             return
153         user['hash'] = hash_password(password)
154
155     write_passwd(users)
156     logging.info(f"Changed user {login}")
157     print("User changed")
158
159
160 def edit_user():
161     login = input("Enter login of user: ").strip()
162     if not login:
163         print("Cant be empty!")
164         return
165     _edit_user_by_login(login)
166
167
168 # Delete
169 def delete_user():
170     login = input("Enter user login: ").strip()

```

```

171     users = read_passwd()
172     user = next((u for u in users if u['login'] == login), None)
173     if not user:
174         print("User not found")
175         return
176
177     confirm = input(f"Are u sure to delete bro: {login}? (y/n): ").strip()
178     confirm = confirm.lower()
179     if confirm != 'y':
180         print("Canceled")
181         return
182
183     uid = user['uid']
184     users.remove(user)
185     write_passwd(users)
186     logging.info(f"User deleted {login} (UID={uid})")
187     print("Deleted user")
188
189 def main():
190     try:
191         if os.geteuid() != 0:
192             print("Oops! You should be root. Or use sudo python3 user_manager.py ...")
193             sys.exit(1)
194         os.makedirs(LOG_DIR, mode=0o700, exist_ok=True)
195         if os.path.exists(AUTH_LOG):
196             os.chmod(AUTH_LOG, 0o600)
197     except (PermissionError, OSError) as e:
198         if getattr(e, 'errno', None) == 13 or isinstance(e, PermissionError):
199             print("Oops! You should be root. Use with sudo.")
200         else:
201             print(f"Error: {e}")
202             sys.exit(1)
203
204     if len(sys.argv) < 2:
205         print("user_manager.py {add|edit|delete}")
206         sys.exit(1)
207
208     command = sys.argv[1].lower()
209     if command == 'add':
210         add_user()
211     elif command == 'edit':
212         edit_user()
213     elif command == 'delete':
214         delete_user()
215     else:
216         print(f"Unknow command: {command}")
217         sys.exit(1)
218
219 if __name__ == "__main__":
220     main()

```

config.py

```

1 import os
2
3 # base tree
4 BASE_DIR = "/Users/practice2"
5 PASSWD_FILE = os.path.join(BASE_DIR, "etc", "passwd")
6 CONFDATA_DIR = os.path.join(BASE_DIR, "confdata")

```

```

7  BIN_DIR = os.path.join(BASE_DIR, "bin")
8  LOG_DIR = os.path.join(BASE_DIR, "log")
9  AUTH_LOG = os.path.join(LOG_DIR, "auth.log")
10 ACCESS_LOG = os.path.join(LOG_DIR, "access.log")
11
12 # Algohash
13 HASH_ALGO = 'sha256'
14
15 # ASCII + 1 is char
16 PASSWORD_ALPHABET = "
    abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$
    %^&*()"
17 FIRST_CHAR_ALPHABET = "
    abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

```

access_DAC.py

```
1 import os
2 import sys
3 import getpass
4 import hashlib
5 import logging
6 import json
7 from config_DAC import (PASSWD_FILE, CONFDATA_DIR, ACCESS_LOG, LOG_DIR,
8                          HASH_ALGO, PASSWORD_ALPHABET, FIRST_CHAR_ALPHABET,
9                          ACL_FILE)
10
11 _SAVED_EUID = None
12
13
14 # Temporarily raise EUID to the owner of PASSWD_FILE (root in lab setup).
15 # Real UID (UID) is not changed.
16 def elevate():
17     global _SAVED_EUID
18     if _SAVED_EUID is not None:
19         return
20     try:
21         st = os.stat(PASSWD_FILE)
22     except OSError:
23         return
24     target_euid = st.st_uid
25     current_euid = os.geteuid()
26     if current_euid == target_euid:
27         return
28     _SAVED_EUID = current_euid
29     try:
30         os.seteuid(target_euid)
31     except PermissionError:
32         _SAVED_EUID = None
33
34
35 # Drop temporary elevated EUID back to the original one.
36 def drop():
37     global _SAVED_EUID
38     if _SAVED_EUID is None:
39         return
40     try:
41         os.seteuid(_SAVED_EUID)
42     finally:
43         _SAVED_EUID = None
44
45
46 # Return EUID to the real user (after setuid login).
47 def drop_to_real():
48     if os.geteuid() != os.getuid():
49         try:
50             os.seteuid(os.getuid())
51         except (PermissionError, OSError):
52             pass
53
54
55 def _check_passwd_access():
56     uid, euid = os.getuid(), os.geteuid()
```

```

57 path = PASSWD_FILE
58 exists = os.path.exists(path)
59 err_no_elevate = None
60 try:
61     with open(path, "r") as f:
62         f.read(1)
63 except Exception as e:
64     err_no_elevate = e
65 log_lines = [
66     "",
67     f" uid={uid} euid={euid} (euid=0 needed to read root file)",
68     f" passwd={path} exists={exists}",
69 ]
70 if err_no_elevate:
71     log_lines.append(f" error opening: {type(err_no_elevate).__name__}: {err_no_elevate}")
72     log_lines.append(" 600 - access denied.")
73 else:
74     log_lines.append(" file opened.")
75 log_lines.append("---")
76 text = "\n".join(log_lines) + "\n"
77 print(text, file=sys.stderr, flush=True)
78 try:
79     with open("/tmp/access_debug.log", "a") as f:
80         f.write(text)
81 except Exception:
82     pass
83
84
85 def hash_password(password):
86     h = hashlib.new(HASH_ALGO)
87     h.update(password.encode('utf-8'))
88     return h.hexdigest()
89
90 # DAC format: login:hash:uid:fullname
91 def read_passwd():
92     users = {}
93     elevate()
94     try:
95         if not os.path.exists(PASSWD_FILE):
96             return users
97         with open(PASSWD_FILE, 'r') as f:
98             for line in f:
99                 line = line.strip()
100                 if not line or line.startswith('#'):
101                     continue
102                 parts = line.split(':', 4)
103                 if len(parts) != 5:
104                     continue
105                 login, pwd_hash, uid, sec_level, fullname = parts
106                 try:
107                     uid_int = int(uid)
108                 except ValueError:
109                     continue
110                 users[login] = {
111                     'hash': pwd_hash,
112                     'uid': uid_int,
113                     'security_level': sec_level,
114                     'fullname': fullname
115                 }
116     except Exception:

```



```

117         pass
118     finally:
119         drop()
120     return users
121
122
123 def write_passwd(users):
124     elevate()
125     try:
126         with open(PASSWD_FILE, 'w') as f:
127             for login, u in users.items():
128                 f.write(f"{login}:{u['hash']}:{u['uid']}:{u.get('security_level', '0')}:{u['fullname']}\n")
129     try:
130         os.chmod(PASSWD_FILE, 0o600)
131     except OSError:
132         pass
133     finally:
134         drop()
135
136
137 def validate_password(password):
138     if len(password) < 1:
139         return False, "Cannot be empty"
140     if password[0] not in FIRST_CHAR_ALPHABET:
141         return False, "First character must be a letter"
142     for ch in password:
143         if ch not in PASSWORD_ALPHABET:
144             return False, f"Invalid character: {ch}"
145     return True, "OK"
146
147
148 # JSON: { "filename": { "owner": "login", "acl": { "login": "rwd", ... } } }
149 def read_acl():
150     if not os.path.exists(ACL_FILE):
151         return {}
152     try:
153         with open(ACL_FILE, 'r') as f:
154             return json.load(f)
155     except (json.JSONDecodeError, IOError):
156         return {}
157
158 def write_acl(acl_data):
159     with open(ACL_FILE, 'w') as f:
160         json.dump(acl_data, f, indent=2, ensure_ascii=False)
161     try:
162         os.chmod(ACL_FILE, 0o600)
163     except OSError:
164         pass
165
166 def authenticate(users):
167     while True:
168         login = input("Login: ").strip()
169         if not login:
170             continue
171         password = getpass.getpass("Password: ")
172         user = users.get(login)
173         if user and user['hash'] == hash_password(password):
174             return user, login
175         print("Oops! Wrong login or password. Try again:")
176

```

```

177 def check_dac(acl_data, filename, login, required_perm):
178     file_acl = acl_data.get(filename)
179     if not file_acl:
180         return False
181     user_perms = file_acl.get('acl', {}).get(login, '')
182     return all(p in user_perms for p in required_perm)
183
184
185 def is_owner(acl_data, filename, login):
186     file_acl = acl_data.get(filename)
187     if not file_acl:
188         return False
189     return file_acl.get('owner') == login
190
191 def cmd_create(args, login, user_data):
192     filename = args[0] if args else None
193
194     while True:
195         if not filename:
196             filename = input("Enter filename (with extension, ugabuga.txt):
197                             ").strip()
198
199         if not filename:
200             print("Filename cannot be empty!")
201             filename = None
202             continue
203
204         name_part, _, ext_part = filename.rpartition('.')
205         if not name_part or not ext_part:
206             print("Ooops! You must specify an extension (.txt)")
207             print("Try again:")
208             filename = None
209             continue
210
211         break
212
213     if os.path.isabs(filename) or '..' in filename.split(os.sep):
214         print("Invalid filename!")
215         return
216
217     filepath = os.path.join(CONFDATA_DIR, filename)
218     if os.path.exists(filepath):
219         print(f"File {filename} already exists!")
220         return
221
222     try:
223         with open(filepath, 'w') as f:
224             f.write("")
225     except Exception as e:
226         print(f"Error creating file: {e}")
227         return
228
229     acl_data = read_acl()
230     acl_data[filename] = {
231         'owner': login,
232         'acl': {login: 'rwd'}
233     }
234     write_acl(acl_data)
235
236     print(f"File '{filename}' created!")
237     print(f"  Owner: {login} (permissions: rwd)")

```

```

237     logging.info(f"User {login} created file {filename}")
238
239
240 def cmd_read(args, login, user_data):
241     if not args:
242         print("Usage: read <filename>")
243         return
244     filename = args[0]
245
246     if os.path.isabs(filename) or '..' in filename.split(os.sep):
247         print("Invalid filename!")
248         return
249
250     filepath = os.path.join(CONFDATA_DIR, filename)
251     if not os.path.exists(filepath):
252         print(f"File {filename} not found")
253         return
254
255     acl_data = read_acl()
256
257     if not check_dac(acl_data, filename, login, 'r'):
258         print("ACCESS DENIED (DAC): You don't have read permission")
259         logging.warning(f"DAC DENIED: {login} tried to read {filename}")
260         return
261
262     try:
263         with open(filepath, 'r') as f:
264             content = f.read()
265             print(content if content else "(file is empty)")
266             logging.info(f"User {login} read file {filename}")
267     except Exception as e:
268         print(f"Error: {e}")
269
270
271 def cmd_write(args, login, user_data):
272     if not args:
273         print("Usage: write <filename>")
274         return
275     filename = args[0]
276
277     if os.path.isabs(filename) or '..' in filename.split(os.sep):
278         print("Invalid filename!")
279         return
280
281     filepath = os.path.join(CONFDATA_DIR, filename)
282     if not os.path.exists(filepath):
283         print(f"File {filename} not found")
284         return
285
286     acl_data = read_acl()
287
288     if not check_dac(acl_data, filename, login, 'w'):
289         print("ACCESS DENIED (DAC): You don't have write permission")
290         logging.warning(f"DAC DENIED: {login} tried to write {filename}")
291         return
292
293     print("Enter text (empty line to save):")
294     lines = []
295     while True:
296         line = input()
297         if line == "":

```

```

298         break
299     lines.append(line)
300
301     if not lines:
302         print("Nothing to write")
303         return
304
305     try:
306         with open(filepath, 'a') as f:
307             for line in lines:
308                 f.write(line + '\n')
309             print("Data written!")
310             logging.info(f"User {login} wrote to file {filename}")
311     except Exception as e:
312         print(f"Error: {e}")
313
314
315 def cmd_delete(args, login, user_data):
316     if not args:
317         print("Usage: delete <filename>")
318         return
319     filename = args[0]
320
321     if os.path.isabs(filename) or '..' in filename.split(os.sep):
322         print("Invalid filename!")
323         return
324
325     filepath = os.path.join(CONFDATA_DIR, filename)
326     if not os.path.exists(filepath):
327         print(f"File {filename} not found")
328         return
329
330     acl_data = read_acl()
331
332     if not check_dac(acl_data, filename, login, 'd'):
333         print("ACCESS DENIED (DAC): You don't have delete permission")
334         logging.warning(f"DAC DENIED: {login} tried to delete {filename}")
335         return
336
337     try:
338         os.remove(filepath)
339         if filename in acl_data:
340             del acl_data[filename]
341             write_acl(acl_data)
342         print(f"File {filename} deleted")
343         logging.info(f"User {login} deleted file {filename}")
344     except Exception as e:
345         print(f"Error: {e}")
346
347
348 def cmd_grant(args, login, user_data):
349     if len(args) < 3:
350         print("Usage: grant <filename> <login> <perms>")
351         print("perms: combination of r, w, d")
352         return
353
354     filename, target_login, perms = args[0], args[1], args[2]
355
356     if not all(ch in 'rwd' for ch in perms):
357         print("Permissions must be combination of: r, w, d")
358         return

```

```

359
360 acl_data = read_acl()
361
362 if not is_owner(acl_data, filename, login):
363     print("ACCESS DENIED: Only the file owner can grant permissions")
364     return
365
366 users = read_passwd()
367 if target_login not in users:
368     print(f"User {target_login} not found")
369     return
370
371 acl_data[filename]['acl'][target_login] = perms
372 write_acl(acl_data)
373 print(f"Granted '{perms}' on '{filename}' to {target_login}")
374 logging.info(f"User {login} granted '{perms}' on {filename} to {
    target_login}")
375
376
377 def cmd_revoke(args, login, user_data):
378     if len(args) < 2:
379         print("Usage: revoke <filename> <login>")
380         return
381
382     filename, target_login = args[0], args[1]
383
384     acl_data = read_acl()
385
386     if not is_owner(acl_data, filename, login):
387         print("ACCESS DENIED: Only the file owner can revoke permissions")
388         return
389
390     if target_login == login:
391         print("Cannot revoke your own permissions (you are the owner)")
392         return
393
394     file_acl = acl_data.get(filename, {}).get('acl', {})
395     if target_login not in file_acl:
396         print(f"User {target_login} has no permissions on {filename}")
397         return
398
399     del acl_data[filename]['acl'][target_login]
400     write_acl(acl_data)
401     print(f"Revoked permissions for {target_login} on {filename}")
402     logging.info(f"User {login} revoked permissions for {target_login} on {
        filename}")
403
404
405 # List files in confdata/ with access info.
406 def cmd_list(args, login, user_data):
407     acl_data = read_acl()
408
409     if not os.path.exists(CONFDATA_DIR):
410         print("confdata/ does not exist")
411         return
412
413     files = [f for f in os.listdir(CONFDATA_DIR)
414              if os.path.isfile(os.path.join(CONFDATA_DIR, f))]
415     if not files:
416         print("No files in confdata/")
417         return

```

```

418
419 print(f"{'File':<25} {'Owner':<12} {'Perms':<12}")
420 print("-" * 50)
421 for f in sorted(files):
422     file_acl = acl_data.get(f, {})
423     owner = file_acl.get('owner', '?')
424     my_perms = file_acl.get('acl', {}).get(login, '-')
425     print(f"{f:<25} {owner:<12} {my_perms:<12}")
426
427
428 # Show full ACL for a file (owner only).
429 def cmd_acl(args, login, user_data):
430     if not args:
431         print("Usage: acl <filename>")
432         return
433     filename = args[0]
434
435     acl_data = read_acl()
436     if filename not in acl_data:
437         print(f"No ACL data for {filename}")
438         return
439
440     if not is_owner(acl_data, filename, login):
441         print("ACCESS DENIED: Only the file owner can view the ACL")
442         return
443
444     file_acl = acl_data[filename]
445     print(f"File: {filename}")
446     print(f"Owner: {file_acl['owner']}")
447     print("Access Control List:")
448     for user, perms in file_acl.get('acl', {}).items():
449         print(f"{user}: {perms}")
450
451
452 def cmd_info(args, login, user_data):
453     print(f"Login: {login}")
454     print(f"Full name: {user_data['fullname']}")
455     print(f"UID: {user_data['uid']}")
456
457
458 def cmd_edit_my_info(login, user_data):
459     users = read_passwd()
460     if login not in users:
461         print("User not found.")
462         return user_data['fullname']
463     u = users[login]
464     print("Edit your info (press Enter to keep current).")
465     new_fullname = input(f"Full name [{user_data['fullname']}]: ").strip()
466     if new_fullname:
467         u['fullname'] = new_fullname
468         user_data['fullname'] = new_fullname
469     change_pwd = input("Change password? (y/n): ").strip().lower()
470     if change_pwd == 'y':
471         password = getpass.getpass("New password: ")
472         password_confirm = getpass.getpass("Confirm password: ")
473         if password != password_confirm:
474             print("Passwords do not match.")
475             return user_data['fullname']
476         valid, msg = validate_password(password)
477         if not valid:
478             print(f"Invalid password: {msg}")

```

```

479         return user_data['fullname']
480     u['hash'] = hash_password(password)
481     print("Password updated.")
482     write_passwd(users)
483     logging.info(f"User {login} updated their info")
484     print("Info updated.")
485     return user_data['fullname']
486
487
488 def cmd_help(args):
489     print("We have here:")
490     print()
491     print("create <filename>")
492     print("read <filename>")
493     print("write <filename>")
494     print("delete <filename>")
495     print("grant <file> <user> <perms> (owner only)")
496     print("revoke <file> <user> (owner only)")
497     print("list files with access info")
498     print("acl <filename> (owner only)")
499     print("info (user)")
500     print("edit my info")
501     print("help")
502     print("exit")
503
504
505 def _setup_logging():
506     elevate()
507     try:
508         logging.basicConfig(filename=ACCESS_LOG, level=logging.INFO,
509                             format='%(asctime)s - %(levelname)s - %(message)s')
510         os.makedirs(LOG_DIR, mode=0o700, exist_ok=True)
511         if os.path.exists(ACCESS_LOG):
512             os.chmod(ACCESS_LOG, 0o600)
513     except (PermissionError, OSError):
514         logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s',
515                             stream=sys.stderr, force=True)
516     # do not drop EUID here; otherwise read_passwd() in "edit my info" or "
517     # grant" won't see passwd on macOS
518
519 def main():
520     _setup_logging()
521     try:
522         if not os.path.exists(CONFDATA_DIR):
523             elevate()
524             try:
525                 os.makedirs(CONFDATA_DIR, mode=0o700, exist_ok=True)
526             finally:
527                 drop_to_real()
528     except (PermissionError, OSError) as e:
529         print(f"Error: {e}")
530         sys.exit(1)
531
532     users = read_passwd()
533     if not users:
534         _check_passwd_access()
535         print("Not initialized. No users.")
536         sys.exit(1)

```

```

537
538     try:
539         user, login = authenticate(users)
540     except KeyboardInterrupt:
541         print("\nExited by ^C")
542         sys.exit(0)
543
544     print(f"\nInfo-Protection Lab3 (DAC), {user['fullname']}")
545     print("Type 'help' for commands\n")
546
547     while True:
548         try:
549             cmd_line = input(f"{login}> ").strip()
550             if not cmd_line:
551                 continue
552             parts = cmd_line.split()
553             cmd = parts[0].lower()
554             args = parts[1:]
555
556             if cmd == 'exit':
557                 break
558             elif cmd == 'help':
559                 cmd_help(args)
560             elif cmd == 'create':
561                 cmd_create(args, login, user)
562             elif cmd == 'read':
563                 cmd_read(args, login, user)
564             elif cmd == 'write':
565                 cmd_write(args, login, user)
566             elif cmd == 'delete':
567                 cmd_delete(args, login, user)
568             elif cmd == 'grant':
569                 cmd_grant(args, login, user)
570             elif cmd == 'revoke':
571                 cmd_revoke(args, login, user)
572             elif cmd == 'list':
573                 cmd_list(args, login, user)
574             elif cmd == 'acl':
575                 cmd_acl(args, login, user)
576             elif cmd == 'info':
577                 cmd_info(args, login, user)
578             elif cmd == 'edit' and len(args) >= 2 and args[0] == 'my' and
579                 args[1] == 'info':
580                 cmd_edit_my_info(login, user)
581             else:
582                 print(f"Unknown command: {cmd}. Type 'help'")
583         except KeyboardInterrupt:
584             print("\n^C exited")
585             break
586         except Exception as e:
587             print(f"Error: {e}")
588
589 if __name__ == "__main__":
590     main()

```

access_MAC.py

```

1     import os
2     import sys
3     import getpass

```



```

4 import hashlib
5 import logging
6 import json
7 from config_MAC import (PASSWD_FILE, CONFDATA_DIR, ACCESS_LOG, LOG_DIR,
8                          HASH_ALGO, PASSWORD_ALPHABET,
9                          FIRST_CHAR_ALPHABET,
10                         ACL_FILE, FILE_LABELS_FILE,
11                         SECURITY_LEVELS)
12
13 _SAVED_EUID = None
14
15 # Temporarily raise EUID to the owner of PASSWD_FILE (root in lab setup)
16
17 # Real UID (UID) stays unchanged.
18 def elevate():
19     global _SAVED_EUID
20     if _SAVED_EUID is not None:
21         return
22     try:
23         st = os.stat(PASSWD_FILE)
24     except OSError:
25         return
26     target_euid = st.st_uid
27     current_euid = os.geteuid()
28     if current_euid == target_euid:
29         return
30     _SAVED_EUID = current_euid
31     try:
32         os.seteuid(target_euid)
33     except PermissionError:
34         _SAVED_EUID = None
35
36 # Drop temporary elevated EUID back to the original one.
37 def drop():
38     global _SAVED_EUID
39     if _SAVED_EUID is None:
40         return
41     try:
42         os.seteuid(_SAVED_EUID)
43     finally:
44         _SAVED_EUID = None
45
46 # Return EUID to the real user (after setuid login).
47 def drop_to_real():
48     if os.geteuid() != os.getuid():
49         try:
50             os.seteuid(os.getuid())
51         except (PermissionError, OSError):
52             pass
53
54 def _check_passwd_access():
55     uid, euid = os.getuid(), os.geteuid()
56     path = PASSWD_FILE
57     exists = os.path.exists(path)
58     err_no_elevate = None
59     try:
60         with open(path, "r") as f:

```

```

63         f.read(1)
64     except Exception as e:
65         err_no_elevate = e
66     log_lines = [
67         "", "-",
68         f" uid={uid}  euid={euid}  (euid=0 needed to read root file)",
69         f" passwd={path}  exists={exists}",
70     ]
71     if err_no_elevate:
72         log_lines.append(f" error opening: {type(err_no_elevate).__name__}: {err_no_elevate}")
73         log_lines.append(" With sudo: euid=0. Without sudo: euid=your uid, file 0600 root - access denied.")
74     else:
75         log_lines.append("+")
76     log_lines.append("---")
77     text = "\n".join(log_lines) + "\n"
78     print(text, file=sys.stderr, flush=True)
79     try:
80         with open("/tmp/access_debug.log", "a") as f:
81             f.write(text)
82     except Exception:
83         pass
84
85
86 def hash_password(password):
87     h = hashlib.new(HASH_ALGO)
88     h.update(password.encode('utf-8'))
89     return h.hexdigest()
90
91 def read_passwd():
92     users = {}
93     elevate()
94     try:
95         if not os.path.exists(PASSWD_FILE):
96             return users
97         with open(PASSWD_FILE, 'r') as f:
98             for line in f:
99                 line = line.strip()
100                 if not line or line.startswith('#'):
101                     continue
102                 parts = line.split(':', 4)
103                 if len(parts) != 5:
104                     continue
105                 login, pwd_hash, uid, sec_level, fullname = parts
106                 try:
107                     uid_int = int(uid)
108                     sec_int = int(sec_level)
109                 except ValueError:
110                     continue
111                 users[login] = {
112                     'hash': pwd_hash,
113                     'uid': uid_int,
114                     'security_level': sec_int,
115                     'fullname': fullname
116                 }
117     except Exception:
118         pass
119     finally:
120         drop()
121     return users

```

```

122
123
124 def write_passwd(users):
125     elevate()
126     try:
127         with open(PASSWD_FILE, 'w') as f:
128             for login, u in users.items():
129                 f.write(f"{login}:{u['hash']}:{u['uid']}:{u['
130                     security_level']}:{u['fullname']}\n")
131
132         try:
133             os.chmod(PASSWD_FILE, 0o600)
134         except OSError:
135             pass
136     finally:
137         drop()
138
139 def validate_password(password):
140     if len(password) < 1:
141         return False, "Cannot be empty"
142     if password[0] not in FIRST_CHAR_ALPHABET:
143         return False, "First character must be a letter"
144     for ch in password:
145         if ch not in PASSWORD_ALPHABET:
146             return False, f"Invalid character: {ch}"
147     return True, "OK"
148
149 # acl
150 def read_acl():
151     if not os.path.exists(ACL_FILE):
152         return {}
153     try:
154         with open(ACL_FILE, 'r') as f:
155             return json.load(f)
156     except (json.JSONDecodeError, IOError):
157         return {}
158
159 def write_acl(acl_data):
160     with open(ACL_FILE, 'w') as f:
161         json.dump(acl_data, f, indent=2, ensure_ascii=False)
162     try:
163         os.chmod(ACL_FILE, 0o600)
164     except OSError:
165         pass
166
167 # labels
168 def read_file_labels():
169     if not os.path.exists(FILE_LABELS_FILE):
170         return {}
171     try:
172         with open(FILE_LABELS_FILE, 'r') as f:
173             return json.load(f)
174     except (json.JSONDecodeError, IOError):
175         return {}
176
177 def write_file_labels(labels):
178     with open(FILE_LABELS_FILE, 'w') as f:
179         json.dump(labels, f, indent=2, ensure_ascii=False)
180     try:
181         os.chmod(FILE_LABELS_FILE, 0o600)
182     except OSError:

```

```

182         pass
183
184     def authenticate(users):
185         while True:
186             login = input("Login: ").strip()
187             if not login:
188                 continue
189             password = getpass.getpass("Password: ")
190             user = users.get(login)
191             if user and user['hash'] == hash_password(password):
192                 return user, login
193             print("Oops! Wrong login or password. Try again:")
194
195
196     # Bell-Lapadula
197     def check_mac_read(subject_level, object_level):
198         return subject_level >= object_level
199
200
201     def check_mac_write(subject_level, object_level):
202         return subject_level <= object_level
203
204     # DAC utils
205     def check_dac(acl_data, filename, login, required_perm):
206         file_acl = acl_data.get(filename)
207         if not file_acl:
208             return False
209         user_perms = file_acl.get('acl', {}).get(login, '')
210         return all(p in user_perms for p in required_perm)
211
212     def is_owner(acl_data, filename, login):
213         file_acl = acl_data.get(filename)
214         if not file_acl:
215             return False
216         return file_acl.get('owner') == login
217
218     def cmd_create(args, login, user_data):
219         filename = args[0] if args else None
220
221         while True:
222             if not filename:
223                 filename = input("Enter filename (with extension, ugabuga.
224                                     txt): ").strip()
225
226             if not filename:
227                 print("Filename cannot be empty!")
228                 filename = None
229                 continue
230
231             name_part, _, ext_part = filename.rpartition('.')
232             if not name_part or not ext_part:
233                 print("Ooops! You must specify an extension (.txt)")
234                 print("Try again:")
235                 filename = None
236                 continue
237
238             break
239
240     if os.path.isabs(filename) or '..' in filename.split(os.sep):
241         print("Invalid filename!")
242         return

```

```

242
243     filepath = os.path.join(CONFDATA_DIR, filename)
244     if os.path.exists(filepath):
245         print(f"File {filename} already exists!")
246         return
247
248     try:
249         with open(filepath, 'w') as f:
250             f.write("")
251     except Exception as e:
252         print(f"Error creating file: {e}")
253         return
254
255     acl_data = read_acl()
256     acl_data[filename] = {
257         'owner': login,
258         'acl': {login: 'rwd'}
259     }
260     write_acl(acl_data)
261
262     labels = read_file_labels()
263     labels[filename] = user_data['security_level']
264     write_file_labels(labels)
265
266     sec_name = SECURITY_LEVELS.get(user_data['security_level'], '?')
267     print(f"File '{filename}' created!")
268     print(f"Owner: {login} (permissions: rwd)")
269     print(f"Security label: {sec_name}")
270     logging.info(f"User {login} created file {filename} (security: {
271         sec_name})")
272
273 def cmd_read(args, login, user_data):
274     if not args:
275         print("Usage: read <filename>")
276         return
277     filename = args[0]
278
279     if os.path.isabs(filename) or '..' in filename.split(os.sep):
280         print("Invalid filename!")
281         return
282
283     filepath = os.path.join(CONFDATA_DIR, filename)
284     if not os.path.exists(filepath):
285         print(f"File {filename} not found")
286         return
287
288     acl_data = read_acl()
289     labels = read_file_labels()
290
291     if not check_dac(acl_data, filename, login, 'r'):
292         print("ACCESS DENIED (DAC): You don't have read permission")
293         logging.warning(f"DAC DENIED: {login} tried to read {filename}")
294         return
295
296     file_level = labels.get(filename, 0)
297     user_level = user_data['security_level']
298     if not check_mac_read(user_level, file_level):
299         file_label = SECURITY_LEVELS.get(file_level, '?')
300         user_label = SECURITY_LEVELS.get(user_level, '?')
301         print(f"ACCESS DENIED (MAC): No Read Up - your level ({

```

```

302         user_label})) < file level ({file_label}))")
303     logging.warning(f"MAC NRU DENIED: {login} (lvl {user_level}) ->
304         {filename} (lvl {file_level}))")
305     return
306
307     try:
308         with open(filepath, 'r') as f:
309             content = f.read()
310             print(content if content else "(file is empty)")
311             logging.info(f"User {login} read file {filename}")
312     except Exception as e:
313         print(f"Error: {e}")
314
315 def cmd_write(args, login, user_data):
316     if not args:
317         print("Usage: write <filename>")
318         return
319     filename = args[0]
320
321     if os.path.isabs(filename) or '..' in filename.split(os.sep):
322         print("Invalid filename!")
323         return
324
325     filepath = os.path.join(CONFDATA_DIR, filename)
326     if not os.path.exists(filepath):
327         print(f"File {filename} not found")
328         return
329
330     acl_data = read_acl()
331     labels = read_file_labels()
332
333     if not check_dac(acl_data, filename, login, 'w'):
334         print("ACCESS DENIED (DAC): You don't have write permission")
335         logging.warning(f"DAC DENIED: {login} tried to write {filename}")
336         return
337
338     file_level = labels.get(filename, 0)
339     user_level = user_data['security_level']
340     if not check_mac_write(user_level, file_level):
341         file_label = SECURITY_LEVELS.get(file_level, '?')
342         user_label = SECURITY_LEVELS.get(user_level, '?')
343         print(f"ACCESS DENIED (MAC): No Write Down - your level ({
344             user_label}) > file level ({file_label})")
345         logging.warning(f"MAC NWD DENIED: {login} (lvl {user_level}) ->
346             {filename} (lvl {file_level}))")
347         return
348
349     print("Enter text (empty line to save):")
350     lines = []
351     while True:
352         line = input()
353         if line == "":
354             break
355         lines.append(line)
356
357     if not lines:
358         print("Nothing to write")
359         return

```

```

358     try:
359         with open(filepath, 'a') as f:
360             for line in lines:
361                 f.write(line + '\n')
362             print("Data written!")
363             logging.info(f"User {login} wrote to file {filename}")
364     except Exception as e:
365         print(f"Error: {e}")
366
367
368 def cmd_delete(args, login, user_data):
369     if not args:
370         print("Usage: delete <filename>")
371         return
372     filename = args[0]
373
374     if os.path.isabs(filename) or '..' in filename.split(os.sep):
375         print("Invalid filename!")
376         return
377
378     filepath = os.path.join(CONFDATA_DIR, filename)
379     if not os.path.exists(filepath):
380         print(f"File {filename} not found")
381         return
382
383     acl_data = read_acl()
384
385     if not check_dac(acl_data, filename, login, 'd'):
386         print("ACCESS DENIED (DAC): You don't have delete permission")
387         logging.warning(f"DAC DENIED: {login} tried to delete {filename}")
388         return
389
390     try:
391         os.remove(filepath)
392         if filename in acl_data:
393             del acl_data[filename]
394             write_acl(acl_data)
395         labels = read_file_labels()
396         if filename in labels:
397             del labels[filename]
398             write_file_labels(labels)
399         print(f"File {filename} deleted")
400         logging.info(f"User {login} deleted file {filename}")
401     except Exception as e:
402         print(f"Error: {e}")
403
404
405 def cmd_grant(args, login, user_data):
406     if len(args) < 3:
407         print("Usage: grant <filename> <login> <perms>")
408         print("perms: combination of r, w, d")
409         return
410
411     filename, target_login, perms = args[0], args[1], args[2]
412
413     if not all(ch in 'rwd' for ch in perms):
414         print("Permissions must be combination of: r, w, d")
415         return
416
417     acl_data = read_acl()

```

```

418
419     if not is_owner(acl_data, filename, login):
420         print("ACCESS DENIED: Only the file owner can grant permissions"
421             )
422         return
423
424     users = read_passwd()
425     if target_login not in users:
426         print(f"User {target_login} not found")
427         return
428
429     acl_data[filename]['acl'][target_login] = perms
430     write_acl(acl_data)
431     print(f"Granted '{perms}' on '{filename}' to {target_login}")
432     logging.info(f"User {login} granted '{perms}' on {filename} to {
433         target_login}")
434
435 def cmd_revoke(args, login, user_data):
436     if len(args) < 2:
437         print("Usage: revoke <filename> <login>")
438         return
439
440     filename, target_login = args[0], args[1]
441
442     acl_data = read_acl()
443
444     if not is_owner(acl_data, filename, login):
445         print("ACCESS DENIED: Only the file owner can revoke permissions
446             ")
447         return
448
449     if target_login == login:
450         print("Cannot revoke your own permissions (you are the owner)")
451         return
452
453     file_acl = acl_data.get(filename, {}).get('acl', {})
454     if target_login not in file_acl:
455         print(f"User {target_login} has no permissions on {filename}")
456         return
457
458     del acl_data[filename]['acl'][target_login]
459     write_acl(acl_data)
460     print(f"Revoked permissions for {target_login} on {filename}")
461     logging.info(f"User {login} revoked permissions for {target_login}
462         on {filename}")
463
464 def cmd_list(args, login, user_data):
465     acl_data = read_acl()
466     labels = read_file_labels()
467
468     if not os.path.exists(CONFDATA_DIR):
469         print("confdata/ does not exist")
470         return
471
472     files = [f for f in os.listdir(CONFDATA_DIR)
473         if os.path.isfile(os.path.join(CONFDATA_DIR, f))]
474     if not files:
475         print("No files in confdata/")
476         return

```



```

475 user_level = user_data['security_level']
476 print(f"{'File':<25} {'Owner':<12} {'Your Perms':<12} {'Security'
477       ':<15} {'MAC-R':<8} {'MAC-W'}")
478 print("-" * 85)
479 for f in sorted(files):
480     file_acl = acl_data.get(f, {})
481     owner = file_acl.get('owner', '?')
482     my_perms = file_acl.get('acl', {}).get(login, '-')
483     file_level = labels.get(f, 0)
484     sec_name = SECURITY_LEVELS.get(file_level, '?')
485     can_r = "YES" if check_mac_read(user_level, file_level) else "NO"
486     can_w = "YES" if check_mac_write(user_level, file_level) else "NO"
487     print(f"{f:<25} {owner:<12} {my_perms:<12} {sec_name:<15} {can_r}
488           :<8} {can_w}")
489
490 def cmd_acl(args, login, user_data):
491     if not args:
492         print("Usage: acl <filename>")
493         return
494     filename = args[0]
495
496     acl_data = read_acl()
497     if filename not in acl_data:
498         print(f"No ACL data for {filename}")
499         return
500
501     if not is_owner(acl_data, filename, login):
502         print("ACCESS DENIED: Only the file owner can view the ACL")
503         return
504
505     file_acl = acl_data[filename]
506     labels = read_file_labels()
507     file_level = labels.get(filename, 0)
508
509     print(f"File: {filename}")
510     print(f"Owner: {file_acl['owner']}")
511     print(f"Security label: {SECURITY_LEVELS.get(file_level, '?')}")
512     print("Access Control List:")
513     for user, perms in file_acl.get('acl', {}).items():
514         print(f"  {user}: {perms}")
515
516
517 def cmd_info(args, login, user_data):
518     sec_name = SECURITY_LEVELS.get(user_data['security_level'], '?')
519     print(f"Login: {login}")
520     print(f"Full name: {user_data['fullname']}")
521     print(f"UID: {user_data['uid']}")
522     print(f"Security level: {user_data['security_level']} ({sec_name})")
523
524
525 def cmd_edit_my_info(login, user_data):
526     users = read_passwd()
527     if login not in users:
528         print("User not found.")
529         return user_data['fullname']
530     u = users[login]
531     print("Edit your info (/ press enter).")

```

```

532     new_fullname = input(f"Full name [{user_data['fullname']}]: ").strip
533     ()
534     if new_fullname:
535         u['fullname'] = new_fullname
536         user_data['fullname'] = new_fullname
537     change_pwd = input("Change password? (y/n): ").strip().lower()
538     if change_pwd == 'y':
539         password = getpass.getpass("New password: ")
540         password_confirm = getpass.getpass("Confirm password: ")
541         if password != password_confirm:
542             print("Passwords do not match.")
543             return user_data['fullname']
544         valid, msg = validate_password(password)
545         if not valid:
546             print(f"Invalid password: {msg}")
547             return user_data['fullname']
548         u['hash'] = hash_password(password)
549         print("Password updated.")
550     write_passwd(users)
551     logging.info(f"User {login} updated their info")
552     print("Info updated.")
553     return user_data['fullname']
554
555 def cmd_help(args):
556     print("We have here:")
557     print()
558     print("create <filename>")
559     print("read <filename> (No Read Up)")
560     print("write <filename> (No Write Down)")
561     print("delete <filename>")
562     print("grant <file> <user> <perms> (owner only)")
563     print("revoke <file> <user> (owner only)")
564     print("list (files with access info)")
565     print("acl <filename> (owner only)")
566     print("info (user)")
567     print("edit my info")
568     print("help")
569     print("exit")
570
571 def _setup_logging():
572     elevate()
573     try:
574         logging.basicConfig(filename=ACCESS_LOG, level=logging.INFO,
575                             format='%(asctime)s - %(levelname)s - %(
576                                 message)s')
577         os.makedirs(LOG_DIR, mode=0o700, exist_ok=True)
578         if os.path.exists(ACCESS_LOG):
579             os.chmod(ACCESS_LOG, 0o600)
580     except (PermissionError, OSError):
581         logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(
582             levelname)s - %(message)s',
583                             stream=sys.stderr, force=True)
584     # not drop EUID here; otherwise read_passwd()
585
586 def main():
587     _setup_logging()
588     try:
589         if not os.path.exists(CONFDATA_DIR):
590             elevate()

```

```

590         try:
591             os.makedirs(CONFDATA_DIR, mode=0o700, exist_ok=True)
592         finally:
593             drop_to_real()
594     except (PermissionError, OSError) as e:
595         print(f"Error: {e}")
596         sys.exit(1)
597
598     users = read_passwd()
599     if not users:
600         _check_passwd_access()
601         print("Not initialized. No users.")
602         sys.exit(1)
603
604     try:
605         user, login = authenticate(users)
606     except KeyboardInterrupt:
607         print("\nExited by ^C")
608         sys.exit(0)
609
610     sec_name = SECURITY_LEVELS.get(user['security_level'], '?')
611     print(f"\nInfo-Protection Lab3 (MAC), {user['fullname']}")
612     print(f"Security level: {sec_name}")
613     print("Type 'help' for commands\n")
614
615     while True:
616         try:
617             cmd_line = input(f"{login}> ").strip()
618             if not cmd_line:
619                 continue
620             parts = cmd_line.split()
621             cmd = parts[0].lower()
622             args = parts[1:]
623
624             if cmd == 'exit':
625                 break
626             elif cmd == 'help':
627                 cmd_help(args)
628             elif cmd == 'create':
629                 cmd_create(args, login, user)
630             elif cmd == 'read':
631                 cmd_read(args, login, user)
632             elif cmd == 'write':
633                 cmd_write(args, login, user)
634             elif cmd == 'delete':
635                 cmd_delete(args, login, user)
636             elif cmd == 'grant':
637                 cmd_grant(args, login, user)
638             elif cmd == 'revoke':
639                 cmd_revoke(args, login, user)
640             elif cmd == 'list':
641                 cmd_list(args, login, user)
642             elif cmd == 'acl':
643                 cmd_acl(args, login, user)
644             elif cmd == 'info':
645                 cmd_info(args, login, user)
646             elif cmd == 'edit' and len(args) >= 2 and args[0] == 'my'
647                 and args[1] == 'info':
648                 cmd_edit_my_info(login, user)
649             else:
650                 print(f"Unknown command: {cmd}. Type 'help'")

```

```

650         except KeyboardInterrupt:
651             print("\n^C exited")
652             break
653         except Exception as e:
654             print(f"Error: {e}")
655
656     if __name__ == "__main__":
657         main()

```

config_DAC.py

```

1  import os
2
3  # base tree
4  BASE_DIR = "/Users/practice3"
5  PASSWD_FILE = os.path.join(BASE_DIR, "etc", "passwd")
6  ACL_FILE = os.path.join(BASE_DIR, "etc", "acl.json")
7  CONFDATA_DIR = os.path.join(BASE_DIR, "confdata")
8  BIN_DIR = os.path.join(BASE_DIR, "bin")
9  LOG_DIR = os.path.join(BASE_DIR, "log")
10 AUTH_LOG = os.path.join(LOG_DIR, "auth.log")
11 ACCESS_LOG = os.path.join(LOG_DIR, "access.log")
12
13 # Algohash
14 HASH_ALGO = 'sha256'
15
16 # ASCII + 1 is char
17 PASSWORD_ALPHABET = "
18     abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#%$%^&*() "
19 FIRST_CHAR_ALPHABET = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

```

config_MAC.py

```

1  import os
2
3  # base tree
4  BASE_DIR = "/Users/practice3"
5  PASSWD_FILE = os.path.join(BASE_DIR, "etc", "passwd")
6  ACL_FILE = os.path.join(BASE_DIR, "etc", "acl.json")
7  FILE_LABELS_FILE = os.path.join(BASE_DIR, "etc", "file_labels.json")
8  CONFDATA_DIR = os.path.join(BASE_DIR, "confdata")
9  BIN_DIR = os.path.join(BASE_DIR, "bin")
10 LOG_DIR = os.path.join(BASE_DIR, "log")
11 AUTH_LOG = os.path.join(LOG_DIR, "auth.log")
12 ACCESS_LOG = os.path.join(LOG_DIR, "access.log")
13
14 # Algohash
15 HASH_ALGO = 'sha256'
16
17 # ASCII + 1 is char
18 PASSWORD_ALPHABET = "
19     abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#%$%^&*() "
20 FIRST_CHAR_ALPHABET = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
21
22 # security levels
23 SECURITY_LEVELS = {
24     0: "unclassified",
25     1: "DSP",
26     2: "secret"
27 }
28 SECURITY_LEVEL_NAMES = {v: k for k, v in SECURITY_LEVELS.items()}

```

user_manager_DAC.py

```
1  #!/usr/bin/env python3
2  import os
3  import sys
4  import getpass
5  import hashlib
6  import logging
7  import json
8  from config_DAC import (PASSWD_FILE, AUTH_LOG, LOG_DIR, HASH_ALGO,
9                          PASSWORD_ALPHABET, FIRST_CHAR_ALPHABET,
10                         ACL_FILE, CONFDATA_DIR)
11
12  logging.basicConfig(filename=AUTH_LOG, level=logging.INFO,
13                      format='%(asctime)s - %(levelname)s - %(message)s')
14
15  def hash_password(password):
16      h = hashlib.new(HASH_ALGO)
17      h.update(password.encode('utf-8'))
18      return h.hexdigest()
19
20
21  def validate_password(password):
22      if len(password) < 1:
23          return False, "Cant be empty!!!"
24      first = password[0]
25      if first not in FIRST_CHAR_ALPHABET:
26          return False, "First letter should be Letter"
27      for ch in password:
28          if ch not in PASSWORD_ALPHABET:
29              return False, f"Cant use: {ch}, only UTF-8"
30      return True, "OK"
31
32
33  # passwd format (DAC): login:hash:uid:fullname
34  def read_passwd():
35      users = []
36      if not os.path.exists(PASSWD_FILE):
37          return users
38      with open(PASSWD_FILE, 'r') as f:
39          for line in f:
40              line = line.strip()
41              if not line or line.startswith('#'):
42                  continue
43              parts = line.split(':', 4)
44              if len(parts) != 5:
45                  continue
46              login, pwd_hash, uid, sec_level, fullname = parts
47              try:
48                  uid_int = int(uid)
49              except ValueError:
50                  continue
51              users.append({
52                  'login': login,
53                  'hash': pwd_hash,
54                  'uid': uid_int,
55                  'security_level': sec_level,
56                  'fullname': fullname
57              })
58      return users
59
```

```

60
61 def write_passwd(users):
62     with open(PASSWD_FILE, 'w') as f:
63         for u in users:
64             f.write(f"{u['login']}:{u['hash']}:{u['uid']}:{u['
security_level']}:{u['fullname']}\n")
65
66     try:
67         os.chmod(PASSWD_FILE, 0o600)
68     except OSError:
69         pass
70
71 def get_next_uid(users):
72     if not users:
73         return 1000
74     return max(u['uid'] for u in users) + 1
75
76
77 def add_user():
78     print("Adding new user (DAC)")
79     login = input("Login: ").strip()
80     if not login:
81         print("Cant be empty!")
82         return
83     users = read_passwd()
84     if any(u['login'] == login for u in users):
85         print("User with this login already exists.")
86         a = input("Edit this user? (y/n): ").strip().lower()
87         if a == 'y':
88             _edit_user_by_login(login)
89         return
90
91     fullname = input("LnFnSn: ").strip()
92     if not fullname:
93         print("LnFnSn cant be empty")
94         return
95
96     password = getpass.getpass("Password: ")
97     password_confirm = getpass.getpass("Confirm password: ")
98     if password != password_confirm:
99         print("Oops! Isn't same")
100         return
101
102     valid, msg = validate_password(password)
103     if not valid:
104         print(f"Invalid pass!!!: {msg}")
105         return
106
107     pwd_hash = hash_password(password)
108     uid = get_next_uid(users)
109     users.append({
110         'login': login,
111         'hash': pwd_hash,
112         'uid': uid,
113         'security_level': '0',
114         'fullname': fullname
115     })
116     write_passwd(users)
117     logging.info(f"Added {login} (UID={uid})")
118     print("User added")
119

```

```

120 def _edit_user_by_login(login):
121     users = read_passwd()
122     user = next((u for u in users if u['login'] == login), None)
123     if not user:
124         print("Oops! Not found")
125         return
126
127     print("Keep empty to make no change")
128     new_fullname = input(f"LFS ({user['fullname']}): ").strip()
129     if new_fullname:
130         user['fullname'] = new_fullname
131
132     change_password = input("Change password? (y/n): ").strip().lower()
133     if change_password == 'y':
134         password = getpass.getpass("New password: ")
135         password_confirm = getpass.getpass("Confirm password: ")
136         if password != password_confirm:
137             print("Oops! Not the same")
138             return
139         valid, msg = validate_password(password)
140         if not valid:
141             print(f"Invalid pass: {msg}")
142             return
143         user['hash'] = hash_password(password)
144
145     write_passwd(users)
146     logging.info(f"Changed user {login}")
147     print("User changed")
148
149
150 def edit_user():
151     login = input("Enter login of user: ").strip()
152     if not login:
153         print("Cant be empty!")
154         return
155     _edit_user_by_login(login)
156
157
158 def delete_user():
159     login = input("Enter user login: ").strip()
160     users = read_passwd()
161     user = next((u for u in users if u['login'] == login), None)
162     if not user:
163         print("User not found")
164         return
165
166     confirm = input(f"Are u sure to delete bro: {login}? (y/n): ").strip()
167     if confirm != 'y':
168         print("Canceled")
169         return
170
171     uid = user['uid']
172     users.remove(user)
173     write_passwd(users)
174     logging.info(f"User deleted {login} (UID={uid})")
175     print("Deleted user")
176
177
178 def list_users():
179     users = read_passwd()

```

```

180     if not users:
181         print("No users found")
182         return
183     print(f"{'Login':<15} {'UID':<8} {'LnFnSn'}")
184     print("-" * 40)
185     for u in users:
186         print(f"{u['login']:<15} {u['uid']:<8} {u['fullname']}")
187
188
189     def read_acl():
190         if not os.path.exists(ACL_FILE):
191             return {}
192         try:
193             with open(ACL_FILE, 'r') as f:
194                 return json.load(f)
195         except (json.JSONDecodeError, IOError):
196             return {}
197
198
199     def show_matrix():
200         users = read_passwd()
201         acl_data = read_acl()
202
203         if not users:
204             print("No users found")
205             return
206
207         logins = [u['login'] for u in users]
208
209         files = []
210         if os.path.exists(CONFDATA_DIR):
211             files = sorted(f for f in os.listdir(CONFDATA_DIR)
212                            if os.path.isfile(os.path.join(CONFDATA_DIR, f)))
213         if not files:
214             print("No files in confdata/")
215             return
216
217         col_w = max(max(len(f) for f in files), 6) + 2
218         login_w = max(max(len(l) for l in logins), 10) + 2
219
220         header = "Subject".ljust(login_w)
221         for f in files:
222             owner = acl_data.get(f, {}).get('owner', '?')
223             label = f"{f}({owner})"
224             header += label.ljust(col_w + len(owner) + 2)
225         print("-" * 60)
226         print("DAC: Subj, Obj")
227         print("-" * 60)
228
229         col_w_real = col_w + 8
230         header = "Subject".ljust(login_w)
231         for f in files:
232             owner = acl_data.get(f, {}).get('owner', '?')
233             header += f"{f}[{owner}]"
234             header = header.ljust(col_w_real)
235         print(header)
236         print("-" * (login_w + col_w_real * len(files)))
237
238         for login in logins:
239             row = login.ljust(login_w)
240             for f in files:
241                 file_acl = acl_data.get(f, {})

```



```

241         perms = file_acl.get('acl', {}).get(login, '-')
242         row += perms.ljust(col_w_real)
243     print(row)
244
245     print()
246
247
248 def main():
249     try:
250         if os.geteuid() != 0:
251             print("Oops! You should be root. Or use sudo python3
252                   user_manager_DAC.py ...")
253             sys.exit(1)
254         os.makedirs(LOG_DIR, mode=0o700, exist_ok=True)
255         if os.path.exists(AUTH_LOG):
256             os.chmod(AUTH_LOG, 0o600)
257     except (PermissionError, OSError) as e:
258         if getattr(e, 'errno', None) == 13 or isinstance(e,
259                 PermissionError):
260             print("Oops! You should be root. Use with sudo.")
261         else:
262             print(f"Error: {e}")
263             sys.exit(1)
264
265     if len(sys.argv) < 2:
266         print("user_manager_DAC.py {add|edit|delete|list|matrix}")
267         sys.exit(1)
268
269     command = sys.argv[1].lower()
270     if command == 'add':
271         add_user()
272     elif command == 'edit':
273         edit_user()
274     elif command == 'delete':
275         delete_user()
276     elif command == 'list':
277         list_users()
278     elif command == 'matrix':
279         show_matrix()
280     else:
281         print(f"Unknown command: {command}")
282         sys.exit(1)
283
284 if __name__ == "__main__":
285     main()

```

user_manager_MAC.py

```

1  import os
2  import sys
3  import getpass
4  import hashlib
5  import logging
6  import json
7  from config_MAC import (PASSWD_FILE, AUTH_LOG, LOG_DIR, HASH_ALGO,
8                          PASSWORD_ALPHABET, FIRST_CHAR_ALPHABET,
9                          SECURITY_LEVELS, ACL_FILE, FILE_LABELS_FILE,
10                         CONFDATA_DIR)
11  logging.basicConfig(filename=AUTH_LOG, level=logging.INFO,
12                      format='%(asctime)s - %(levelname)s - %(message)s')
13

```

```

14 def hash_password(password):
15     h = hashlib.new(HASH_ALGO)
16     h.update(password.encode('utf-8'))
17     return h.hexdigest()
18
19
20 def validate_password(password):
21     if len(password) < 1:
22         return False, "Cant be empty!!!"
23     first = password[0]
24     if first not in FIRST_CHAR_ALPHABET:
25         return False, "First letter should be Letter"
26     for ch in password:
27         if ch not in PASSWORD_ALPHABET:
28             return False, f"Cant use: {ch}, only UTF-8"
29     return True, "OK"
30
31 # passwd format (MAC): login:hash:uid:security_level:fullname
32 def read_passwd():
33     users = []
34     if not os.path.exists(PASSWD_FILE):
35         return users
36     with open(PASSWD_FILE, 'r') as f:
37         for line in f:
38             line = line.strip()
39             if not line or line.startswith('#'):
40                 continue
41             parts = line.split(':', 4)
42             if len(parts) != 5:
43                 continue
44             login, pwd_hash, uid, sec_level, fullname = parts
45             try:
46                 uid_int = int(uid)
47                 sec_int = int(sec_level)
48             except ValueError:
49                 continue
50             users.append({
51                 'login': login,
52                 'hash': pwd_hash,
53                 'uid': uid_int,
54                 'security_level': sec_int,
55                 'fullname': fullname
56             })
57     return users
58
59
60 def write_passwd(users):
61     with open(PASSWD_FILE, 'w') as f:
62         for u in users:
63             f.write(f"{u['login']}:{u['hash']}:{u['uid']}:{u['security_level']}:{u['fullname']}\n")
64     try:
65         os.chmod(PASSWD_FILE, 0o600)
66     except OSError:
67         pass
68
69
70 def get_next_uid(users):
71     if not users:
72         return 1000
73     return max(u['uid'] for u in users) + 1

```

```

74
75
76 def add_user():
77     print("Adding new user (MAC)")
78     login = input("Login: ").strip()
79     if not login:
80         print("Cant be empty!")
81         return
82     users = read_passwd()
83     if any(u['login'] == login for u in users):
84         print("User with this login already exists.")
85         a = input("Edit this user? (y/n): ").strip().lower()
86         if a == 'y':
87             _edit_user_by_login(login)
88         return
89
90     fullname = input("LnFnSn: ").strip()
91     if not fullname:
92         print("LnFnSn cant be empty")
93         return
94
95     print("Security levels (MAC):")
96     for k, v in SECURITY_LEVELS.items():
97         print(f" {k} - {v}")
98     sec_input = input("Security level (0/1/2): ").strip()
99     try:
100         sec_level = int(sec_input)
101         if sec_level not in SECURITY_LEVELS:
102             print("Invalid security level!")
103             return
104     except ValueError:
105         print("Security level must be a number!")
106         return
107
108     password = getpass.getpass("Password: ")
109     password_confirm = getpass.getpass("Confirm password: ")
110     if password != password_confirm:
111         print("Oops! Isn't same")
112         return
113
114     valid, msg = validate_password(password)
115     if not valid:
116         print(f"Invalid pass!!!: {msg}")
117         return
118
119     pwd_hash = hash_password(password)
120     uid = get_next_uid(users)
121     users.append({
122         'login': login,
123         'hash': pwd_hash,
124         'uid': uid,
125         'security_level': sec_level,
126         'fullname': fullname
127     })
128     write_passwd(users)
129     logging.info(f"Added {login} (UID={uid}, security={SECURITY_LEVELS[
130         sec_level]})")
131     print(f"User added with security level: {SECURITY_LEVELS[sec_level]}")
132

```

```

133 def _edit_user_by_login(login):
134     users = read_passwd()
135     user = next((u for u in users if u['login'] == login), None)
136     if not user:
137         print("Oops! Not found")
138         return
139
140     print("Keep empty to make no change")
141     new_fullname = input(f"LFS ({user['fullname']}): ").strip()
142     if new_fullname:
143         user['fullname'] = new_fullname
144
145     cur_sec = user['security_level']
146     print(f"Current security level: {cur_sec} ({SECURITY_LEVELS[cur_sec]}")
147     print("Security levels (MAC):")
148     for k, v in SECURITY_LEVELS.items():
149         print(f"    {k} - {v}")
150     new_sec = input(f"Security level [{cur_sec}]: ").strip()
151     if new_sec:
152         try:
153             sec = int(new_sec)
154             if sec not in SECURITY_LEVELS:
155                 print("Invalid security level!")
156                 return
157             user['security_level'] = sec
158         except ValueError:
159             print("Security level must be a number!")
160             return
161
162     change_password = input("Change password? (y/n): ").strip().lower()
163     if change_password == 'y':
164         password = getpass.getpass("New password: ")
165         password_confirm = getpass.getpass("Confirm password: ")
166         if password != password_confirm:
167             print("Oops! Not the same")
168             return
169         valid, msg = validate_password(password)
170         if not valid:
171             print(f"Invalid pass: {msg}")
172             return
173         user['hash'] = hash_password(password)
174
175     write_passwd(users)
176     logging.info(f"Changed user {login}")
177     print("User changed")
178
179
180 def edit_user():
181     login = input("Enter login of user: ").strip()
182     if not login:
183         print("Cant be empty!")
184         return
185     _edit_user_by_login(login)
186
187
188 def delete_user():
189     login = input("Enter user login: ").strip()
190     users = read_passwd()
191     user = next((u for u in users if u['login'] == login), None)
192     if not user:

```

```

193         print("User not found")
194         return
195
196     confirm = input(f"Are u sure to delete bro: {login}? (y/n): ").strip
197     ().lower()
198     if confirm != 'y':
199         print("Canceled")
200         return
201
202     uid = user['uid']
203     users.remove(user)
204     write_passwd(users)
205     logging.info(f"User deleted {login} (UID={uid})")
206     print("Deleted user")
207
208 def list_users():
209     users = read_passwd()
210     if not users:
211         print("No users found")
212         return
213     print(f"{'Login':<15} {'UID':<8} {'Security':<20} {'LnFnSn'}")
214     print("-" * 60)
215     for u in users:
216         sec_name = SECURITY_LEVELS.get(u['security_level'], 'unknown')
217         print(f"{u['login']:<15} {u['uid']:<8} {sec_name:<20} {u[''
218             fullname']}"")
219
220 def read_acl():
221     if not os.path.exists(ACL_FILE):
222         return {}
223     try:
224         with open(ACL_FILE, 'r') as f:
225             return json.load(f)
226     except (json.JSONDecodeError, IOError):
227         return {}
228
229 def read_file_labels():
230     if not os.path.exists(FILE_LABELS_FILE):
231         return {}
232     try:
233         with open(FILE_LABELS_FILE, 'r') as f:
234             return json.load(f)
235     except (json.JSONDecodeError, IOError):
236         return {}
237
238 def show_matrix():
239     users = read_passwd()
240     acl_data = read_acl()
241     labels = read_file_labels()
242
243     if not users:
244         print("No users found")
245         return
246
247     logins = [u['login'] for u in users]
248     user_levels = {u['login']: u['security_level'] for u in users}
249
250     files = []

```

```

252     if os.path.exists(CONFDATA_DIR):
253         files = sorted(f for f in os.listdir(CONFDATA_DIR)
254                        if os.path.isfile(os.path.join(CONFDATA_DIR, f)))
255     if not files:
256         print("No files in confdata/")
257         return
258
259     sec_short = {0: "U", 1: "D", 2: "S"}
260
261     print("-" * 70)
262     print("macdac):  Subj x Obj")
263     print("NRU = No Read Up, NWD = No Write Down")
264     print("U = unclassified, D = DSP, S = secret")
265     print("-" * 70)
266
267     col_w = 18
268     login_w = 16
269
270     header = "Subject(lvl)".ljust(login_w)
271     for f in files:
272         file_level = labels.get(f, 0)
273         sl = sec_short.get(file_level, '?')
274         header += f"{{f}}[{{sl}}]".ljust(col_w)
275     print(header)
276     print("-" * (login_w + col_w * len(files)))
277
278     for login in logins:
279         u_lvl = user_levels[login]
280         sl = sec_short.get(u_lvl, '?')
281         row = f"{{login}}({{sl}})".ljust(login_w)
282         for f in files:
283             file_acl = acl_data.get(f, {})
284             dac_perms = file_acl.get('acl', {}).get(login, '-')
285             file_level = labels.get(f, 0)
286             can_r = u_lvl >= file_level
287             can_w = u_lvl <= file_level
288             mac_flags = ""
289             if not can_r:
290                 mac_flags += "!R"
291             if not can_w:
292                 mac_flags += "!W"
293             cell = dac_perms
294             if mac_flags:
295                 cell += f" {{mac_flags}}"
296             row += cell.ljust(col_w)
297         print(row)
298
299     print()
300     print("Legend: !R = MAC blocks read (NRU), !W = MAC blocks write (
301           NWD)")
302     print()
303
304     def main():
305         try:
306             if os.geteuid() != 0:
307                 print("Oops! You should be root. Or use sudo python3
308                       user_manager_MAC.py ...")
309                 sys.exit(1)
310             os.makedirs(LOG_DIR, mode=0o700, exist_ok=True)
311             if os.path.exists(AUTH_LOG):

```

```

311         os.chmod(AUTH_LOG, 0o600)
312     except (PermissionError, OSError) as e:
313         if getattr(e, 'errno', None) == 13 or isinstance(e,
314             PermissionError):
315             print("Oops! You should be root. Use with sudo.")
316         else:
317             print(f"Error: {e}")
318             sys.exit(1)
319
320     if len(sys.argv) < 2:
321         print("user_manager_MAC.py {add|edit|delete|list|matrix}")
322         sys.exit(1)
323
324     command = sys.argv[1].lower()
325     if command == 'add':
326         add_user()
327     elif command == 'edit':
328         edit_user()
329     elif command == 'delete':
330         delete_user()
331     elif command == 'list':
332         list_users()
333     elif command == 'matrix':
334         show_matrix()
335     else:
336         print(f"Unknown command: {command}")
337         sys.exit(1)
338
339 if __name__ == "__main__":
340     main()

```

setup_practice4.py

```

1  import os
2  import sys
3  import subprocess
4  from pathlib import Path
5
6  ROOT = Path(__file__).parent
7  sys.path.insert(0, str(ROOT))
8  try:
9      from config import ALLOWED_PING_FROM_IP
10 except ImportError:
11     ALLOWED_PING_FROM_IP = os.environ.get("ALLOWED_PING_FROM_IP", "
12         192.168.1.100")
13
14 ANCHOR_FILE = "/etc/pf.anchors/practice4"
15 PF_CONF = "/etc/pf.conf"
16 ANCHOR_MARKER = 'anchor "practice4"'
17 LOAD_MARKER = 'load anchor "practice4" from "/etc/pf.anchors/practice4"'
18
19 from firewall_rules import get_rules
20
21 def ensure_anchor_in_pf_conf() -> bool:
22     try:
23         content = Path(PF_CONF).read_text(encoding="utf-8", errors="replace"
24             )
25     except OSError as e:
26         print(f"Error reading {PF_CONF}: {e}")
27         return False
28
29     if ANCHOR_MARKER in content and LOAD_MARKER in content:
30         return True
31
32     try:
33         with open(PF_CONF, "a", encoding="utf-8") as f:
34             f.write("\n# practice4 anchor\n")
35             f.write(ANCHOR_MARKER + "\n")
36             f.write(LOAD_MARKER + "\n")
37         print(f"Anchor practice4 added in {PF_CONF}")
38         return True
39     except OSError as e:
40         print(f"Error writing {PF_CONF}: {e}")
41         return False
42
43 def write_anchor(rules: str) -> bool:
44     try:
45         Path(ANCHOR_FILE).write_text(rules, encoding="utf-8")
46         return True
47     except OSError as e:
48         print(f"Error writing {ANCHOR_FILE}: {e}")
49         print("Run with sudo.")
50         return False
51
52
53 def run_pfctl(args: list[str]) -> tuple[int, str]:
54     try:

```



```

55         r = subprocess.run(["pfctl"] + args, capture_output=True, text=True,
56                               timeout=10)
57         return r.returncode, (r.stdout or "") + (r.stderr or "")
58     except Exception as e:
59         return -1, str(e)
60
61 def main() -> int:
62     if os.geteuid() != 0:
63         print("Run with sudo: sudo python3 setup_practice4.py")
64         return 1
65
66     if sys.platform != "darwin":
67         print("Error")
68         return 1
69
70     if not ensure_anchor_in_pf_conf():
71         return 1
72
73     if not write_anchor(get_rules()):
74         return 1
75
76     code, out = run_pfctl(["-nf", PF_CONF])
77     if code != 0:
78         print("Error syntax rules:")
79         print(out)
80         return 1
81
82     run_pfctl(["-e"])
83
84     code, out = run_pfctl(["-F", "all", "-q", "-f", PF_CONF])
85     if code != 0:
86         print("Error loading pf:")
87         print(out)
88         return 1
89
90     print("Rules applied. ALLOWED_PING_FROM_IP =", ALLOWED_PING_FROM_IP)
91     return 0
92
93
94 if __name__ == "__main__":
95     sys.exit(main())

```

firewall_rules.py

```

1  import os
2  import sys
3  import subprocess
4  from pathlib import Path
5
6  ROOT = Path(__file__).parent
7  sys.path.insert(0, str(ROOT))
8  try:
9      from config import ALLOWED_PING_FROM_IP
10 except ImportError:
11     ALLOWED_PING_FROM_IP = os.environ.get("ALLOWED_PING_FROM_IP", "
12         192.168.1.100")
13
14 ANCHOR_FILE = "/etc/pf.anchors/practice4"
15 PF_CONF = "/etc/pf.conf"

```

```

16 def get_rules() -> str:
17     return f"""
18     # ALLOWED_PING_FROM_IP = {ALLOWED_PING_FROM_IP}
19
20
21     set block-policy drop
22     scrub in all
23
24     block all
25
26     pass quick on lo0
27
28     pass out quick proto udp to any port 53 keep state
29     pass out quick proto tcp to any port 53 keep state
30
31     pass out quick inet proto icmp all icmp-type 8 code 0 keep state
32
33     pass in quick inet proto icmp all icmp-type 0 code 0
34
35     pass in quick inet proto icmp from {ALLOWED_PING_FROM_IP} icmp-type 8
36         code 0 keep state
37
38     pass out quick proto tcp to any port {{ 80 443 }} keep state
39     """
40
41 def get_flush_rules() -> str:
42     return """# Flush all traffic
43     pass all
44     """
45
46
47 def run_pfctl(args: list[str]) -> tuple[int, str]:
48     try:
49         r = subprocess.run(["pfctl"] + args, capture_output=True, text=
50             True, timeout=10)
51         return r.returncode, (r.stdout or "") + (r.stderr or "")
52     except Exception as e:
53         return -1, str(e)
54
55 def apply_rules() -> int:
56     if os.geteuid() != 0:
57         print("Run with sudo.")
58         return 1
59     if sys.platform != "darwin":
60         print("Error")
61         return 1
62
63     try:
64         Path(ANCHOR_FILE).write_text(get_rules(), encoding="utf-8")
65     except OSError as e:
66         print(f"Error {ANCHOR_FILE}: {e}")
67         return 1
68
69     code, out = run_pfctl(["-nf", PF_CONF])
70     if code != 0:
71         print("Error:")
72         print(out)
73         return 1
74

```

```

75     run_pfctl(["-e"])
76     code, out = run_pfctl(["-F", "all", "-q", "-f", PF_CONF])
77     if code != 0:
78         print("Error")
79         print(out)
80         return 1
81
82     print("Rules applied. ALLOWED_PING_FROM_IP =", ALLOWED_PING_FROM_IP)
83     return 0
84
85
86 def flush_rules() -> int:
87     if os.geteuid() != 0:
88         print("Run with sudo.")
89         return 1
90
91     try:
92         Path(ANCHOR_FILE).write_text(get_flush_rules(), encoding="utf-8"
93         )
94     except OSError as e:
95         print(f"Error {ANCHOR_FILE}: {e}")
96         return 1
97
98     run_pfctl(["-f", PF_CONF])
99     print("Rules flushed (pass all).")
100     return 0
101
102 def show_rules() -> int:
103     code, out = run_pfctl(["-sr"])
104     print(out)
105     return 0 if code == 0 else 1
106
107
108 def main() -> int:
109     action = (sys.argv[1] if len(sys.argv) > 1 else "apply").lower()
110
111     if action == "apply":
112         return apply_rules()
113     elif action == "flush":
114         return flush_rules()
115     elif action == "show":
116         return show_rules()
117     else:
118         print("Usage: sudo python3 firewall_rules.py [apply|flush|show]"
119         )
120         return 1
121
122 if __name__ == "__main__":
123     sys.exit(main())

```

config.py

```

1 MY_IP = "192.168.0.2"
2 ALLOWED_PING_FROM_IP = "192.168.0.4"
3 DNS_SERVERS = ["8.8.8.8", "8.8.4.4", "1.1.1.1"]

```

add_user.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import getpass
6  import sys
7  from pathlib import Path
8
9  _ROOT = Path(__file__).resolve().parent.parent
10 if str(_ROOT / "src") not in sys.path:
11     sys.path.insert(0, str(_ROOT / "src"))
12
13 from auth import hash_line, load_passwd # noqa: E402
14
15
16 def main() -> None:
17     p = argparse.ArgumentParser()
18     default_passwd = _ROOT / "confdata" / "passwd"
19     p.add_argument("--passwd", type=Path, default=default_passwd)
20     p.add_argument("login")
21     args = p.parse_args()
22     passwd_path = args.passwd.resolve()
23     login = args.login.strip()
24     if not login:
25         print("Login cannot be empty.", file=sys.stderr)
26         sys.exit(1)
27     db = load_passwd(passwd_path)
28     if login in db:
29         print("Ooops! User with this login exists!", file=sys.stderr)
30         sys.exit(1)
31     pw = getpass.getpass("Password: ")
32     passwd_path.parent.mkdir(parents=True, exist_ok=True)
33     line = hash_line(login, pw) + "\n"
34     with passwd_path.open("a", encoding="utf-8") as f:
35         f.write(line)
36     print(f"Added to {passwd_path}")
37
38
39 if __name__ == "__main__":
40     main()

```

auth.py

```

1  from __future__ import annotations
2
3  import hashlib
4  import secrets
5  from pathlib import Path
6
7
8  def _parse_line(line: str) -> tuple[str, bytes, bytes] | None:
9     line = line.strip()
10     if not line or line.startswith("#"):
11         return None
12     if ":" not in line:
13         return None

```

```

14     login, rest = line.split(":", 1)
15     parts = rest.split("$")
16     if len(parts) != 3 or parts[0] != "100000":
17         return None
18     try:
19         salt = bytes.fromhex(parts[1])
20         digest = bytes.fromhex(parts[2])
21     except ValueError:
22         return None
23     return login.strip(), salt, digest
24
25
26 def load_passwd(path: Path) -> dict[str, tuple[bytes, bytes]]:
27     out: dict[str, tuple[bytes, bytes]] = {}
28     if not path.is_file():
29         return out
30     for line in path.read_text(encoding="utf-8").splitlines():
31         parsed = _parse_line(line)
32         if parsed:
33             login, salt, digest = parsed
34             out[login] = (salt, digest)
35     return out
36
37
38 def verify_password(login: str, password: str, db: dict[str, tuple[bytes,
bytes]]) -> bool:
39     if login not in db:
40         return False
41     salt, expected = db[login]
42     got = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt,
100000)
43     return secrets.compare_digest(got, expected)
44
45
46 def hash_line(login: str, password: str) -> str:
47     salt = secrets.token_bytes(16)
48     digest = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt,
100000)
49     return f"{login}:100000${salt.hex()}${digest.hex()}"

```

client.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import errno
6  import getpass
7  import json
8  import os
9  import socket
10 import sys
11 import threading
12
13 from crypto import (
14     dh_client,
15     load_master_key,
16     pack_message,
17     parse_message,
18     read_frame,
19     write_frame,

```

```

20 )
21
22
23 def read_line(sock: socket.socket) -> str:
24     buf = bytearray()
25     while True:
26         chunk = sock.recv(1)
27         if not chunk:
28             raise ConnectionError("connection broken")
29         if chunk == b"\n":
30             break
31         buf.extend(chunk)
32         if len(buf) > 4096:
33             raise ValueError("too long string")
34     return buf.decode("utf-8")
35
36
37 def _reader(sock: socket.socket, master_key: bytes) -> None:
38     try:
39         while True:
40             body = read_frame(sock)
41             payload = parse_message(body, master_key=master_key)
42             print(payload.text, flush=True)
43     except OSError as e:
44         if e.errno not in (errno.EBADF, errno.ENOTCONN, errno.ECONNRESET):
45             print(f"[chat] {e}", flush=True)
46     except ValueError as e:
47         err_str = str(e)
48         print(file=sys.stderr)
49         print("=" * 60, file=sys.stderr)
50         print("!!! INTEGRITY CHECK FAILED !!!", file=sys.stderr)
51         print(err_str, file=sys.stderr)
52         print("Disconnected due to integrity violation.", file=sys.stderr)
53         print("=" * 60, file=sys.stderr)
54         sys.stderr.flush()
55     try:
56         sock.shutdown(socket.SHUT_RDWR)
57         sock.close()
58     except OSError:
59         pass
60     os._exit(1)
61 except (ConnectionError, UnicodeError) as e:
62     print(f"[chat] {e}", flush=True)
63
64
65 def main() -> None:
66     p = argparse.ArgumentParser()
67     p.add_argument("host", nargs="?", default="localhost")
68     p.add_argument("port", type=int, nargs="?", default=9000)
69     p.add_argument("--user", "-u", default=None)
70     p.add_argument("--password", "-p", default=None)
71     p.add_argument("--key-file", type=str, default=None)
72     p.add_argument("--encrypt", "-e", action="store_true")
73     p.add_argument("--integrity", "-i", action="store_true")
74     p.add_argument("--test-integrity", "-t", action="store_true")
75     p.add_argument("--dh", action="store_true")
76     args = p.parse_args()
77
78     if args.encrypt:
79         if args.dh and args.key_file:
80             raise SystemExit("Either --dh or --key-file")

```

```

81         if not args.dh and not args.key_file:
82             raise SystemExit("For --encrypt specify --key-file or --dh")
83     if args.key_file and args.dh:
84         raise SystemExit("Cannot use --key-file and --dh together")
85     if args.test_integrity and not args.integrity:
86         raise SystemExit("--test-integrity only with --integrity")
87
88     if args.encrypt and args.key_file:
89         from pathlib import Path
90
91         master_key = load_master_key(Path(args.key_file))
92     else:
93         master_key = b"\x00" * 32
94
95     user = args.user or input("Login: ")
96     password = args.password
97     if password is None:
98         password = getpass.getpass("Password: ")
99
100    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
101    sock.connect((args.host, args.port))
102
103    auth = json.dumps({"user": user, "pass": password}, ensure_ascii=False)
104    sock.sendall(auth.encode("utf-8") + b"\n")
105
106    resp = read_line(sock)
107    if resp != "OK":
108        if resp == "BUSY":
109            print(
110                "BUSY: Login already in chat. Close another client or use\n",
111                "another user.",
112                file=sys.stderr,
113            )
114        else:
115            print(resp, file=sys.stderr)
116            sys.exit(1)
117
118    if args.dh:
119        master_key = dh_client(sock)
120
121    t = threading.Thread(target=_reader, args=(sock, master_key), daemon=
122    True)
123    t.start()
124
125    try:
126        for line in sys.stdin:
127            text = line.rstrip("\n\r")
128            if text == r"\q":
129                body = pack_message(
130                    r"\q",
131                    master_key=master_key,
132                    use_encrypt=args.encrypt,
133                    use_integrity=args.integrity,
134                    test_bad_hash=args.test_integrity,
135                )
136                write_frame(sock, body)
137                break
138            body = pack_message(
139                text,
140                master_key=master_key,
141                use_encrypt=args.encrypt,

```

```

140         use_integrity=args.integrity,
141         test_bad_hash=args.test_integrity,
142     )
143     write_frame(sock, body)
144 except (BrokenPipeError, OSError):
145     pass
146 finally:
147     try:
148         sock.shutdown(socket.SHUT_RDWR)
149     except OSError:
150         pass
151     t.join(timeout=5.0)
152     try:
153         sock.close()
154     except OSError:
155         pass
156
157
158 if __name__ == "__main__":
159     main()

```

crypto.py

```

1 from __future__ import annotations
2
3 import hashlib
4 import os
5 import stat
6 import struct
7 from pathlib import Path
8 from typing import NamedTuple
9
10 from cryptography.hazmat.primitives import hashes
11 from cryptography.hazmat.primitives.asymmetric import x25519
12 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
13 from cryptography.hazmat.primitives.kdf.hkdf import HKDF
14
15 from cryptography.hazmat.primitives import serialization
16
17 MAX_MESSAGE_CHARS = 80
18 BLAKE2B_DIGEST = 32
19 CHACHA_KEY_SIZE = 32
20 CHACHA_NONCE_SIZE = 16
21 SALT_SIZE = 16
22 MIN_ENCRYPT_BLOB = SALT_SIZE + CHACHA_NONCE_SIZE + 1
23
24 FLAG_ENCRYPT = 1 << 0
25 FLAG_INTEGRITY = 1 << 1
26 FLAG_TEST_BAD_HASH = 1 << 2
27
28 KIND_DH_CLIENT = 0xC1
29 KIND_DH_SERVER = 0xC2
30 KIND_USER = 0x01
31
32
33 class UserPayload(NamedTuple):
34     text: str
35     encrypt: bool
36     integrity: bool
37     test_bad_hash: bool
38

```



```

39
40 def _check_key_file_permissions(path: Path) -> None:
41     if os.name == "nt":
42         return
43     st = path.stat()
44     mode = stat.S_IMODE(st.st_mode)
45     if st.st_uid != os.geteuid():
46         raise PermissionError(f"Key file {path} must belong to the current
47                                user")
48     if mode & 0o077:
49         raise PermissionError(f"Key file {path}: chmod 400 for owner")
50     if not (mode & stat.S_IRUSR):
51         raise PermissionError(f"Key file {path}: no read for owner")
52
53 def load_master_key(path: Path) -> bytes:
54     _check_key_file_permissions(path)
55     raw = path.read_text(encoding="utf-8").strip()
56     if len(raw) < 32:
57         raise ValueError("Key in file must be at least 32 characters")
58     return raw.encode("utf-8")
59
60
61 def derive_session_key_from_shared_secret(shared_secret: bytes) -> bytes:
62     hkdf = HKDF(algorithm=hashes.SHA256(), length=CHACHA_KEY_SIZE, salt=None
63                 , info=b"lab5-x25519-session")
64     return hkdf.derive(shared_secret)
65
66 def derive_message_key(master_key: bytes, salt: bytes) -> bytes:
67     hkdf = HKDF(
68         algorithm=hashes.SHA256(),
69         length=CHACHA_KEY_SIZE,
70         salt=salt,
71         info=b"lab5-chacha20-msg",
72     )
73     return hkdf.derive(master_key)
74
75
76 def chacha20_encrypt(plaintext: bytes, master_key: bytes) -> bytes:
77     if len(plaintext) > MAX_MESSAGE_CHARS * 4:
78         raise ValueError("too long message")
79     salt = os.urandom(SALT_SIZE)
80     msg_key = derive_message_key(master_key, salt)
81     nonce = os.urandom(CHACHA_NONCE_SIZE)
82     cipher = Cipher(algorithms.ChaCha20(msg_key, nonce), mode=None)
83     enc = cipher.encryptor()
84     return salt + nonce + enc.update(plaintext) + enc.finalize()
85
86
87 def chacha20_decrypt(blob: bytes, master_key: bytes) -> bytes:
88     if len(blob) < SALT_SIZE + CHACHA_NONCE_SIZE:
89         raise ValueError("invalid packet")
90     salt = blob[:SALT_SIZE]
91     nonce = blob[SALT_SIZE : SALT_SIZE + CHACHA_NONCE_SIZE]
92     ciphertext = blob[SALT_SIZE + CHACHA_NONCE_SIZE :]
93     msg_key = derive_message_key(master_key, salt)
94     cipher = Cipher(algorithms.ChaCha20(msg_key, nonce), mode=None)
95     dec = cipher.decryptor()
96     return dec.update(ciphertext) + dec.finalize()
97

```

```

98
99 def blake2b_digest(data: bytes) -> bytes:
100     return hashlib.blake2b(data, digest_size=BLAKE2B_DIGEST).digest()
101
102
103 def verify_message_text(text: str) -> None:
104     if len(text) > MAX_MESSAGE_CHARS:
105         raise ValueError(f"No more than {MAX_MESSAGE_CHARS} characters")
106
107
108 def read_exact(sock, n: int) -> bytes:
109     buf = bytearray()
110     while len(buf) < n:
111         chunk = sock.recv(n - len(buf))
112         if not chunk:
113             raise ConnectionError("connection closed")
114         buf.extend(chunk)
115     return bytes(buf)
116
117
118 def write_frame(sock, body: bytes) -> None:
119     sock.sendall(struct.pack("!I", len(body)) + body)
120
121
122 def read_frame(sock) -> bytes:
123     (length,) = struct.unpack("!I", read_exact(sock, 4))
124     if length > 0x100000:
125         raise ValueError("invalid frame length")
126     return read_exact(sock, length)
127
128
129 def _raw_pub(key: x25519.X25519PublicKey) -> bytes:
130     return key.public_bytes(
131         encoding=serialization.Encoding.Raw,
132         format=serialization.PublicFormat.Raw,
133     )
134
135 def dh_client(sock) -> bytes:
136     priv = x25519.X25519PrivateKey.generate()
137     write_frame(sock, bytes([KIND_DH_CLIENT]) + _raw_pub(priv.public_key()))
138     body = read_frame(sock)
139     if len(body) != 33 or body[0] != KIND_DH_SERVER:
140         raise ValueError("Expected DH server")
141     peer = x25519.X25519PublicKey.from_public_bytes(body[1:33])
142     return derive_session_key_from_shared_secret(priv.exchange(peer))
143
144
145 def dh_server(sock) -> bytes:
146     body = read_frame(sock)
147     if len(body) != 33 or body[0] != KIND_DH_CLIENT:
148         raise ValueError("Expected DH client")
149     peer = x25519.X25519PublicKey.from_public_bytes(body[1:33])
150     priv = x25519.X25519PrivateKey.generate()
151     write_frame(sock, bytes([KIND_DH_SERVER]) + _raw_pub(priv.public_key()))
152     return derive_session_key_from_shared_secret(priv.exchange(peer))
153
154 def pack_message(
155     text: str,
156     *,
157     master_key: bytes,
158     use_encrypt: bool,

```

```

159     use_integrity: bool,
160     test_bad_hash: bool,
161 ) -> bytes:
162     verify_message_text(text)
163     pt = text.encode("utf-8")
164     flags = 0
165     if use_encrypt:
166         flags |= FLAG_ENCRYPT
167     if use_integrity:
168         flags |= FLAG_INTEGRITY
169     if test_bad_hash:
170         flags |= FLAG_TEST_BAD_HASH
171
172     if test_bad_hash and len(pt) > 0:
173         pt_corrupted = bytearray(pt)
174         idx = len(pt_corrupted) // 2
175         pt_corrupted[idx] = (pt_corrupted[idx] + 1) \% 256
176         pt_corrupted = bytes(pt_corrupted)
177     else:
178         pt_corrupted = pt
179
180     inner = bytearray([flags])
181     if use_encrypt:
182         inner.extend(chacha20_encrypt(pt_corrupted, master_key))
183     else:
184         inner.append(len(pt_corrupted))
185         inner.extend(pt_corrupted)
186
187     if use_integrity:
188         h = blake2b_digest(pt)
189         inner.extend(h)
190
191     return bytes([KIND_USER]) + bytes(inner)
192
193
194 def parse_message(body: bytes, *, master_key: bytes) -> UserPayload:
195     if not body or body[0] != KIND_USER:
196         raise ValueError("not user packet")
197     pos = 1
198     flags = body[pos]
199     pos += 1
200     encrypt = bool(flags & FLAG_ENCRYPT)
201     integrity = bool(flags & FLAG_INTEGRITY)
202     test_sent = bool(flags & FLAG_TEST_BAD_HASH)
203     recv_hash: bytes | None = None
204
205     if encrypt:
206         blob = body[pos:]
207         if integrity:
208             if len(blob) < MIN_ENCRYPT_BLOB + BLAKE2B_DIGEST:
209                 raise ValueError("short packet")
210             ct_part = blob[:-BLAKE2B_DIGEST]
211             recv_hash = blob[-BLAKE2B_DIGEST:]
212         else:
213             if len(blob) < MIN_ENCRYPT_BLOB:
214                 raise ValueError("short packet")
215             ct_part = blob
216         pt = chacha20_decrypt(ct_part, master_key)
217         try:
218             text = pt.decode("utf-8")
219         except UnicodeDecodeError:

```

```

220         text = pt.decode("utf-8", errors="replace")
221     else:
222         ln = body[pos]
223         pos += 1
224         if integrity:
225             end = len(body) - BLAKE2B_DIGEST
226             recv_hash = body[end : end + BLAKE2B_DIGEST]
227             pt = body[pos:end]
228         else:
229             pt = body[pos : pos + ln]
230             text = pt.decode("utf-8")
231
232     if integrity:
233         expected = blake2b_digest(pt)
234         if recv_hash is None or recv_hash != expected:
235             raise ValueError(
236                 f"Integrity check failed: invalid Blake2b. Corrupted data
237                     received: {text!r}"
238             )
239
240     verify_message_text(text)
241     return UserPayload(text=text, encrypt=encrypt, integrity=integrity,
242                        test_bad_hash=test_sent)

```

server.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import json
6  import re
7  import socket
8  import threading
9  from pathlib import Path
10
11  _ROOT = Path(__file__).resolve().parent.parent
12
13  from auth import load_passwd, verify_password
14  from crypto import (
15      dh_server,
16      load_master_key,
17      pack_message,
18      parse_message,
19      read_frame,
20      write_frame,
21  )
22
23  clients_lock = threading.Lock()
24  clients: dict[str, socket.socket] = {}
25  client_keys: dict[str, bytes] = {}
26
27  _MENTION = re.compile(r"@([A-Za-z0-9_]+)")
28
29
30  def _mentioned(text: str) -> list[str]:
31      return _MENTION.findall(text)
32
33
34  def _has_mentions(text: str) -> bool:
35      return bool(_MENTION.search(text))

```

```

36
37
38 def _chat_line(username: str, text: str) -> str:
39     return f"{username} : {text}"
40
41
42 def read_line(sock: socket.socket) -> str:
43     buf = bytearray()
44     while True:
45         chunk = sock.recv(1)
46         if not chunk:
47             raise ConnectionError("connection broken while reading line")
48         if chunk == b"\n":
49             break
50         buf.extend(chunk)
51         if len(buf) > 4096:
52             raise ValueError("too long string")
53     return buf.decode("utf-8")
54
55
56 def send_line(sock: socket.socket, text: str) -> None:
57     sock.sendall(text.encode("utf-8"))
58
59
60 def send_frame_safe(
61     sock: socket.socket,
62     text: str,
63     *,
64     master_key: bytes,
65     encrypt: bool,
66     integrity: bool,
67     test_bad: bool,
68 ) -> None:
69     body = pack_message(
70         text,
71         master_key=master_key,
72         use_encrypt=encrypt,
73         use_integrity=integrity,
74         test_bad_hash=test_bad,
75     )
76     write_frame(sock, body)
77
78
79 def broadcast_system(
80     line: str,
81     *,
82     encrypt: bool,
83     integrity: bool,
84     test_bad: bool,
85     exclude: set[str] | None = None,
86 ) -> None:
87     exclude = exclude or set()
88     with clients_lock:
89         items = [(u, clients[u], client_keys[u]) for u in clients if u not
90                 in exclude]
91     for _u, sock, key in items:
92         try:
93             send_frame_safe(sock, line, master_key=key, encrypt=encrypt,
94                             integrity=integrity, test_bad=test_bad)
95         except OSError:
96             pass

```

```

95
96
97 def send_to_users(
98     names: set[str],
99     line: str,
100     *,
101     encrypt: bool,
102     integrity: bool,
103     test_bad: bool,
104 ) -> None:
105     with clients_lock:
106         items = [(u, clients[u], client_keys[u]) for u in names if u in
107                  clients]
108         for _u, sock, key in items:
109             try:
110                 send_frame_safe(sock, line, master_key=key, encrypt=encrypt,
111                                integrity=integrity, test_bad=test_bad)
112             except OSError:
113                 pass
114
115 def handle_client(
116     conn: socket.socket,
117     addr: tuple,
118     passwd_path: Path,
119     file_master_key: bytes,
120     encrypt: bool,
121     integrity: bool,
122     test_bad: bool,
123     use_dh: bool,
124 ) -> None:
125     username: str | None = None
126     left_announced = False
127     try:
128         raw = read_line(conn)
129         try:
130             auth = json.loads(raw)
131             user = str(auth.get("user", "")).strip()
132             password = str(auth.get("pass", ""))
133         except (json.JSONDecodeError, TypeError):
134             print(f"[{addr}] auth FAIL: invalid JSON")
135             send_line(conn, "FAIL\n")
136             return
137
138         db = load_passwd(passwd_path)
139         if not user or not verify_password(user, password, db):
140             print(f"[{addr}] auth FAIL: user={user!r}")
141             send_line(conn, "FAIL\n")
142             return
143
144         with clients_lock:
145             if user in clients:
146                 print(f"[{addr}] auth BUSY: {user} already in chat")
147                 send_line(conn, "BUSY\n")
148                 return
149
150         print(f"[{addr}] auth OK: {user} logged in")
151         send_line(conn, "OK\n")
152
153         if use_dh:
154             session_key = dh_server(conn)

```

```

154     else:
155         session_key = file_master_key
156
157     with clients_lock:
158         clients[user] = conn
159         client_keys[user] = session_key
160     username = user
161
162     print(f"[{user}] joined chat")
163
164     while True:
165         body = read_frame(conn)
166         payload = parse_message(body, master_key=client_keys[user])
167         text = payload.text.strip()
168         if text == r"\q":
169             print(f"[{user}] left chat")
170             left_announced = True
171             return
172
173         line_out = _chat_line(user, payload.text)
174         with clients_lock:
175             online = set(clients.keys())
176             if _has_mentions(payload.text):
177                 targets = set(_mentioned(payload.text)) & online
178                 print(f"[{user}] -> {'', '.join(f'@{t}' for t in targets)}: {payload.text[:50]!r}")
179             else:
180                 targets = online
181                 print(f"[{user}] broadcast: {payload.text[:50]!r}")
182
183         send_to_users(targets, line_out, encrypt=encrypt, integrity=integrity, test_bad=test_bad)
184
185     except ValueError as e:
186         who = username or addr
187         err_msg = str(e)
188         print(f"[{who}] {err_msg}")
189         if username:
190             m = re.search(r"Corrupted data received: (.+)\$", err_msg)
191             corrupt_part = m.group(1)[:50] if m else err_msg[:50]
192             notice = (f"!!! Integrity FAIL from {username}: {corrupt_part}")[:76]
193             disconnect_msg = (f"{username} disconnected (integrity error)")[:76]
194             with clients_lock:
195                 all_clients = set(clients.keys())
196                 if all_clients:
197                     send_to_users(all_clients, notice, encrypt=encrypt, integrity=integrity, test_bad=False)
198                     send_to_users(all_clients, disconnect_msg, encrypt=encrypt, integrity=integrity, test_bad=False)
199     except (ConnectionError, OSError, UnicodeError) as e:
200         who = username or addr
201         print(f"[{who}] error/disconnect: {e}")
202     finally:
203         if username:
204             with clients_lock:
205                 clients.pop(username, None)
206                 client_keys.pop(username, None)
207             if not left_announced:
208                 print(f"[{username}] disconnected (no \q)")

```

```

209         try:
210             conn.shutdown(socket.SHUT_RDWR)
211         except OSError:
212             pass
213         conn.close()
214
215
216 def main() -> None:
217     p = argparse.ArgumentParser()
218     p.add_argument("--port", "-P", type=int, default=9000)
219     p.add_argument("--passwd", type=Path, default=_ROOT / "confdata" / "
        passwd")
220     p.add_argument("--key-file", type=Path, default=None)
221     p.add_argument("--encrypt", "-e", action="store_true")
222     p.add_argument("--integrity", "-i", action="store_true")
223     p.add_argument("--test-integrity", "-t", action="store_true")
224     p.add_argument("--dh", action="store_true")
225     args = p.parse_args()
226
227     if args.encrypt:
228         if args.dh and args.key_file:
229             raise SystemExit("Either --dh or --key-file")
230         if not args.dh and not args.key_file:
231             raise SystemExit("For --encrypt specify --key-file or --dh")
232     if args.key_file and args.dh:
233         raise SystemExit("Cannot use --key-file and --dh together")
234     if args.test_integrity and not args.integrity:
235         raise SystemExit("--test-integrity only with --integrity")
236
237     if args.encrypt and args.key_file:
238         file_master_key = load_master_key(args.key_file)
239     else:
240         file_master_key = b"\x00" * 32
241
242     sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
243     sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
244     sock.bind(("0.0.0.0", args.port))
245     sock.listen(50)
246     print(f"Yaaay! 0.0.0.0:{args.port}", flush=True)
247
248     try:
249         while True:
250             conn, addr = sock.accept()
251             print(f"[server] new connection from {addr}")
252             t = threading.Thread(
253                 target=handle_client,
254                 args=(conn, addr, args.passwd, file_master_key, args.encrypt
255                     , args.integrity, args.test_integrity, args.dh),
256                 daemon=True,
257             )
258             t.start()
259         except KeyboardInterrupt:
260             print("\nStop.")
261         finally:
262             sock.close()
263
264 if __name__ == "__main__":
265     main()

```


show_hash.py

```
1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import hashlib
6  import json
7  import secrets
8  import socket
9  import sys
10 from pathlib import Path
11
12 BLAKE2B_DIGEST = 32
13 SALT_SIZE = 16
14
15
16 def read_line(sock: socket.socket) -> str:
17     buf = bytearray()
18     while True:
19         chunk = sock.recv(1)
20         if not chunk:
21             raise ConnectionError("connection broken")
22         if chunk == b"\n":
23             break
24         buf.extend(chunk)
25         if len(buf) > 4096:
26             raise ValueError("too long string")
27     return buf.decode("utf-8")
28
29
30 def main() -> None:
31     p = argparse.ArgumentParser(description="Show session hash (Blake2b with
32                                     salt)")
33     p.add_argument("host", nargs="?", default="localhost")
34     p.add_argument("port", type=int, nargs="?", default=9000)
35     p.add_argument("--user", "-u", default=None)
36     p.add_argument("--password", "-p", default=None)
37     p.add_argument("--key-file", type=str, default=None)
38     p.add_argument("--dh", action="store_true")
39     args = p.parse_args()
40
41     if args.dh and args.key_file:
42         sys.exit("Cannot use --key-file and --dh together")
43
44     if args.key_file:
45         from crypto import load_master_key
46
47         master_key = load_master_key(Path(args.key_file))
48     else:
49         master_key = b"\x00" * 32
50
51     user = args.user or input("Login: ")
52     if args.password is not None:
53         password = args.password
54     else:
55         import getpass
56
57         password = getpass.getpass("Password: ")
58
59     try:
```

```

59     sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
60     sock.settimeout(5)
61     sock.connect((args.host, args.port))
62 except (socket.error, OSError) as e:
63     print("Ooops! Looks like someone forget to run server... (" , file=
64           sys.stderr)
65     sys.exit(1)
66
67 try:
68     auth = json.dumps({"user": user, "pass": password}, ensure_ascii=
69                       False)
70     sock.sendall(auth.encode("utf-8") + b"\n")
71
72     resp = read_line(sock)
73     if resp != "OK":
74         if resp == "BUSY":
75             print("BUSY: Login already in chat.", file=sys.stderr)
76         else:
77             print(resp, file=sys.stderr)
78             sys.exit(1)
79
80     if args.dh:
81         from crypto import dh_client, read_frame, write_frame
82
83         master_key = dh_client(sock)
84
85         salt = secrets.token_bytes(SALT_SIZE)
86         h = hashlib.blake2b(master_key, salt=salt, digest_size=
87                               BLAKE2B_DIGEST).digest()
88         print(f"Your hash: \{salt.hex()}\}\${h.hex()}\")
89     finally:
90         sock.close()
91
92 if __name__ == "__main__":
93     main()

```

scripts/connect.sh

```

1  #!/bin/bash
2  cd "$(dirname "$0")/.."
3  python src/client.py -e --key-file confdata/key.txt "$@"

```

scripts/start_server.sh

```

1  #!/bin/bash
2  cd "$(dirname "$0")/.."
3  python src/server.py -e --key-file confdata/key.txt

```